



BT897X 无线麦 SDK 开发指南

版本号：1.0.0

2025/8/6

目 录

目录	2
1 概述	5
1.1 SDK 功能简介	5
1.2 SDK 框架	5
1.2.1 系统层	6
1.2.2 功能模块层	6
2 开发环境的搭建	6
2.1 资料下载	6
2.2 安装编译环境	6
2.3 开发板	6
3 SDK 目录结构	7
3.1 SDK 整体目录	7
3.2 modules 目录结构	8
3.3 Project 目录结构	9
3.4 output 文件夹说明	10
3.5 下载文件介绍	11
4 程序空间说明	12
4.1 RAM 空间分配	12
4.1.1 RAM.ld 文件	13
4.1.2 RAM 的使用与查看	15
4.1.3 RAM 的复用	17
4.2 FLASH 存储空间	17
5 SDK 开发	18
5.1 调试信息	18
5.2 按键及消息	19
5.2.1 按键方式选择	19
5.2.2 按键扫描与消息处理	20

5.2.3	自定义按键的添加	23
5.3	显示模块	24
5.3.1	LED 灯效	25
5.3.2	LED 状态指示灯	26
5.3.3	LED 电源指示灯	28
5.3.4	数码管显示	29
5.4	提示音	34
5.4.1	提示音资源机制	35
5.4.1	添加用户提示音资源	37
5.5	充电及电量	37
5.5.1	充电功能简介	38
5.5.2	充电流程实现	38
5.5.3	充电参数配置	40
5.5.4	电量检测	42
5.6	外设驱动	43
5.6.1	SPI	43
5.6.2	IIC	45
5.6.3	IIS	47
5.6.4	UART	48
5.7	休眠与关机	50
5.7.1	休眠功能	51
5.7.2	关机功能	52
5.7.3	唤醒开机	54
5.8	无线麦功能	55
5.8.1	2.4G 连接	56
5.8.2	蓝牙直连	62
5.9	APP 及 OTA 相关	65
6	算法相关	71
6.1	音效算法	71
6.1.1	EQ (PACC)	72
6.1.2	DRC (PACC)	73
6.1.3	ECHO	74
6.2	通话算法	75
7	产测相关	76
8	认证相关	77

8.1 BQB 认证.....	77
8.2 FCC 认证.....	77
修订历史	79
声明	80

Bluetrum

1 概述

本文档是 BT897X 系列 SDK 的应用开发指导文档，旨在帮助用户提高对 SDK 结构的了解，以使用户能更快速的上手。

1.1 SDK 功能简介

SDK 应用主要分为领夹麦和无线话筒两种产品，涵盖了开发所需要的 2.4G 连接、手机直连、音频流算法、按键、充电以及外设等等功能的介绍

1.2 SDK 框架

SDK 模块框图

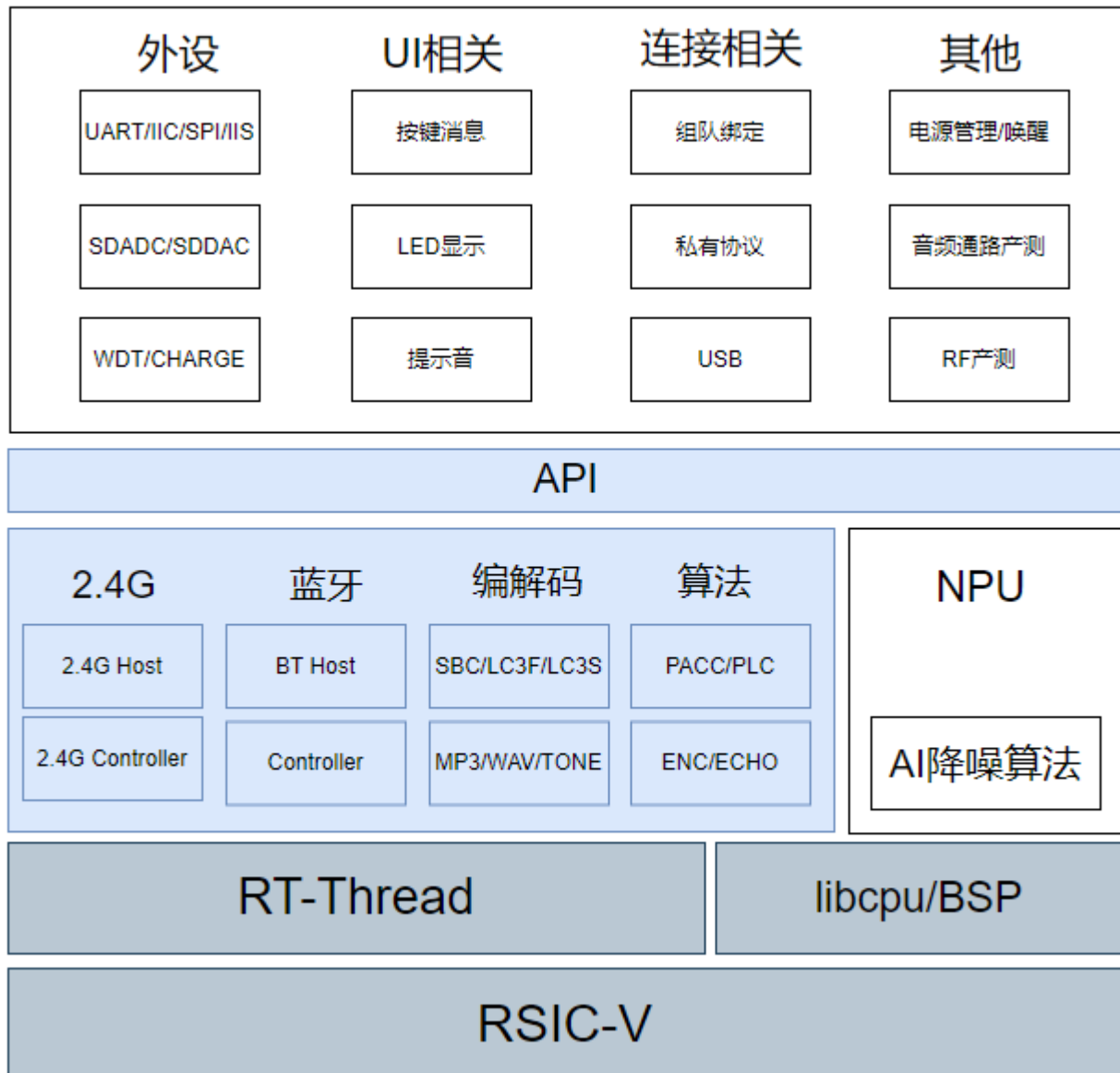


图 1.2.1 SDK 模块框图

1.2.1 系统层

BT897x 使用 RT-Thread 操作系统，负责分配系统资源、根据系统实时要求协调任务调度等

1.2.2 功能模块层

2.4G 模块：2.4G Controller 主要是链路层相关的，2.4G Host 部分主要内容是 2.4g 协议

蓝牙模块：Controller 主要是蓝牙底层相关的，Host 部分主要内容是蓝牙协议

编解码模块：主要是音频格式编解码相关，包括 LC3F/LC3S/SBC/MP3/WAV 等格式

算法模块：主要涉及语音信号处理算法，包括自研的 ENC/AI ENC/ECHO/PACC/PLC 等算法模块

NPU 模块：主要是给 AI 降噪算法做加速优化使用

2 开发环境的搭建

2.1 资料下载

关于工具的下载和使用说明，可通过关注中科蓝讯微信公众号，输入 wiki 进入蓝皮书，搜索新手指南进行获取，也可直接点击 [新手指南 \[蓝皮书\] \(bluetrum.com\)](https://bluetrum.com) 通过浏览器的方式进行获取。如想获取更多详细的文档资料，需开通中科蓝讯 wiki 蓝皮书权限，可联系中科蓝讯助理申请权限的开通。

2.2 安装编译环境

downloader 版本：downloader_v3.3.7 以上

编译器使用的是 codeblock 20.0 版本

编译链版本：rv32-toolchain_v1.5.3 版本以上，需要使用 RISC-V32-V3 编译

2.3 开发板

pcb.25-187B，详细原理图资料可参考 wiki 上无线麦无线话筒 897x 相关资料

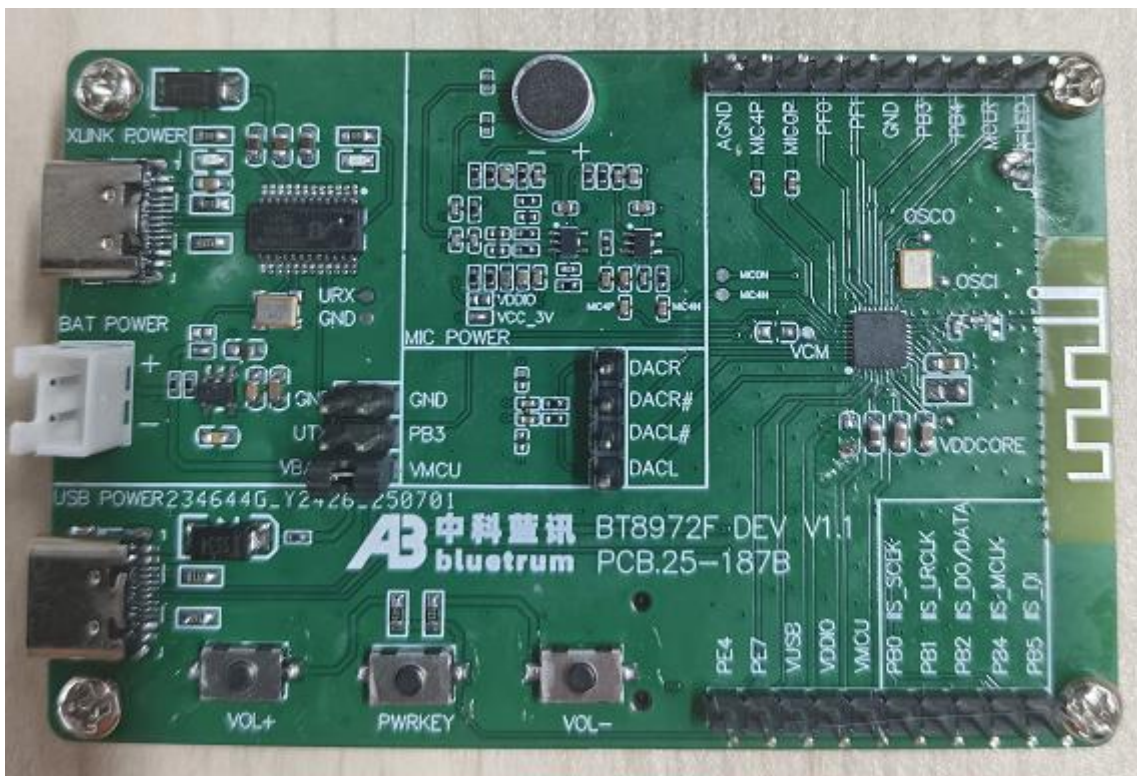


图 2.3.1 开发板实物图

3 SDK 目录结构

3.1 SDK 整体目录

介绍 SDK 文件存放情况

3rd-party	2025/7/22 21:02	文件夹
bsp	2025/7/22 21:02	文件夹
functions	2025/7/22 21:02	文件夹
include	2025/7/22 21:02	文件夹
libs	2025/7/22 21:02	文件夹
modules	2025/7/22 21:02	文件夹
os	2025/7/22 21:02	文件夹
projects	2025/7/22 21:02	文件夹
system	2025/7/22 21:02	文件夹

图 3.1.1 SDK 整体目录

3rd-party: 主要是第三方外设的驱动模块，如 sc7a20 驱动模块等等

- bsp** 存放芯片内置串口，定时器，i2c 等相关调用接口
- functions**: 存放的是各个模式的函数模块，如 func_device、func_adapter
- include**: 存放的是全局的头文件
- libs**: 存放静态库的.a 文件和.h 文件，库相关的 api 函数都在对应的.h 文件里面申明。
- modules**: 存放的是通用公共模块，下面会展开叙述
- os**: 主要是操作系统的模块，如对应的 alg 线程、dec 线程等模块
- project**: 存放显示模式，主线程消息处理模块，以及系统编译信息模块
- system**: 主要是存放公共调用的模块，如系统各个模块的初始化，记忆模块等

3.2 modules 目录结构



 audio	2025/7/23 20:54	文件夹	
 bluetooth	2025/7/23 20:54	文件夹	
 charge	2025/7/23 20:54	文件夹	
 codec	2025/7/23 20:54	文件夹	
 debug	2025/7/23 20:54	文件夹	
 effect	2025/7/23 20:54	文件夹	
 gui	2025/7/23 20:54	文件夹	
 key	2025/7/23 20:54	文件夹	
 mfi_iap2	2025/7/23 20:54	文件夹	
 periph	2025/7/23 20:54	文件夹	
 record	2025/7/23 20:54	文件夹	
 test	2025/7/23 20:54	文件夹	
 tool	2025/7/23 20:54	文件夹	
 usb_device	2025/7/23 20:54	文件夹	
 warning	2025/7/23 20:54	文件夹	
 wireless	2025/7/23 20:54	文件夹	
 modules.h	2025/7/23 15:18	H 文件	2 KB

图 3.2.1 modules 目录结构

- audio**: 存放 mic 和 dac 等相关调用接口模块
- bluetooth**: 经典蓝牙相关的通用接口模块
- charge**: 充电模块
- codec**: 编解码的基础模式配置
- debug**: 系统的 debug 调试信息模块
- effect**: 存放着音效相关的算法接口模块，如降噪/魔音等等

gui: 存放 led 灯和 ledseg7 脚屏模块
key: 存放按键相关模块
mfi_iap2: 主要是跟 lighting 头的 mfi 加密协议相关的内容
periph: 存放 vusb/PB3 产测模块
tool: 工具相关的模块
usb_device: usb 设备枚举信息, 音频接口模块
warning: 提示音播放模块
wireless: 无线麦 2.4G 连接、组包、私有命令等相关模块

3.3 Project 目录结构

下面展开 projects 文件夹下的 microphone 文件夹

display	2025/7/22 21:02	文件夹	
message	2025/7/22 21:02	文件夹	
Output	2025/7/22 21:02	文件夹	
plugin	2025/7/22 21:02	文件夹	
port	2025/7/22 21:02	文件夹	
app.cbp	2025/7/18 22:09	CBP 文件	26 KB
config.c	2025/7/16 17:47	C 文件	2 KB
config.h	2025/7/18 22:09	H 文件	47 KB
effect.c	2025/1/21 14:47	C 文件	1 KB
effect.h	2025/1/21 14:47	H 文件	1 KB
main.c	2025/2/8 13:51	C 文件	2 KB
ram.ld	2025/7/16 19:15	LD 文件	24 KB
res.h	2025/2/12 10:53	H 文件	5 KB
res2.h	2025/1/21 14:47	H 文件	5 KB
xcfg.h	2025/7/16 17:47	H 文件	17 KB

图 3.3.1 project 目录结构

message: 主线程的各个模式下的消息队列处理模块
output: 编译后输出的烧录文件 (dcf 文件) 存放在该文件夹下, 与 dcf 生成相关的文件也都存放在该文件夹下, 下面单独对该文件进行展开描述
plugin: 主要是存放音效和提示音插件的模块
port: 存放 aux/麦等相关口识别输出的模块
app.cbp: cbp 文件为工程打开的入口, 通过 codeblock 编译器打开该文件既可打开整个工程
config.h: 主要是选择对应的工程配置功能开关, 建议打开 cbp 文件后第一时间是打开该文件去选择对应的配置文件

main.c: 定义 main 函数的地方，也是程序的执行入口

ram.ld: 该文件定义了整个 SDK 里面的内存分配，芯片的 ram 资源分配情况也都该文件里面，针对让 ram 的介绍在后面的章节会展开详细描述

res.h: 该文件由编译器生成，下面那两个 res 开头的文件也一样（所以不要试图强制修改这个文件里面的内容），当 output/bin/res 里面有文件的时候，编译工程后就会更新该文件，一般在 res 文件夹里面存放的是一些提示音文件或者其他 bin 文件

xcfg.h: 配置文件对应的头文件，该文件由编译器生成（所以不要试图强制修改这个文件里面的内容），根据 output/bin/xcfg.xml 文件来生成，xcfg.xml 文件在介绍 output 文件夹再介绍

3.4 output 文件夹说明



res	2025/7/22 21:02	文件夹	
res_tone	2025/7/22 21:02	文件夹	
Settings	2025/7/22 21:02	文件夹	
app.bin	2025/7/21 11:54	FTE Binary Expo...	490 KB
app.dcf	2025/7/21 11:54	DCF 文件	626 KB
app.rv32	2025/7/21 11:54	RV32 文件	738 KB
app.xml	2025/1/21 14:47	XM 文件	1 KB
copy_tone.xml	2025/2/5 20:16	XM 文件	5 KB
download.xml	2025/1/21 14:47	XM 文件	1 KB
effect.bts	2025/1/21 14:47	BTS 文件	4 KB
effect.c	2025/1/21 14:47	C 文件	1 KB
effect.h	2025/1/21 14:47	H 文件	1 KB
header.bin	2025/1/21 14:47	FTE Binary Expo...	4 KB
map.txt	2025/7/21 11:54	文本文档	971 KB
postbuild.bat	2025/1/21 14:47	Windows 批处理...	1 KB
prebuild.bat	2025/2/1 21:28	Windows 批处理...	1 KB
res.bin	2025/2/15 16:50	FTE Binary Expo...	39 KB
res.h	2025/2/12 10:53	H 文件	5 KB
res.xml	2025/1/21 14:47	XM 文件	1 KB
unpack.bin	2025/1/21 14:47	FTE Binary Expo...	15 KB
updater.bin	2025/1/21 14:47	FTE Binary Expo...	47 KB
xcfg.bin	2025/7/16 17:47	FTE Binary Expo...	40 KB
xcfg.h	2025/7/16 17:47	H 文件	17 KB
xcfg.xml	2025/7/16 17:47	XM 文件	25 KB

图 3.4.1 output 文件夹说明

res: res 文件夹主要存放一些资源文件 (res_tone 也是如此), 比如提示音资源或者 bin 文件, 编译器编译后会生成 res.bin 文件和 res.h 文件, res.h 里面有各种资源文件的逻辑地址索引, 下面简单举例说明: 在 res 文件夹里面存放一个 user.bin 文件, 编译后生成的 res.h 有如下定义

```
#define RES_BUF_USER_BIN          (*(u32 *)0x110000b8)
#define RES_LEN_USER_BIN          (*(u32 *)0x110000bc)
```

其中 RES_BUF_USER_BIN 表示 user.bin 在程序当中的逻辑地址, 程序上可直接访问该逻辑地址来获取 user.bin 文件里面的数据, RES_LEN_USER_BIN 表示 user.bin 文件的总长度, 获取 user.bin 当中最前面的 512 个字节可以采用如下方式:

```
u8 bu[512];
memcpy(buf,RES_BUF_USER_BIN,512);
```

Settings: 该文件夹用于存放 downloader 工具配置时需要的配置文件。Setting 文件的内容由 xcfg.xml 文件决定, 可通过一般的文件编辑器或 codeblock 增删或修改 xcfg.xml 文件的配置项。xcfg.xml 文件如何修改本文档不再赘述, 具体可以参考 xcfg.xml 中已有的功能进行修改尝试。修改 xcfg.xml 文件后需重新编译整个工程, 编译器会自动更新 xcfg.bin 和 xcfg.h 文件。

app.dcf: 下载文件, 工程编译后最终的下载文件为该 dcf 文件, 可通过 downloader 工具打开并选择该文件进行下载。

app.xml: 该文件平时一般不需要修改, 主要定义了 dcf 文件的组成以及编译所需编译链的最低版本, dcf 文件的组成包含了 header.bin, app.bin, res.bin, xcfg.bin, updater.bin 其他 xml 文件类似的写法原理相同。

effect.bsln: 此文件为调音模块, 跟需要跟蓝讯工具 Toolkit 配合使用, 若没有此文件, 则需要打开 effect.bts 文件, 按照提示创建 effect.bsln, 调音详细步骤需要参考《570x 音频调试手册》

header.bin: 引导程序, 上电后芯片会运行这部分程序, 由该文件里的程序去引导跑到用户程序。

map.txt: 该文件由编译器编译后生成与更新, 程序里面定义的变量与函数的起始地址, 大小等信息都可以在该文件中查看。

3.5 下载文件介绍

拿到 sdk 后, 一般打开这个路径 (app\projects\microphone\Output\bin) 就可以拿到对应的下载文件了

下载文件常规命名为 app.dcf, 也有可能是其他名称, 认清后缀名称格式是 dcf 就行, dcf 文件格式同目录下会存放 Settings 文件夹, 里面主要存放的是 setting 后缀的配置文件, 一般会是 wireless_adapter.setting 接收端配置和 wireless_mic_emit.setting 发射端配置

下载简介: 打开 downloader 软件, 把 app.dcf 拖入 downloader 也可以按照路径打开到 app.dcf, 打开配置选择对应的配置模式

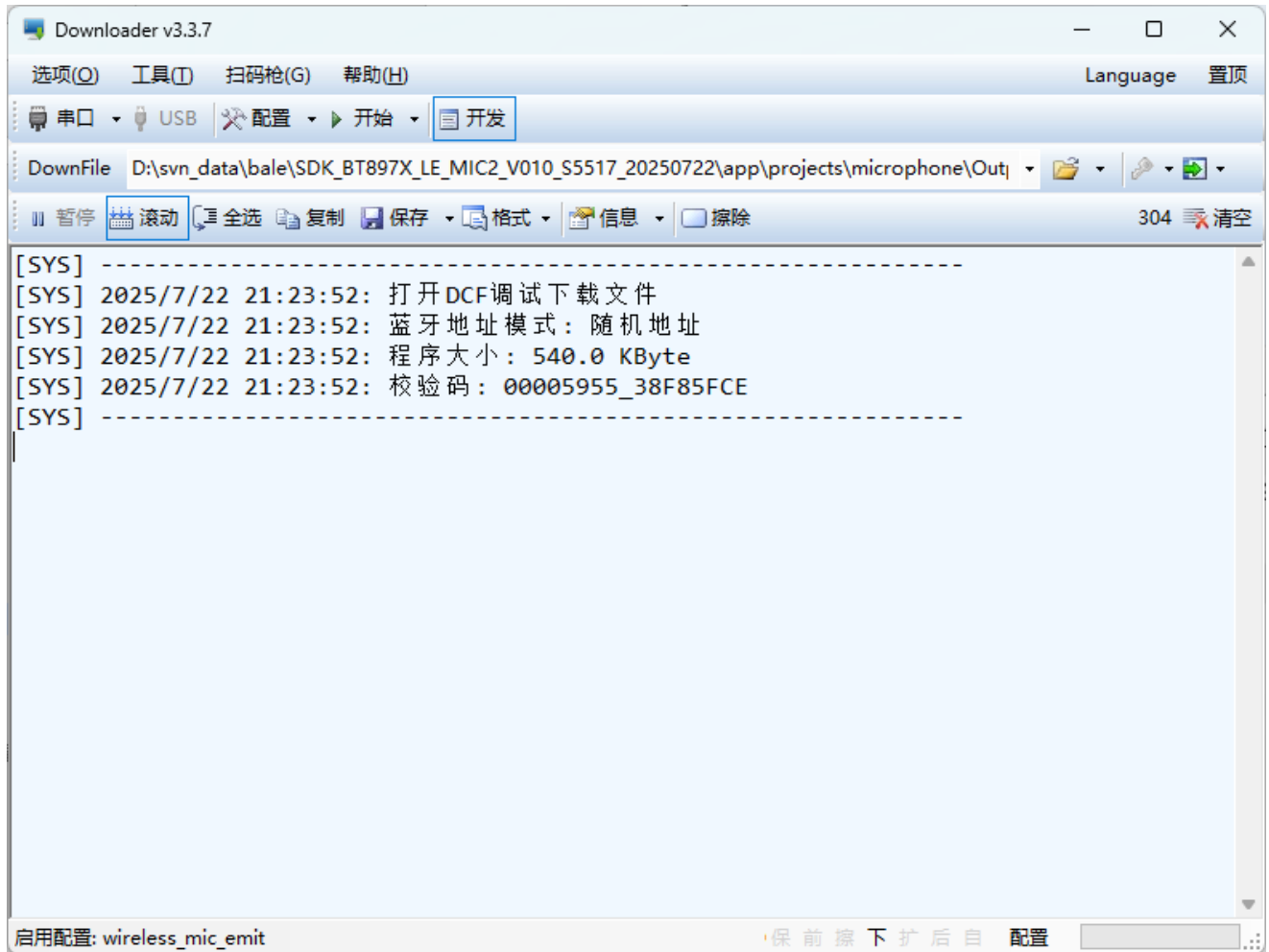


图 3.5.1 downloader 界面显示

downloader 详细说明可以参考 wiki 上《downloader 说明书》
prd 文件也可以参考 wiki 上《downloader 说明书》

4 程序空间说明

4.1 RAM 空间分配

概述 RAM 使用情况，介绍公共区和 FLASH 区概念。

BT897X 系列芯片的 ram 资源一共有 431.5k+216，其中 NPU 100k，cache 区 36k，其他加起来空间为 295.5k+216

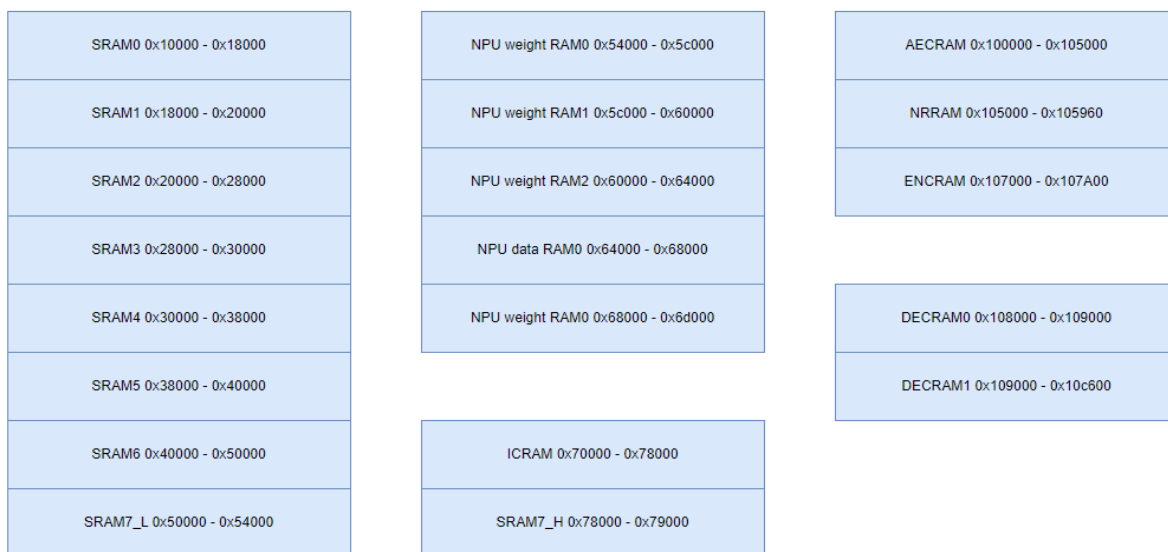


图 4.1.1 ram 资源分布情况

公共区：是指上电后加载到芯片内部 RAM，一直常驻，特点是执行速度快。可以执行需要及时响应的程序，同时也作为加载 Flash 程序时的公共区域（相当于 PC 公共中转站，防止 PC 正在执行的 RAM 被新加载的程序覆盖）。

Flash 区：是指平时存储在 Flash，需要执行时才加载到内部 Cache RAM，它的特点是地址范围大、执行不一定及时。也可以叫它 Cache 区或者 Bank 区

4.1.1 RAM.Id 文件

RAM 资源在 SDK 当做的具体分配情况在 RAM.Id 文件里面定义。

```
// Define the area size

__cache_stack_vma    = 0x10600;  //for loader
__cache_stack_size    = 512;      //for loader

__stack_vma          = 0x10800;
__stack_RAM_size      = 2k;
__comm_vma            = __stack_vma + __stack_ram_size;
__comm_ram_size       = 124k;


__cram_vma            = 0x30000;
__cram_size           = 128k;
__nram_vma            = 0x54000;    //npu
__nram_size           = 100k;
__aram_vma            = 0x100000;
__aram_size           = 20k;


__dram_vma            = 0x108000;
__dram_size           = 0x4600;
__eram_vma            = 0x107000;
__eram_size           = 0xa00;
__nrram_vma           = 0x105000;
__nrram_size          = 2400;
```

可以看到 ld 文件当中定义了一些 RAM 的起始地址以及占用的空间大小，这里定义的 RAM 都是基于 SRAM0 和 SRAM1 来做细分的。

__cache_stack_vma 定义的地址范围内的 RAM 由引导程序 (header.bin) 占用，用户不能进行修改以及使用。

__stack_vma 定义的地址范围内的 RAM 给栈使用，用于低优先级中断处理函数保存现场用，用户不能进行修改和使用。

__comm_vma 定义的地址范围内的 RAM 主要用于存放初始值为 0 的全局变量，即 bss 段的变量，当定义

的全局变量没有 AT 到指定的段且没有赋初值或者初值赋值为 0，那么这些变量会默认存放在这块 RAM 里面，上电该地址范围内的变量都会统一被清零。同时也定义的地址范围内的 RAM 主要用于存放一些执行时需要放到 RAM 的函数（目前的 Id 文件结构，函数要放到 RAM 的话只能放到这块 RAM）以及那些初始值不为 0 的变量，举个例子：

以函数为例，举例如下：

```
AT(.com_text.test)

void test_func_1(void){
    printf("test\n");
}

void test_func_2(void){
    printf("test\n");
}
```

此时，test_func_1 执行时（函数都是存放在 FLASH 里面，芯片上电后会将__comm_vma 地址范围内定义的函数从 FLASH 加载到 RAM 里面）将会在下述定义的地址范围内的 RAM 里面执行，而 test_func_2 没有指定段的位置，所以默认存放在 FLASH 里面，执行时不需要从 FLASH 读取到 cache 里面执行，所以涉及到读 FLASH 的操作，读 FLASH 需要耗时，且操作 FLASH 时功耗也会相对较高。所有在中断中运行的函数（包括子函数）都需要 AT 到 RAM 里，否则将有可能导致系统执行异常，因为中断服务子程序不允许有读取 FLASH 的操作。

__cram_vma 定义的地址范围内的 RAM 主要用于存放与 2.4G 音频传输和相关的算法模块，其中会进一步细分为发射端模块__emit_comm_vma 和接收端模块__adapter_comm_vma 以及公共模块

__ws_com_ram_vma，旨在对于发射端和接收端进行空间使用效率最大化。

__nram_vma 定义的地址范围内的 RAM 主要用于算法模块，该空间有 npu 加速指令集，能提升算法的运行效率，适合跑大模型算法

__aram_vma 定义的地址范围内的 RAM 主要被分时复用于 mp3 解码、usb，fcc 等模块

__dram_vma 定义的地址范围内的 RAM 主要被分时复用于经典蓝牙的 sco 相关模块/以及其他模式下的 i2s 之类的模块

__eram_vma 定义的地址范围内的 RAM 主要被分时复用于 fot 模块/音频格式编码模块

__nrram_vma 定义的地址范围内的 RAM 主要被用于降噪模块

4.1.2 RAM 的使用与查看

下面举例说明如何将变量或者函数放到指定的 RAM 以及如何在 map 文件中查看其对应的地址。以__comm_vma 这个基地址为例，说明如下：

```

__comm_vma          = 0x13800;

__comm_RAM_size     = 18k;

comm(rx)            : org = __comm_vma,          len = __comm_RAM_size

.comm : {

    *(.vector)

    *(.com_text*)

    *(.com_rodata*)

    *debug.o(.rodata*)

    *(.com_sleep*)

    *(.com_periph.*)

    *(.data*)

    *(.sdata*)

    . = ALIGN(512);

} > comm AT > FLASH

```

由以上定义可以看出，comm 的起始地址为 0x13800，大小为 18K，要想将变量或者函数放到这块 RAM，可以将定义的变量或者函数 AT 到 comm 包含的段，如下：

```

int a AT(.data.user);

AT(.com_text.test)

void test_func_1(void){

    printf("test\n");

}

```

变量 a 被 AT 到 .data* 数据段，而函数 test_func_1 被 AT 到 .com_text* 数据段，这两个段都被定义在 comm 中，因此编译器会将对应的变量和函数放到这个地址对应的 RAM。打开 map.txt 文件可查看具体的地址（注意，不同的工程编译出来的地址可能不一样，以实际工程为准）。

Segment	Address	File
.com_text.test	0x00000000000013a00	0xc Output\obj\project\ble_peripheral\main.o
	0x00000000000013a00	test_func_1

可以看到 test_func_1 函数存放的基地址为 0x13a00，与预期符合。

段的名称没有强制性要求，但需要注意的是一定要和 Id 文件中已定义的段名称匹配。用户也可以根据需要在 Id 文件中自行添加想要的段。

4.1.3 RAM 的复用

RAM 复用通常是在两个模块功能互斥，不可能同时被使能使用的前提下，可以考虑将这两个模块所用到的 RAM 进行复用。

4.2 FLASH 存储空间

芯片的 FLASH 存储结构大致如下（这里不考虑双备份升级或者压缩升级后的 FLASH 分布情况，双备份升级或者压缩升级后的 FLASH 分布情况请参考 FOTA 升级对应的参考文档）：



图 4.2.1 flash 存储分布图

引导程序：引导程序 1 和引导程序 2，每个引导程序都占用 4K 的 FLASH 空间，即引导程序 1 存放在 0x0000 – 0x1000 的位置，引导程序 2 存放在 0x1000 – 0x2000 的位置，固定在 FLASH 的最前面，用于引导执行用户程序，正常情况下只有引导程序 1 存在，引导程序 2 区域被擦除，即都是 0xff 的数据，只有在 FOTA 升级时才会使用到引导程序 2 做一些过渡。引导程序里面还包含了配置文件（setting 文件）生成的结构体变量的值，存放在相对引导程序基地址偏移 256 字节的位置，以第一个引导程序为例，配置文件生成的变量存放在 0x100 – 0x400 的位置。配置文件存放的区域也称之为配置区。

用户程序：存放程序。

资源区：资源区即 res 文件夹下存放的资源文件存放在 FLASH 的区域，存放的位置在用户程序之后，所以起始位置不固定，和用户程序大小有关。

空闲区：剩余空间

参数区：参数区用于存放一些掉电需要保存的数据。

SDK 中与 Flash 相关的设置主要在 config.h 文件里面，可在 config.h 文件中找到以下类似的设置项：

```

/*****
 * Module : SPIFLASH配置
 *****/
#define FLASH_SIZE          FSIZE_1M           //LQFP48芯片内置1MB，其它封装芯片内置512KB(实际导出prd文件要小于492K)
#define FLASH_CODE_SIZE     768K              //程序使用空间大小，code_size <= flash_size - [params(20k) + res2 + res.bin]
#define FLASH_RESERVE_SIZE  0x5000            //程序保留空间大小，用作系统参数区params(20k)
#define FLASH_RES2_SIZE     0x0000            //参数保留空间大小，用作在线调参区res2(打开ANC_EQ_RES2_EN需要保留4K)
#define FLASH_ERASE_4K      1                  //是否支持4K擦除
#define FLASH_DUAL_READ     1                  //是否支持2线模式
#define FLASH_QUAD_READ     0                  //是否支持4线模式
#define FLASH_SPEED_UP_EN   1                  //SPI FLASH提速。

```

图 4.2.2 flash 模块配置

软件程序实际使用大小可以根据根据工程编译后的 CODE SIZE 大小来体验，如下

```

Output\bin\postbuild.bat app
D:\svn_data\baile\SDK_BT897X_LE_MIC2_trunk_beta_85575_20250723\app\projects\microphone\Output\bin>riscv32-elf-objcopy -O binary app.rv32 app.bin || goto err
D:\svn_data\baile\SDK_BT897X_LE_MIC2_trunk_beta_85575_20250723\app\projects\microphone\Output\bin>riscv32-elf-xmaker -b appxm.o || goto err
CODE SIZE: 544 KB
save file "app.dcf" successful

```

图 4.2.3 软件程序大小

可以看到编译后的 CODE SIZE 大小为 544KByte，这里的 CODE SIZE 主要由一个引导程序、用户程序、资源文件、以及一些校验信息和头部信息组成

SDK 中对芯片内置 flash 的操作接口在 bsp_eq.h 文件里面，主要有读，写，擦除三个接口，如下：

```

uint os_spiflash_read(void *buf, u32 addr, uint len);
void os_spiflash_program(void *buf, u32 addr, uint len);
void os_spiflash_erase(u32 addr);

```

其中，addr 为 flash 实际的物理地址，如对 flash 4K 位置擦除，addr 为 0x1000。这里需要注意的是，写操作不能跨 page 写，flash 的一个 page 一般为 256 字节，假设调用 os_spiflash_program 接口进行写操作，addr 设置为 100，则此时 len 不能大于 (256 - 100)，否则将出现跨 page 写入的情况，超过 page 的数据将无法成功被写入。读接口没有此限制。擦除接口按 sector 大小进行擦除，flash 的 sector 大小一般为 4K。

5 SDK 开发

5.1 调试信息

常规来说，SDK 基本用 uart0 作为串口调试打印口，打开对应的 config.h 文件找到 UART0_PRINTF_SEL 这个宏，修改对应的 IO，为了方便下载调试，我们一般会找既支持下载又支持打印的 io 去做打印，vusb 口和 PB3 口就可以满足这个要求，选择打印口时要注意引脚复用的情况

```

/*****
 * Module : 系统功能选择配置
 *****/
#define SYS_CLK_SEL          SYS_24M           //选择系统时钟
#define POWKEY_10S_RESET    xcfcg_cb.powkey_10s_reset
#define SOFT_POWER_ON_OFF   1                  //是否使用软开关机功能
#define PWRKEY_2_HW_PWRON   0                  //用PWRKEY模拟硬开关
#define USB_SD_UPDATE_EN    0                  //是否支持UDISK/SD的离线升级
#define GUI_SELECT          GUI_NO            //GUI Display Select
#define UART0_PRINTF_SEL    PRINTF_VUSB       //选择UART打印信息输出IO，或关闭打印信息输出
#define PWRON_ENTER_BTMODE_EN 0                //是否上电默认进蓝牙模式
#define SLEEP_DAC_OFF_EN    (is_sleep_dac_off_enable()) //stfunc_sleep是否关闭DAC，复用MICL检测方案不能关DAC。
#define SYS_VDDIO_LP_EN     1                  //休眠模式是否打开切换VDDIO功能（省电，可能会影响VDDIO供电的外设）

#define OFFLINE_LOG_EN      0                  //是否打开离线日志（打开之前要评估flash空间是否足够）
#define OFFLINE_LOG_SIZE    0x30000           //离线日志存储区域大小
#define OFFLINE_LOG_START    (FLASH_SIZE - CM_SIZE - OFFLINE_LOG_SIZE) //离线日志存储区域起始位置（注意避让FOTA、自定义KEEP等其他功能）

```

图 5.1.1 打印接口配置

```

/*****
* Module : uart0 printf IO列表
*****/
#define PRINTF_NONE 0 //关闭UART0打印信息
#define PRINTF_PA7 1 //与SDOMAP_G1同一IO
#define PRINTF_PB2 2 //与SDOMAP_G2同一IO
#define PRINTF_PB3 3 //与USBDF, SDOMAP_G5, SDOMAP_G6同一IO
#define PRINTF_PE7 4
#define PRINTF_PE0 5
#define PRINTF_VUSB 6

```

图 5.1.2 打印 io 选择

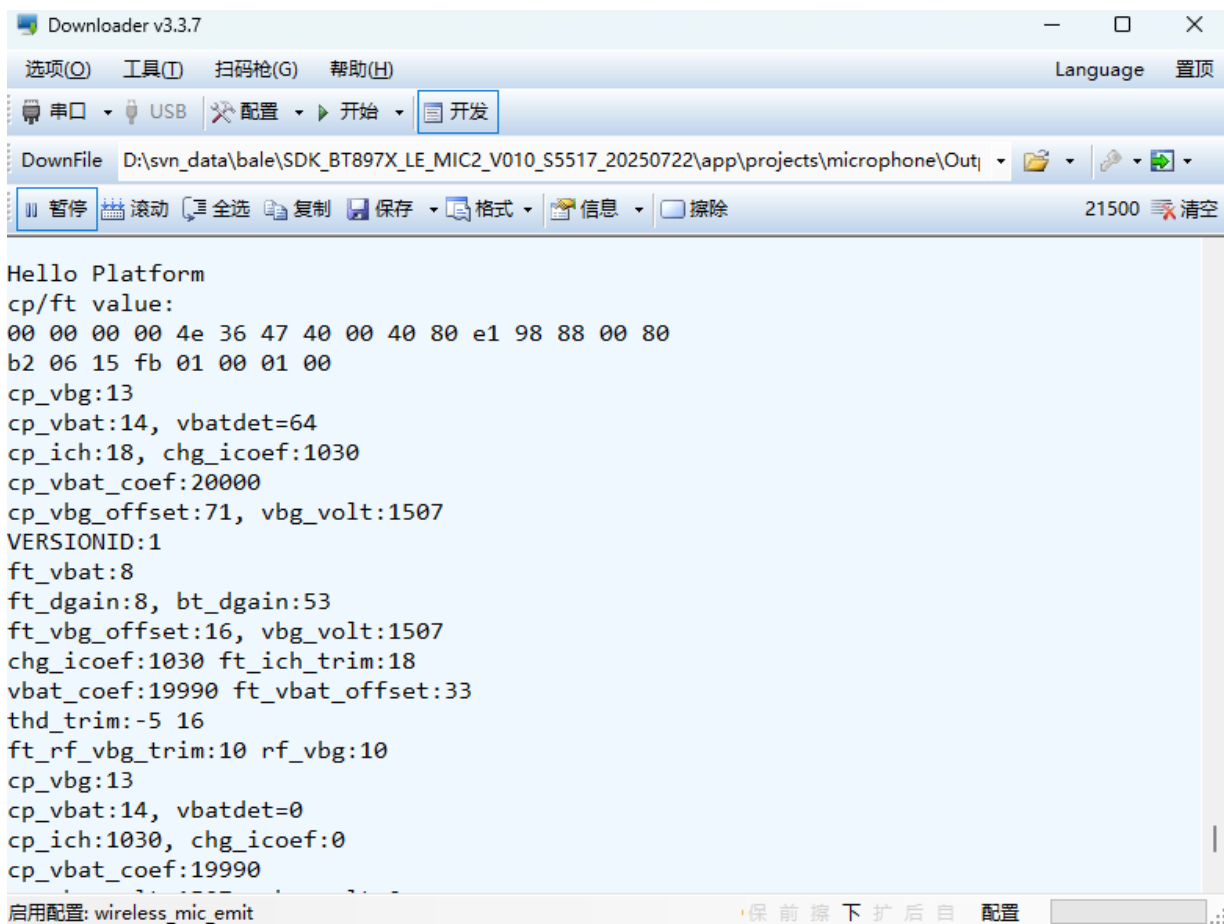


图 5.1.3 打印信息

5.2 按键及消息

5.2.1 按键方式选择

BT897x 无线麦方案支持 pwrkey、touch key、adkey 以及 io key 四种按键触发方式，支持按键短按、长按、保持、双击以及多击的按键操作。四种按键方式的工作方式虽不一样，但是最终按键消息的抛出流程都相同。可在 config.h 文件中选择按键触发方式。

```

config.h key_scan.c port_key.c key_scan.h plugin.c
391 #define KEY_MAX_NB 3 //按键数量（不包括IR）
392 #define DOUBLE_KEY_TIME (xcfg_cb.double_key_time) //按键双击响应时间（单位50ms）
393 #define PWRON_PRESS_TIME (500*xcfg_cb.pwron_press_time) //长按PWRKEY多长时间开机
394 #define PWROFF_PRESS_TIME xcfg_cb.pwroff_press_time //长按PWRKEY多长时间关机
395
396 //pwrkey
397 #define PWRKEY_EN 1 //PWRKEY的使用，0为不使用
398 #define PWRKEY_IS_PRESS() pwrkey_is_pressed()
399
400 //touch key
401 #define TKEY_EN 0 //TouchKEY的使用，0为不使用
402 #define TKEY_SOFT_PWR_EN 0 //是否使用TouchKey进行软开机
403 #define TKEY_LOWPWR_WAKEUP_DIS 0 //是否电池低电时关掉触摸唤醒，无保护板的电池需要打开。
404 #define TKEY_INEAR_EN 0 //是否使用TouchKey的入耳检测功能
405 #define TKEY_TEMP_EN 0 //是否使用TouchKey的温度检测功能
406 #define TKEY_DEBUG_EN 0 //仅调试使用，用于确认TKEY的参数
407 #define TKEY_IS_PRESS() (tkey_is_pressed() && TKEY_SOFT_PWR_EN)
408
409 //adkey
410 #define ADKEY_EN 0 //ADKEY的使用，0为不使用
411 #define ADKEY_CH ADCCH_PE7
412 #define ADKEY_MUX_SDCLK_EN 0 //是否使用复用SDCLK的ADKEY，共用USER_ADKEY的按键table
413 #define ADKEY_MUX_LED_EN 0 //是否使用ADKEY与LED复用，共用USER_ADKEY的流程（ADKEY与BLED配置同一IO）
414 #define ADKEY_PU10K_EN 1 //ADKEY是否使用内部10K上拉，按键数量及阻值见port_key.c
415 //adkey2
416 #define ADKEY2_EN 0 //ADKEY2的使用，0为不使用
417 #define ADKEY2_CH ADCCH_PE6
418
419 //io key
420 #define IOKEY_EN 0 //IOKEY的使用，0为不使用
421 #define IOKEY_GPIO_SEL_EN 0 //IOKEY操作方式选择，0=直接操作寄存器，1=通过GPIO_SEL0/1选择
422 #define IOKEY_GPIO_SEL0 IO_PFO
423 #define IOKEY_GPIO_SEL1 IO_PFI
424 #define IOKEY_MID01_EN 0

```

图 5.2.1 按键方式的选择宏

5.2.2 按键扫描与消息处理

以 Pwrkey 做示例，介绍按键的操作流程。Pwrkey 和 ADkey 都是通过采样 IO 口电压值，区分不同按键。按键未按下时，由于有内部上拉 IO，pwrkey 默认电平为高电平，当有按键按下，IO 电平会被拉低，电平发生变化且大于自定义阈值，则判断为有按键按下。大致流程如下图所示。

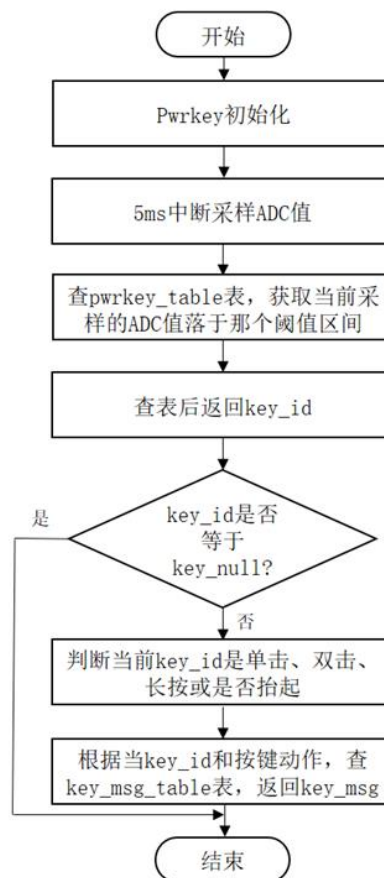


图 5.2.2 按键扫描流程图

1、Pwrkey 按键初始化，需初始化 IO 上内部上拉，配置唤醒通道和 ADC 采样通道。

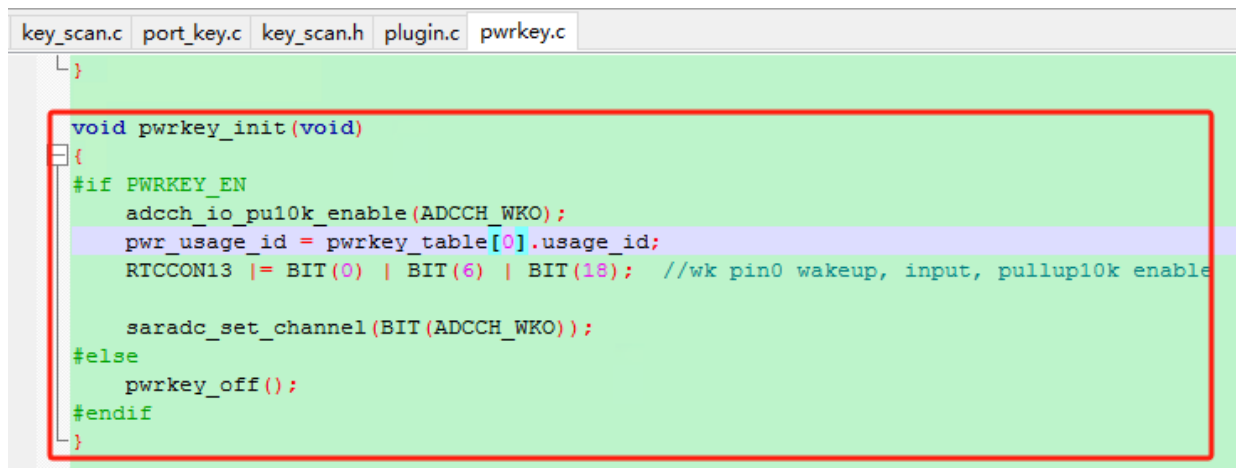


图 5.2.3 pwrkey 按键初始化

2、ADC 的采样、按键 id 判断、按键消抖和按下状态（单击、长按、保持）判断，在 bsp_key_scan() 中调用接口去完成。

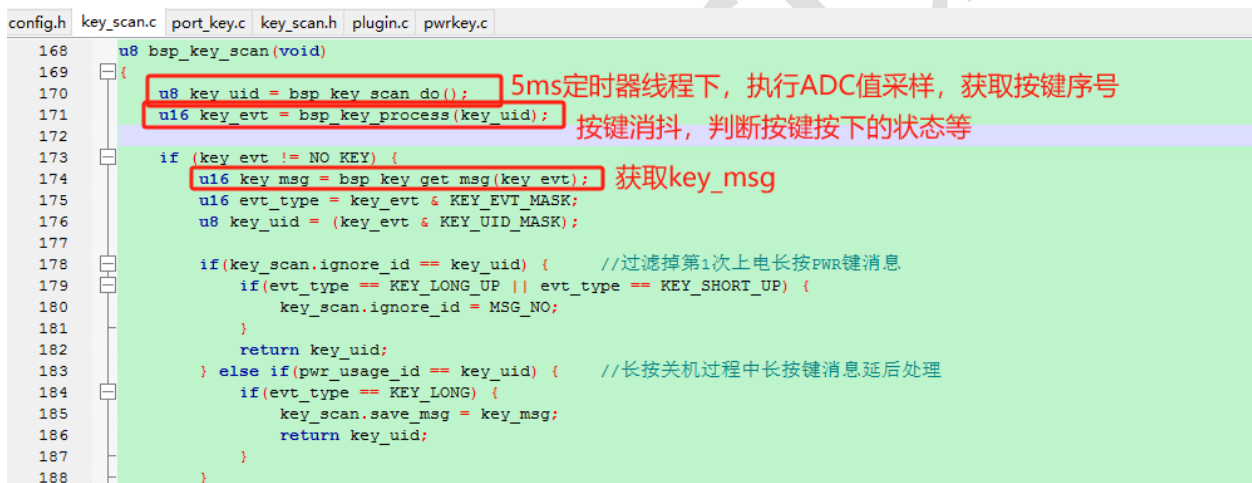


图 5.2.4 adc 值处理以及消息获取

其中，根据 ADC 采样值，判断按键序号通过查表完成，如下图所示是 Pwrkey 的键值表。如采样 ADC 值为 0xB0，ADC 值落在阈值 0x0A~0xAF 之间，则 key_2 按下。

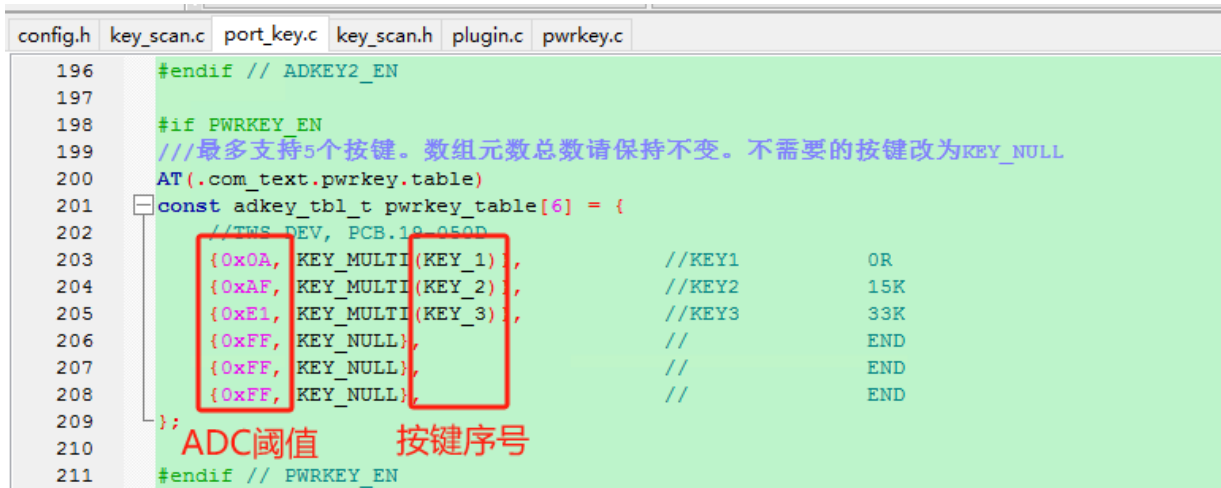


图 5.2.5 pwrkey 的键与值表

按键号关联到按键消息，可在按键消息列表中有定义，在本 SDK 中区分了不用模式下的按键消息，如图所示是 BT music 和 device 模式下的按键消息列表。



图 5.2.6 按键状态与按键消息的关联

按键的消息定义在 key_scan.h 中有枚举类型的定义。



图 5.2.7 按键消息的定义

3、每一次完成按键的检测，都会抛出 msg 消息，转到主线程中处理消息。

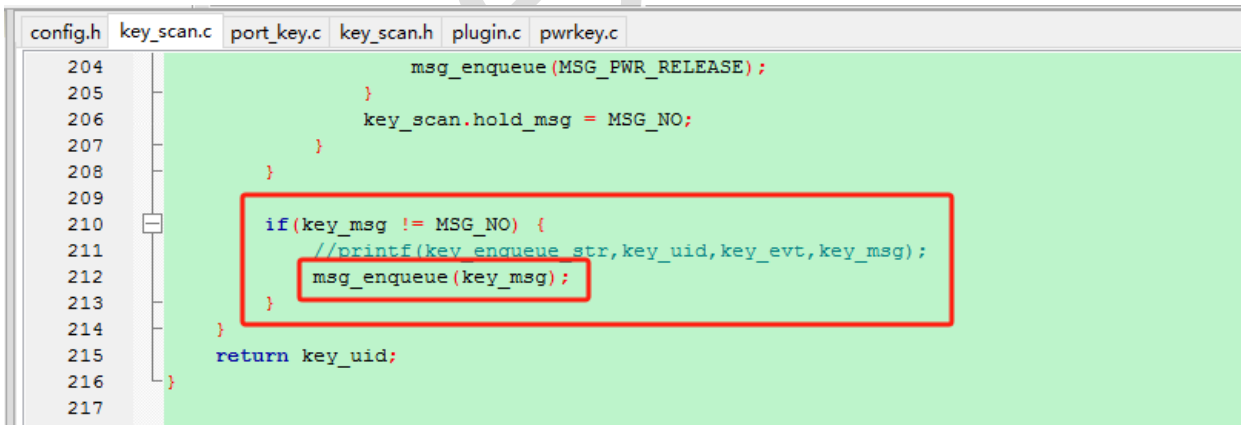


图 5.2.8 按键消息的抛出

5.2.3 自定义按键的添加

当需要添加自己的按键消息时，可以通过以下步骤完成。

例如，需要 device 模式下，key_1 的双击添加为“切换模式”，添加 key_4，单击为“切换灯效”，双击为“关闭灯效”。

1、在 key_scan.h 文件中，添加按键消息定义；

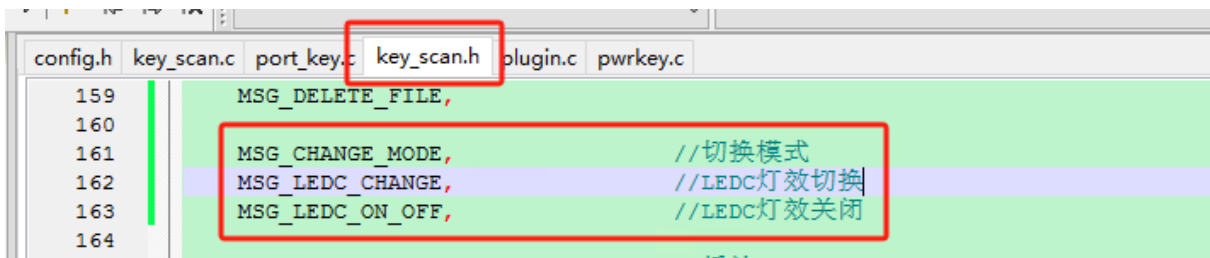


图 5.2.9 用户按键消息的定义

2、port_key.c 文件中，在对应的模式中，将按键消息添加进去；

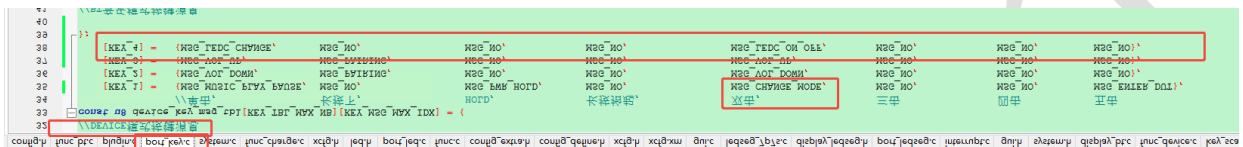


图 5.2.10 用户按键消息与按键状态的关联

添加 key_4 按键，修改 ad 的阈值，添加 key_4 的可触发方式。这里的 AD 阈值需要注意，设定阈值的时候需要先用 saradc_get_value8(ADCCH_WKO)获取按键按下的 ADC 值，再去设定阈值。比如图 5.2.10 中，设定的几个阈值是{0x0A, 0x50, 0x78, 0x95, 0xFF}, 0xFF 代表引脚电压被拉高，无按键按下，在使用 saradc_get_value8(ADCCH_WKO)获取 key_1~key_4 的 ADC 值过程中，获取到的是{0x00, 0x20, 0x68, 0x80}，由此可见，程序里设定任意两个 AD 阈值，是设定在将某个按键的实际 ADC 值之间的。

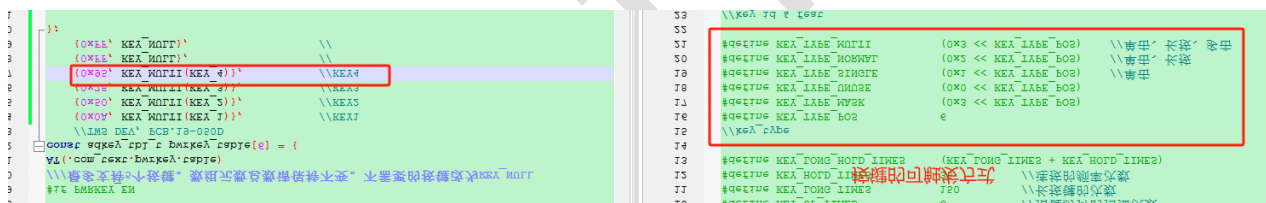


图 5.2.11 新添用户按键操作

3、在对应模式的 msg.c 文件中添加消息的处理（将按键消息处理放到 func.c 中也可以）。

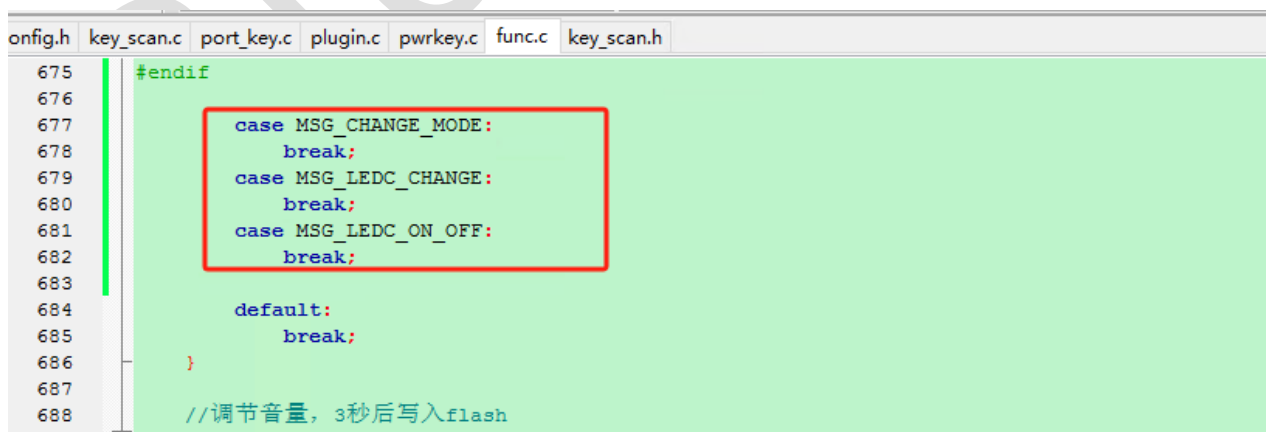


图 5.2.12 用户按键消息的处理

5.3 显示模块

5.3.1 LED 灯效

默认 SDK 自带一个 LED 状态指示灯(蓝灯)和电源指示灯(红灯)的 LED 灯效，默认有开机初始化灯效、配对灯效、回连灯效、连接状态灯效、蓝牙工作状态灯效、低电灯效、满电灯效以及充电灯效。相关宏的控制以及灯的控制 SDK 中有如图 5.3.1 的定义。

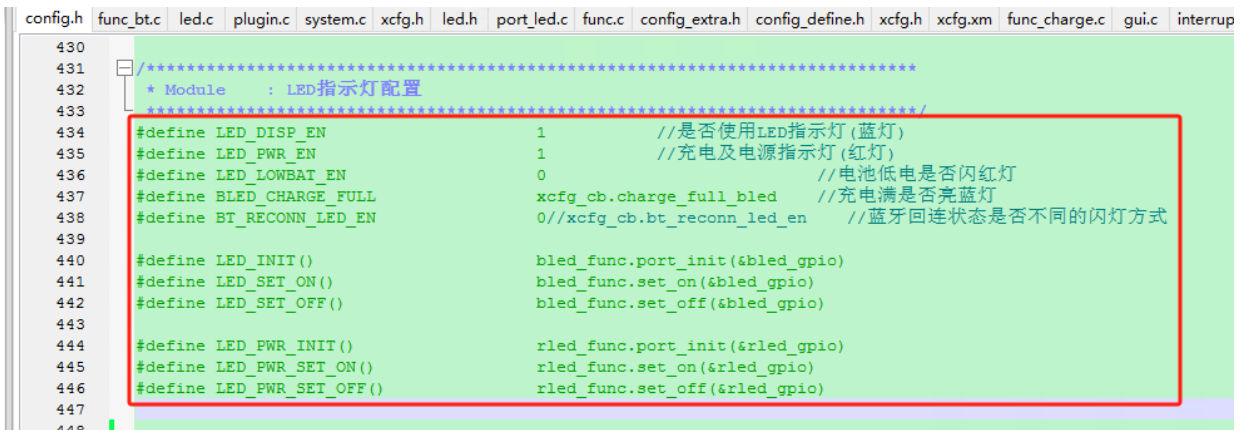


图 5.3.1 LED 灯效控制宏

SDK 自带的 LED 灯效与配置文件具有高度关联，可以在配置文件中自定义灯效和引脚。



图 5.3.2 LED 灯效配置文件

自定义LED闪灯状态	☒
开机红灯	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
开机蓝灯	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
开机闪灯单位(50ms)	10 (0 ~ 255)
开机闪灯间隔(50ms)	255 (0 ~ 255)
关机红灯	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
关机蓝灯	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
关机闪灯单位(50ms)	4 (0 ~ 255)
关机闪灯间隔(50ms)	255 (0 ~ 255)
初始化红灯	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
初始化蓝灯	☐ ☒ ☐ ☒ ☐ ☒ ☐ ☒
初始化闪灯单位(50ms)	10 (0 ~ 255)
初始化闪灯间隔(50ms)	0 (0 ~ 255)
回连红灯	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
回连蓝灯	☐ ☒ ☐ ☒ ☐ ☒ ☐ ☒
回连闪灯单位(50ms)	6 (0 ~ 255)

图 5.3.3 LED 自定义灯效配置

5.3.2 LED 状态指示灯效

关于配置文件中的 UI 显示和闪灯时间的控制，下面以配对灯效为例演示。如图中配对灯效的 UI 有 8 个显示位可以选择，这里选择了①③⑤⑦作为点亮，②④⑥⑧作为熄灭，闪灯单位为 $2 \times 50 = 100\text{ms}$ ，闪灯间隔为 $0 \times 50 = 0\text{ms}$ ，这里的闪灯单位是①~⑧两两之间的间隔时间，闪灯间隔为灯效跑到了⑧，完成一个周期了，要重新回到①时需要等待的间隔时间。如图是配对灯均匀闪烁时的电平翻转状态。

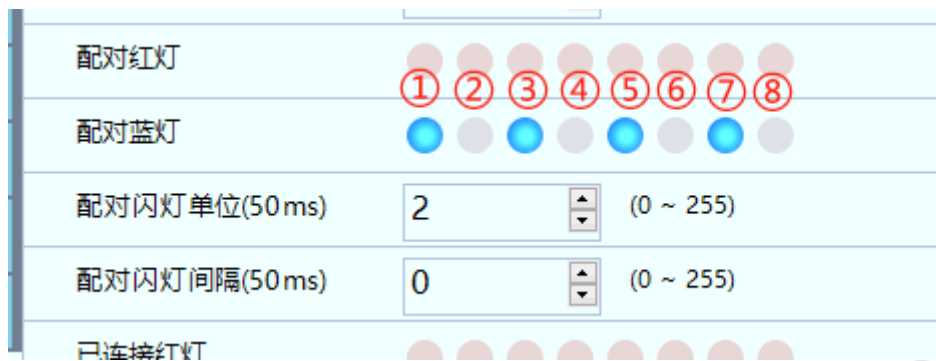


图 5.3.4 LED 蓝灯配置

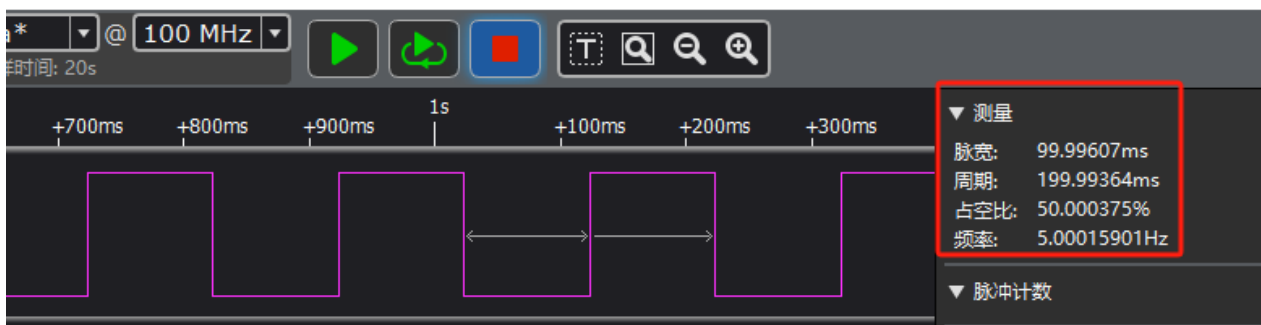


图 5.3.5 LED 蓝灯电平变化

若需要配置快闪带停顿的灯效，可以有如下的配置。



图 5.3.6 LED 蓝灯配置与电平变化

其中，灯效的刷新是 1ms 刷新一次，代码是在 1ms 软中断中执行，如图所示。

```

76 //timer tick interrupt(5ms)
77 AT(.com_text.timer)
78 void usr_tmrlms_isr(void)
79 {
80     #if (GUI_SELECT & DISPLAY_LEDSEG) //7F屏按COM扫描时, 1ms间隔
81         gui_scan();
82     #endif
83     #if LED_DISP_EN
84         port_led_scan();
85     #endif // LED_DISP_EN
86     plugin_tmrlms_isr();
87     led_scan();
88 }
89 //timer tick interrupt(5ms)
90 AT(.com_text.timer)
91 void usr_tmrlms_thread(void)
92 {
93     uint8_t ret;
94     tmrlms_cnt++;
95 }
96
97 void led_cfg_port_init(gpio_t *g);
98 void led_cfg_set_onoff(gpio_t *g, bool onoff);
99
100 AT(.com_text.led_disp)
101 void led_scan(void)
102 {
103     led_cb_t *s;
104     u32 tickn = TICKN_GET();
105     if (!is_led_scan_enable()) {
106         return;
107     }
108     s = (led_cb_lowbat)? &led_lb : &led_cb;
109     if (s->unit == 0) {
110         if (RLED_LOWBAT_FOLLOW_EN) {
111             if (led_cb_lowbat) {
112                 led_cb_lowbat = false;
113             }
114         }
115         return;
116     }
117     u32 tickdiff = TICK SUB(tickn, s->tick inst);

```

5.3.7 LED 灯效扫描代码

5.3.3 LED 电源指示灯效

电源指示灯主要是和充电有关，设计灯效是低电时闪灯、充电时亮灯以及满电时蓝灯亮。默认 SDK 在打开电源指示灯之后，便默认配置了进入充电时灯亮，执行代码如图 5.3.8。

```

43 }
44
45 AT(.text.charge.led)
46 void charge_status_disp_cb(u8 sta)
47 {
48     if (sta == 1) {
49         //充电开启
50         sys_cb.charge_sta = 1;
51         sys_cb.charge_bled_flag = 0;
52         charge_led_on();
53         TRACE(charge_on_str);
54     } else {
55         //充电关闭
56         TRACE(charge_off_str, sta);
57         sys_cb.charge_sta = 0;
58         sys_cb.charge_bled_flag = 0;
59         charge_led_off();
60         if (sta == 2) {
61             sys_cb.charge_sta = 2;
62             //充电满亮蓝灯
63             if (BLED_CHARGE_FULL) {
64                 sys_cb.charge_bled_flag = 1;
65                 sys_cb.ch_bled_cnt = ch_bled_timeout_tbl[BLED_CHARGE_FULL];
66                 charge_bled_on();
67             }
68         }
69     }
70 }
71

```

5.3.8 充电灯效处理代码

低电时闪灯的效果，可以在配置文件中自行定义，如图 5.3.9。

低电红灯		
低电蓝灯		
低电闪灯单位(50ms)	6	(0 ~ 255)
低电闪灯间隔(50ms)	0	(0 ~ 255)

5.3.9 低电灯效配置

充满电时的蓝灯的效果，可以在配置文件中有选择，如图

充电配置	边充电边工作使能	☐
蓝灯配置	充电电压跟随模式	☐
无线配置	充电截止电流	30mA
RF参数	恒流充电电流	110mA
MIC参数	涓流充电电流	30mA
算法参数	充满电蓝灯亮	亮30秒
按键配置	充电截止计时	30min
	充满电自动关机	☐

5.3.10 充电满电配置

5.3.4 数码管显示

默认 SDK 自带多种引脚形式的数码管显示屏，包括 5C7S 数码管、7PIN 数码管、3PIN 7 段数码管和 6C6S 6 段数码管，可在 config.h 文件中自行选用相应的数码管。

```

config.h  func_bt.c  plugin.c  system.c  func_charge.c  xcfg.h  led.h  port_led.c  func.c  config_extra.h  xcfg.h  xcfg.xml
28  * Module : 系统功能选择配置
29  *****
30  #define SYS_CLK_SEL      SYS_24M      //选择系统时钟
31  #define POWKEY_10S_RESET  xcfg_cb.powkey_10s_reset //是否使用软开关机
32  #define SOFT_POWER_ON_OFF 1          //用 powkey 模拟硬件
33  #define POWKEY_2_HW_PWRON 0          //是否支持 UDISK/SD
34  #define USS_SD_UPDATE_EN  0          //GUI Display Selec
35  #define GUI_SELECT       GUI_NO      //选择 Uart 打印信息
36  #define UART0_PRINTR_SED  PRINTR_FPS //是否上电默认显示
37  #define PWRON_ENTER_RPMODE_EN 0      //是否上电默认显示
38  #define SLEEP_DAC_OFF_EN  (is_sleep_dac_off_enable()) //休眠模式是否打开
39  #define SYS_VDDIO_LP_EN   1          //休眠模式是否打开

13  #define DISPLAY_NO      0          //无显示模块
14  #define DISPLAY_LCDSEG  0x200     //选用断码屏做为显示
15  #define DISPLAY_LEDSEG  0x400     //选用数码管做为显示
16
17  #define GUI_NO          DISPLAY_NO //无主题，无显示
18  #define GUI_LCDSEG      (DISPLAY_LCDSEG | 0x00) //断码屏默认主题
19  #define GUI_LEDSEG_5C7S (DISPLAY_LEDSEG | 0x00) //5C7S 数码管
20  #define GUI_LEDSEG_7F7S (DISPLAY_LEDSEG | 0x01) //7F7S 数码管
21  #define GUI_LEDSEG_3P7S (DISPLAY_LEDSEG | 0x02) //3PIN 7段数码管
22  #define GUI_LEDSEG_6C6S (DISPLAY_LEDSEG | 0x04) //6C6S 6段数码管
23
24  *****

```

5.3.11 数码管显示控制宏

下面以最常用的 7PIN 数码管为例，展开介绍数码管的使用。

5.3.4.1 7PIN 数码管初始化

1、定义数码管引脚

```

config.h  config_kbox.h  port_ledseg.c  keytbl_pcb19_050.c  keytbl_k12_5707e.c  api_sys.h  sfunc_adapter.c  mic_first.c  config_define.h  ledseg_7p7s.c  func.c  ledseg_7p7s.h  gui.c  display_ledseg.h
4
5  //-----
6  #define LED7P7S_ANY_IO 1 //Support Any IO drv ledseg7p7s
7
8  #define LEDSEG_ALL_DIRIN() {GPIOADE |=0xC8; GPIOADIR |= 0xC8; GPIOFDE |=0x06; GPIOFDIR |= 0x06;GPIOEDE |=0xF0; GPIOEDIR |= 0xF0;}
9  #define LEDSEG_HDRV_EN() {GPIOADE |=0xC8; GPIOADRV &= ~0xC8; GPIOFDE |=0x06; GPIOFDRV &= ~0x06;GPIOEDE |=0xF0; GPIOEDRV &= ~0xF0;}
10 #define LEDSEG_HDRV_DIS() {GPIOADRV &= ~0xC8; GPIOFDRV &= ~0x06;GPIOEDRV &= ~0xF0;}
11 #define LEDSEG_ALL_CLR() {LEDSEG_ALL_DIRIN(); LEDSEG_HDRV_EN();} //DirIn, Open High Drv.
12 #define LEDSEG_ALL_OFF() {LEDSEG_ALL_DIRIN(); LEDSEG_HDRV_DIS();} //DirIn, Close High Drv.
13
14 //note, must first IO "SET", then set "DIR". Otherwise have some unwanted pulse
15 #define LEDSEG5_H() {GPIOSET = BIT(6); GPIOEDIR &= ~BIT(6);}
16 #define LEDSEG5_L() {GPIOCLR = BIT(6); GPIOEDIR &= ~BIT(6);}
17
18 #define LEDSEG4_H() {GPIOSET = BIT(7); GPIOEDIR &= ~BIT(7);}
19 #define LEDSEG4_L() {GPIOCLR = BIT(7); GPIOEDIR &= ~BIT(7);}
20
21 #define LEDSEG3_H() {GPIOSET = BIT(1); GPIOFDIR &= ~BIT(1);}
22 #define LEDSEG3_L() {GPIOCLR = BIT(1); GPIOFDIR &= ~BIT(1);}
23
24 #define LEDSEG2_H() {GPIOSET = BIT(2); GPIOFDIR &= ~BIT(2);}
25 #define LEDSEG2_L() {GPIOCLR = BIT(2); GPIOFDIR &= ~BIT(2);}
26
27 #define LEDSEG1_H() {GPIOSET = BIT(3); GPIOADIR &= ~BIT(3);}
28 #define LEDSEG1_L() {GPIOCLR = BIT(3); GPIOADIR &= ~BIT(3);}
29
30 #define LEDSEG0_H() {GPIOSET = BIT(7); GPIOADIR &= ~BIT(7);}
31 #define LEDSEG0_L() {GPIOCLR = BIT(7); GPIOADIR &= ~BIT(7);}
32
33 #define LEDSEG6_H() {GPIOSET = BIT(6); GPIOADIR &= ~BIT(6);}
34 #define LEDSEG6_L() {GPIOCLR = BIT(6); GPIOADIR &= ~BIT(6);}

```

这里0xC8的二进制位1100 1000
即bit[7]、bit[6]、bit[3]，对应着下面定义的PA7、PA6和PA3

5.3.12 数码管引脚定义

2、根据实际的 pin 和标准的 7 段数码管，进行编码

```

194
195 //标准7段编码如下:
196 // |--A--|
197 //  F      B
198 // |--G--|
199 //  E      C
200 // |--D--|
201
202 AT(.text.ledseg)
203 void ledseg_7p7s_update_disbuf(void)
204 {
205     u8 dis_buf[7];
206     memset(dis_buf, 0, 7);
207
208     if (ledseg_buf[0] & SEG_A) dis_buf[0] |= PIN1;
209     if (ledseg_buf[0] & SEG_B) dis_buf[0] |= PIN2;
210     if (ledseg_buf[0] & SEG_C) dis_buf[3] |= PIN0;
211     if (ledseg_buf[0] & SEG_D) dis_buf[4] |= PIN0;
212     if (ledseg_buf[0] & SEG_E) dis_buf[0] |= PIN3;
213     if (ledseg_buf[0] & SEG_F) dis_buf[1] |= PIN0;
214     if (ledseg_buf[0] & SEG_G) dis_buf[2] |= PIN0;
215

```

5.3.13 数码管 PIN 脚编码

3、初始化缓存和定时器

开启 7PIN 数码管功能，会开启一个 timer1 用来定时 1ms 清除屏幕显示。

```

1 #include "include.h"
2
3
4 AT(.com_text.timer)
5 void timer_isr(void)
6 {
7     if (TMRICON & BIT(9)) {
8         TMRICPMD = BIT(9); //Clear Pending
9         TMRICON = 0;
10        #if (GUI_SELECT == GUI_LEDSEG_7P7S) //close display
11            ledseg_7p7s_clr();
12        #elif (GUI_SELECT == GUI_LEDSEG_6C6S)
13            ledseg_6c6s_clr();
14        #endif
15    }
16 }

```

5.3.4.2 数码扫描刷新

数码管的显示刷新扫描是在 1ms 软中断中进行，当显示的内容发生了改变，就需要在刷新扫描这里将显示刷新掉。

```

80 //timer tick interrupt (1ms)
81 AT(.com_text.timer)
82 void timer_isr(void)
83 {
84     #if (GUI_SELECT & DISPLAY_LEDSEG) //7S 屏按COM
85         gui_scan();
86     #endif
87
88     #if LED_DISP_EN
89         port_led_scan();
90     #endif // LED_DISP_EN
91
92     #if FUNC_FMRX_EN
93         fmrx_timer_isr();
94     #endif
95
96     plugin_timer_isr();
97
98     led_scan();
99
100 //timer tick interrupt (5ms)
101 AT(.com_text.timer)
102 void timer_isr_5ms(void)
103 {
104     event : 1; //BOX event标志
105     res : 0; //保留bits
106     gui_flasher_t flicker; //显示闪烁控制
107
108     extern gui_box_t box_cb;
109
110     #if (GUI_SELECT & DISPLAY_LEDSEG)
111         #include "ledseg/ledseg_common.h"
112         #include "ledseg/display_ledseg.h"
113
114         #define gui_scan() ledseg_scan()
115         #define gui_display(n) ledseg_display(n)
116         #define gui_off() ledseg_off()
117
118         #define gui_scan()
119         #define gui_display(n)
120         #define gui_off()
121     #endif
122
123     #if (GUI_SELECT == GUI_NO)
124         #define gui_init()
125         #define gui_box_isr()
126         #define gui_box_process()
127         #define gui_box_display()
128
129         DISP_REC_PLAYTIME,
130         DISP_REC_PAUSE,
131         DISP_SFDF,
132         DISP_FMRX,
133         DISP_SPEAKER,
134         DISP_PLAYREV,
135         DISP_IDLE,
136         DISP_OFF,
137     };
138
139     #if (GUI_SELECT == GUI_LEDSEG_7P7S)
140         #include "ledseg/ledseg_7p7s.h"
141         #define ledseg_init() ledseg_7p7s_init()
142         #define ledseg_off() ledseg_7p7s_off()
143         #define ledseg_scan() ledseg_7p7s_scan()
144         #define ledseg_update_dispbuff() ledseg_7p7s_update_dispbuff()
145     #else
146         #define ledseg_init()
147         #define ledseg_off()
148         #define ledseg_scan()
149         #define ledseg_update_dispbuff()
150     #endif
151
152     #endif
153 }

```

5.3.4.3 数码显示

数码的显示以 BT 模式下，显示“BT”字符为例演示定义和代码跳转的流程。

1、数码表的定义

```

1 #ifndef LEDSEG_COMMON_H
2 #define LEDSEG_COMMON_H
3
4 //ASCII Display Definition
5 #define T_LEDSEG_0 (SEG_A | SEG_B | SEG_C | SEG_D | SEG_E | SEG_F)
6 #define T_LEDSEG_1 (SEG_B | SEG_C)
7 #define T_LEDSEG_2 (SEG_A | SEG_B | SEG_G | SEG_E | SEG_D)
8 #define T_LEDSEG_3 (SEG_A | SEG_B | SEG_G | SEG_C | SEG_D)
9 #define T_LEDSEG_4 (SEG_B | SEG_C | SEG_F | SEG_G)
10 #define T_LEDSEG_5 (SEG_A | SEG_C | SEG_D | SEG_F | SEG_G)
11 #define T_LEDSEG_6 (SEG_A | SEG_C | SEG_D | SEG_E | SEG_F | SEG_G)
12 #define T_LEDSEG_7 (SEG_A | SEG_B | SEG_C)
13 #define T_LEDSEG_8 (SEG_A | SEG_B | SEG_C | SEG_D | SEG_E | SEG_F | SEG_G)
14 #define T_LEDSEG_9 (SEG_A | SEG_B | SEG_C | SEG_D | SEG_F | SEG_G)
15 #define T_LEDSEG_A (SEG_A | SEG_B | SEG_C | SEG_E | SEG_F | SEG_G)
16 #define T_LEDSEG_B (SEG_C | SEG_D | SEG_E | SEG_F | SEG_G)
17 #define T_LEDSEG_C (SEG_A | SEG_D | SEG_E | SEG_F)
18 #define T_LEDSEG_D (SEG_B | SEG_C | SEG_D | SEG_E | SEG_G)

```

2、字符显示函数定义

定义一个显示“BT”字符的函数。其中 ledseg_buf[1]和 ledseg_buf[2]表示在数码管的第 1 位和第 2 位显示。


```

158 }
159
160 AT(.text.display.ledseg)
161 void ledseg_disp_bt(void)
162 {
163     if (is_mute_flicker()) {
164         return;
165     }
166
167     #if FUNC_BTHID_EN
168     if (is_bthid_mode()) {
169         ledseg_buf[1] = T_LEDSEG_H;
170         ledseg_buf[2] = T_LEDSEG_I;
171         ledseg_buf[3] = T_LEDSEG_D;
172     } else
173     #endif // FUNC_BTHID_EN
174     {
175         ledseg_buf[1] = T_LEDSEG_B;
176         ledseg_buf[2] = T_LEDSEG_T;
177     }
178 }
179

```

在 ledseg_disp_pfunc 函数指针表中，将显示函数添加进去

```

1 AT(.text.display.ledseg.table)
2 const PFUNC ledseg_disp_pfunc[] = {
3     ledseg_disp_welcome,
4     ledseg_disp_volume,
5     ledseg_disp_playtime,
6     ledseg_disp_pause,
7     ledseg_disp_eq,
8     ledseg_disp_playmode,
9     ledseg_disp_music_num,
10    ledseg_disp_freq,
11    ledseg_disp_freq_ch,
12    ledseg_disp_aux,
13    ledseg_disp_bt,
14    ledseg_disp_usbpc,
15    ledseg_disp_rtctime,
16    ledseg_disp_time_set,
17    ledseg_disp_rectime,
18    ledseg_disp_rec_playtime,
19    ledseg_disp_rec_pause,
20    ledseg_disp_spdif,
21    ledseg_disp_fmam,
22    ledseg_disp_speaker,
23    ledseg_disp_playdev,
24    ledseg_disp_idle,
25    ledseg_disp_off,
26 };
27 #endif

```

在数码显示的枚举类型表的尾部，添加 DISP_BT 变量。

```

20 #define ICON_FM          SEG_0
21 #define ICON_AUX        0x00
22
23 enum {
24     DISP_POWERON = 0,
25     DISP_VOL,
26     DISP_MUSIC_PLAY,
27     DISP_MUSIC_PAUSE,
28     DISP_EQ,
29     DISP_MODEPLAY,
30     DISP_NUM,
31     DISP_FREQ,
32     DISP_FQ_CHAN,
33     DISP_AUX,
34     DISP_BT,
35     DISP_USBPC,
36     DISP_RTCTIME,
37     DISP_TIME_SET,
38     DISP_REC_TIME,
39     DISP_REC_PLAYTIME,
40     DISP_REC_PAUSE,
41     DISP_SPDIF,
42     DISP_FMAM,
43     DISP_SPEAKER,
44     DISP_PLAYDEV,
45     DISP_IDLE,
46     DISP_OFF,
47 };
48
  
```

3、BT 模式下的字符显示

```

17 break;
18 }
19
20 AT(.text.func.bt.display)
21 void func_bt_enter_display(void)
22 {
23     gui_display(DISP_BT);
24 }
25
26 AT(.text.func.bt.display)
27 void func_bt_exit_display(void)
28 {
29 }
30 #endif
31
32
33
34
35
36
37 gui_box_t;
38 extern gui_box_t box_cb;
39
40 #if (GUI_SELECT & DISPLAY_LEDSEG)
41 #include "ledseg/ledseg_common.h"
42 #include "ledseg/display_ledseg.h"
43
44 #define gui_scan()          ledseg_scan()
45 #define gui_display(n)      ledseg_display(n)
46 #define gui_off()          ledseg_off()
47
48 #else
49 #define gui_scan()
50 #define gui_display(n) ②关联
51 #define gui_off()
52 #endif
53
54 #if (GUI_SELECT == GUI_NO)
55 #define gui_init()
56
57 AT(.text.ledseg)
58 void ledseg_display(u8 disp_num)
59 {
60     u8 buf_bak[5];
61     void (*pfunc)(void);
62     memcpy(buf_bak, ledseg_buf, 5);
63     memmove(ledseg_buf, sizeof(ledseg_buf));
64     pfunc = ledseg_disp_pfunc[disp_num];
65     (*pfunc)();
66     if (memcmp(buf_bak, ledseg_buf, 5) == 0) {
67         return;
68     }
69     ledseg_disp_num = disp_num;
70     ledseg_update_dispbuff();
71 }
72 #endif
73
74
75
  
```

①根据枚举的序号显示内容

②关联

③根据需要查找显示相应的内容

4、字符刷新

ledseg_display()接口执行，查表执行相应的显示函数后，全局变量 ledseg_buf[] 内容刷新，然后调用 ledseg_update_dispbuff()->ledseg_update_dispbuff()->

ledseg_7p7s_update_dispbuff_do 接口，根据 ledseg_buf[] 内容去刷新 dis_buf[]，即 pin 脚将要推的电平。将 dis_buf[] 内容给到 ledseg_cb.buf，最后在 1ms 软中断中根据 ledseg_cb.buf 刷新显示 pin 脚电平。

```

239
240     if (ledseg_buf[4] & ICON_PLAY)    dis_buf[0] |= PIN5;
241     if (ledseg_buf[4] & ICON_PAUSE)   dis_buf[2] |= PIN5;
242     if (ledseg_buf[4] & ICON_USB)     dis_buf[5] |= PIN1;
243     if (ledseg_buf[4] & ICON_SD)      dis_buf[0] |= PIN4;
244     if (ledseg_buf[4] & ICON_DDOT)    dis_buf[2] |= PIN3;
245     if (ledseg_buf[4] & ICON_FM)      dis_buf[6] |= PIN2;
246     if (ledseg_buf[4] & ICON_MP3)    dis_buf[2] |= PIN6;
247
248     ledseg_7p7s_update_dispbuf_do(dis_buf);
249

```

刷新pin脚的显示

```

AT(.text.ledseg)
void ledseg_7p7s_update_dispbuf_do(u8 *disp_buf)
{
    ledseg_cb.disp_en = 0;
    memcpy(ledseg_cb.buf, disp_buf, sizeof(ledseg_cb.buf));
    ledseg_cb.disp_en = 1;
}

```

赋值

```

40 //按COM扫描, 1ms扫描间隔
41 AT(.com_text.ledseg)
42 void ledseg_7p7s_scan(void)
43 {
44     u8 disp_seg;
45     u8 com_cnt;
46
47     if (ledseg_cb.disp_en == 0) {
48         return;
49     }
50
51     #if KNOB_MUX_LEDSEG_EN
52     if (ledseg_7p7s_reuse_hook()) { //ledseg的复用处理
53         return;
54     }
55     #endif // KNOB_MUX_LEDSEG_EN
56
57     ledseg_7p7s_scan_hook(ledseg_cb.buf);
58     disp_seg = ledseg_cb.buf[ledseg_cb.com_cnt];
59     com_cnt = ledseg_cb.com_cnt;
60     ledseg_cb.com_cnt++;
61     if (ledseg_cb.com_cnt > 6) {
62         ledseg_cb.com_cnt = 0;
63     }
64
65     ledseg_ajust(disp_seg); //可以控制灯的亮暗程度
66     ledseg_7p7s_set(disp_seg, com_cnt); //刷新数码管的显示
67 }
68
69 #endif // #if (GUI_SELECT == GUI_LEDSEG_7P7S)
70

```

5.4 提示音

默认 SDK 支持 TONE、WAV 和 MP3 提示音，提示音的播放接口统一为“bsp_res_play()”，输入参数是提示音的序号。

5.4.1 提示音资源机制

提示音是统一通过提示音序号访问的，具体调用方法如图 5.4.1。

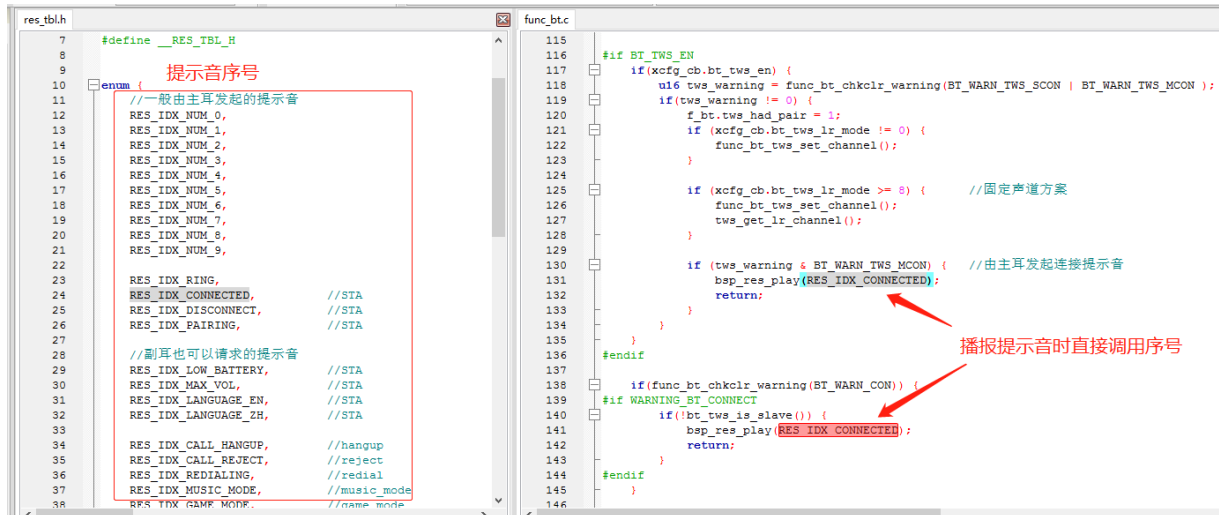


图 5.4.1 提示音序号与播放示例

在 multi_lang.c 中，每个序号会对应访问提示音资源的一项（注意这里有两个表，对应中、英文提示音）。

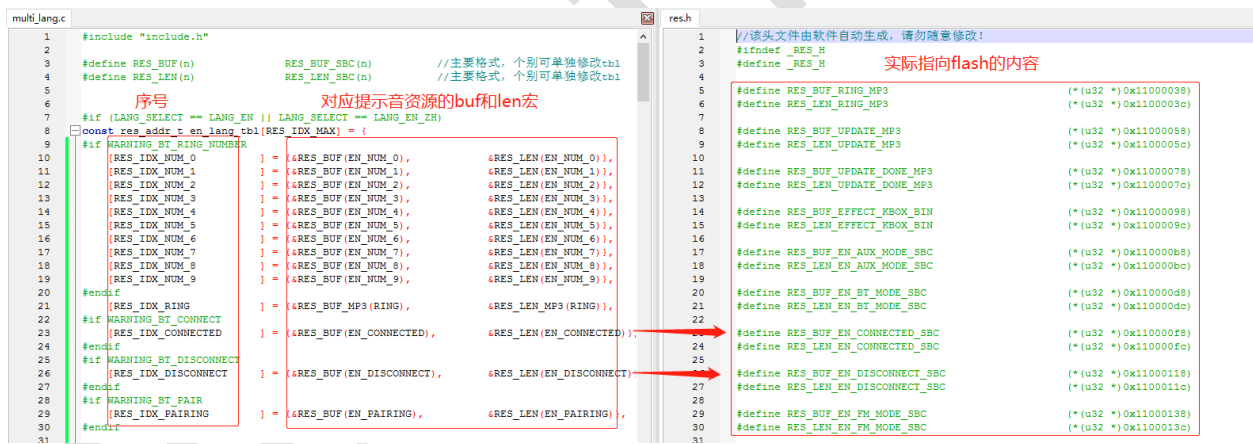


图 5.4.2 提示音定义与资源示例

为了节省 flash 控制，SDK 通过 config.h 的宏，控制一个脚本，决定那些提示音参与编译产生提示音资源（res.h 和 res.bin）。

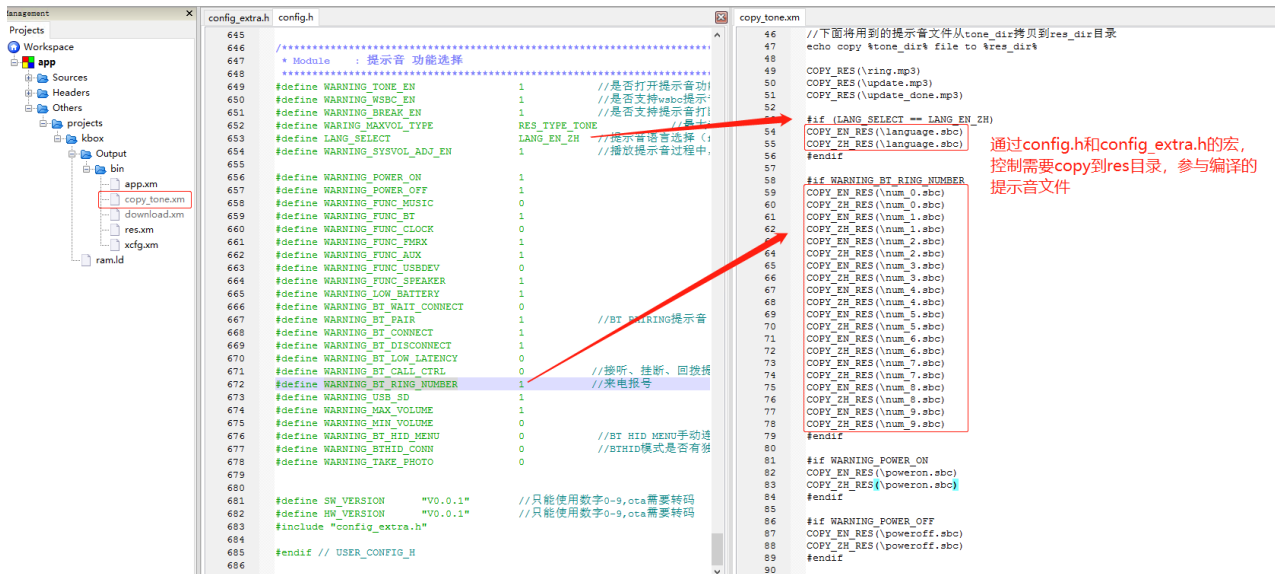


图 5.4.3 提示音脚本控制示例

提示音资源在编译过程中的流程如图 5.4.4.

名称	修改日期	类型	大小
res	2024/9/14 16:11	文件夹	
res_tone	2024/9/14 14:15	文件夹	
Settings	2024/9/14 14:31	文件夹	
app.bin	2024/9/14 16:11	BIN 文件	401 KB
app.dcf	2024/9/14 16:11	DCF 文件	558 KB
app.rv32	2024/9/14 16:11	RV32 文件	650 KB
app.xml	2024/9/6 13:57	XM 文件	1 KB
appxm.o	2024/9/14 16:11	O 文件	1 KB
copy_tone.xml	2024/9/14 16:01	XM 文件	5 KB
download.xml	2024/5/22 11:09	XM 文件	1 KB
header.bin	2024/7/25 15:55	BIN 文件	4 KB
kbox.c	2024/9/6 13:57	C Source	3 KB
kbox.cfg	2024/9/12 16:39	Configuration 源...	5 KB
kbox.h	2024/9/5 14:55	C/C++ Header	3 KB
map.txt	2024/9/14 16:11	文本文档	862 KB
postbuild.bat	2024/9/12 16:39	Windows 批处理...	1 KB
prebuild.bat	2024/9/14 11:08	Windows 批处理...	1 KB
res.bin	2024/9/14 16:11	BIN 文件	54 KB
res.h	2024/9/14 16:11	C/C++ Header	11 KB
res.xml	2024/9/2 11:04	XM 文件	1 KB
unpack.bin	2024/5/22 11:09	BIN 文件	15 KB
updater.bin	2024/9/9 11:40	BIN 文件	46 KB
xcfg.bin	2024/9/14 16:11	BIN 文件	45 KB
xcfg.h	2024/9/14 16:11	C/C++ Header	19 KB
xcfg.xml	2024/9/14 14:31	XM 文件	28 KB

1. 提示音资源仓库 (res_tone目录)

2. 编译时，取出需要的提示音
copy到res目录

3. copy res的控制脚本

4. 编译生成的res.bin和res.h
(即程序中调用的RES_BUF和RES_LEN)

图 5.4.4 提示音编译生成文件示例

5.4.1 添加用户提示音资源

下面以添加一个 mp3 提示音为例，介绍添加 mp3 提示音的流程。

1、首先，在 res 和 res_tone 文件夹中添加提示音

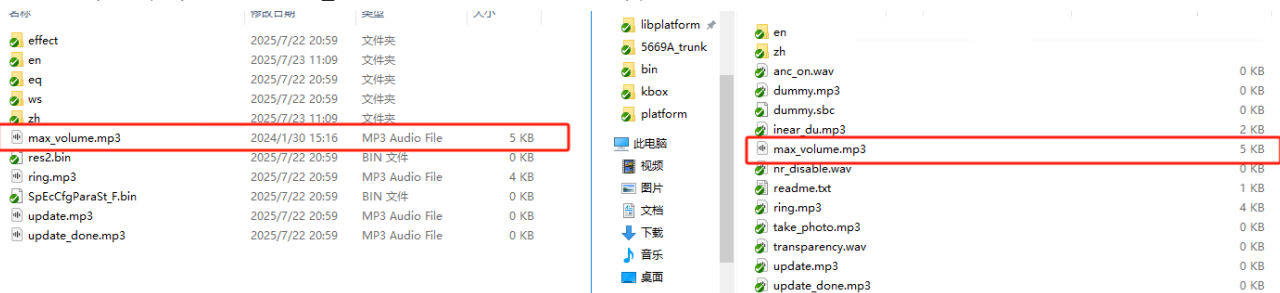


图 5.4.5 用户提示音文件添加

2、代码中将提示音资源关联



图 5.4.6 用户提示音添加流程示例

3、添加完提示音资源后，即可在程序中调用来播放提示音

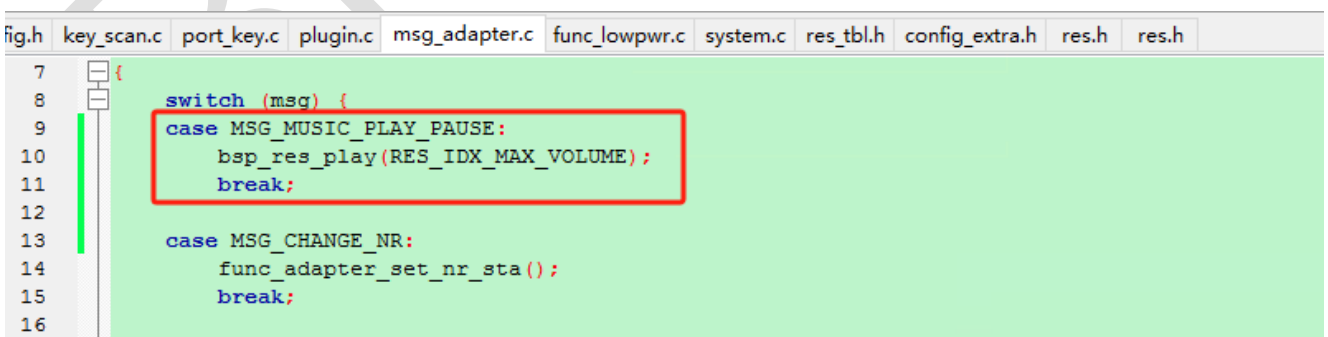


图 5.4.7 用户提示音播放示例

5.5 充电及电量

5.5.1 充电功能简介

芯片内部自带充电管理模块，通过向 VUSB 端插入电压（要求大于电池电压 0.2v 以上，一般为 5V 插入），向 Vbat 所接的锂电池进行充电。该充电模块支持插入 DC 复位系统、充断电检测、涓流电流配置、恒流电流配置、截止电压/电压配置、以及 LDO 配置选择等

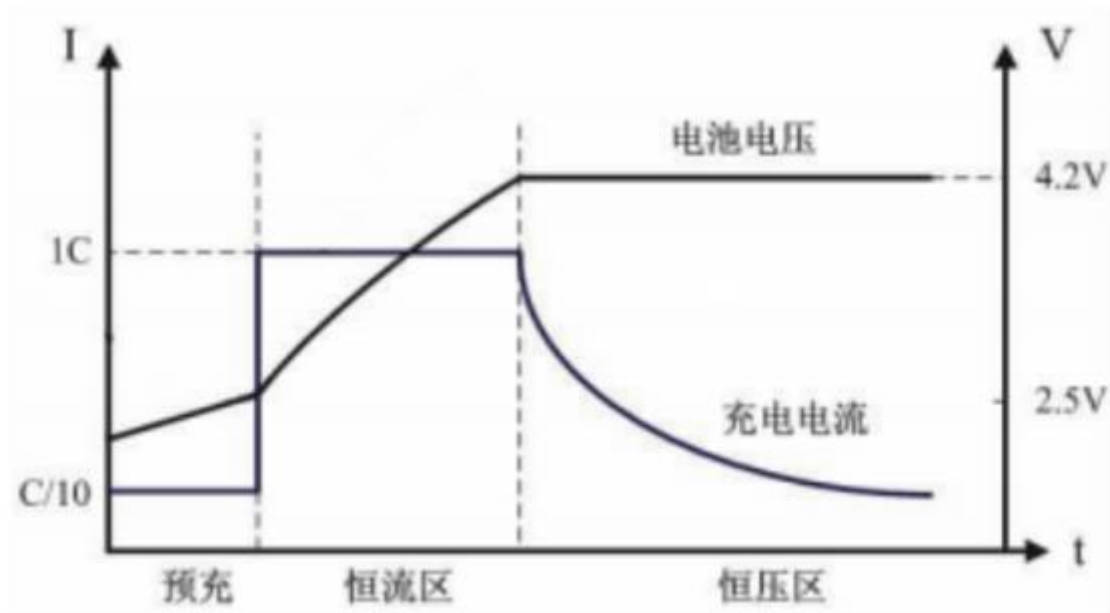


图 5.5.1.1 充电电流电压变化图

充电开始过程中，检测电池电压是否大于涓流截止电压（2.9V/3.0V），选择进入涓流或者恒流充阶段，当电池电压到达截止电压时，自动切换到恒压充阶段，充电电流逐渐减小，当电流小于截止电流时候，判定为充满，停止充电

5.5.2 充电流程实现

充电相关的流程主要集中在 charge.c/charge.c/func_charge.c 文件中，充电过程可以体验在插入检测（charge_process_p），充电处理（charge_dc_insert），拔出/充满检测处理（sfunc_charge_process）三个阶段，根据检测结果切换状态机（charge_ctl.chag_sta）

插入检测：CHARGE_DC_IN()用来判定是否有 5V 插入，当 vusb 有 5v 接入的时候，会根据软件配置是充电不开机模式还是边充电边工作模式跑不同的模式，但是充电流程是一样的

充电处理：主要是根据当前状态去切换设置充电电流，充电灯效等，并持续做充电结束检测；


```

AT(.text.charge.proc)
void charge_dc_insert(void)
{
    if (charge_ctl.chag_sta == CHAG_STA_W4_DCIN || charge_ctl.chag_sta == CHAG_STA_OFF) { //5v插入，但是还没开始充电
        if (!charge_ctl.trick_curr_en || (PWRCON1 & BIT(29))) { //判断电压是否达到2.9/3
            charge_start(&charge_ctl, 1);
        } else {
            charge_start(&charge_ctl, 0); //涓流充电
        }
        charge_status_disp_cb(1);
    } else { //已经开始充电
        if (charge_ctl.chag_sta == CHAG_STA_ON_TRICKLE) { //当前是涓流充电
            if (PWRCON1 & BIT(29)) { //判断电压是否达到2.9/3
                charge_ctl.trick_curr_cnt++;
            } else {
                charge_ctl.trick_curr_cnt = 0;
            }
            if (charge_ctl.trick_curr_cnt > 10) {
                TRACE("charge switch: tri(u8_struct205_charge_ctl_t::trick_curr_cnt)");
                charge_start(&charge_ctl, 1); //切换到恒流充电
                charge_status_disp_cb(1);
            }
        } else { //充电结束检测
            if ((charge_ctl.chag_sta == CHAG_STA_ON_CONSTANT) && charge_off_detect(&charge_ctl)) {
                charge_stop(&charge_ctl, 1);
                charge_status_disp_cb(2);
            }
        }
    }
}

```

图 5.5.2.1 充电处理模块

拔出/充电检测处理：芯片一般是根据截止电压和截止电流来判定是否充满，主流程在 charge_box_charge_on_process 中，通过 CHARGE_DC_IN() 和 CHARGE_INBOX()来判定无线麦的充电状态（出仓，仓内掉电压，仓内充满等等），根据状态去做相应的处理

```

#ifdef CHARGE_EN
    if ((CHARGE_DC_NOT_PWRON) && xcfg_cb.charge_en) {
        charge_sta = sfunc_charge_process(counter++);
        if(charge_sta == CHAG_STA_OFF) {
#ifdef CHARGE_BOX_EN
            if(xcfg_cb.ch_out_auto_pwrn_en) { //出充电仓，拿起开机
                pwrn_flag = true;
                break;
            }
#endif
        }
    }
#endif

```

图 5.5.2.2 充电状态切换

```

//5ms调用一次
AT(.text.charge_box)
u8 charge_box_charge_on_process(void)
{
    u8 inbox_sta, chbox_sta = 0;

    #if (CHARGE_BOX_TYPE == CBOX_SSW)
        chbox_sta = charge_box_ssw_process(1);
    #endif

    if ((charge_box_sta() == 2) || (chbox_sta == 2)) { //是否充满
        return CHAG_STA_OFF_VBUS_PATH;
        // sfunc_charge_exit();
        // charge_box_inbox_wakeup_enable(); //INBOX Wakeup enable
        // bsp_saradc_exit();
        // sfunc_pwrdown(0);
    }

    if (!CHARGE_DC_IN()) { //DC IN online?
        if ((chbox_cb.dc_in == 3) && CHARGE_INBOX()) {
            charge_box_leakage_set(0x81, 6); //用大电流leakage加速VUSB掉电，在检测到低于INBOX电压后关闭leakage(防止VUSB电容导致掉电慢)
        }
        if (chbox_cb.dc_in < 10) {
            chbox_cb.dc_in++;
        }
        charge_box_leakage_set(0, 0);
        inbox_sta = charge_box_inbox_check() & 0x3; //检查inbox online状态
    }
    #if (CHARGE_BOX_TYPE == CBOX_SSW)
        if (chbox_sta) { //智能充电仓已开盖，需要退出充电流程
            charge_box_leakage_set(1, 0);
            if (inbox_sta == 1) { //dc_in = 0, 且inbox = 0
                sys_cb.outbox_pwrton_flag = 1;
            }
            printf("charge_exit_for_open_window\n");
            return CHAG_STA_OFF;
        }
    #endif
}

```

图 5.5.2.3 充电循环处理

充电调试：充电问题 debug 过程中可以参考调用 dump_charge 函数，输出充电状态和相关电压检测状态，其中 CHARGE_DC_IN() 用来判定 5V 是否在线，当 VUSB 电压高于电池电压 0.2V 以上，vusb online；CHARGE_INBOX() 用来判定 inbox 电压是否在线，inbox 电压有 1.1V 和 1.7V 档位可调，当 vusb 电压高于 inbox 电压，inbox online；CHARGE_IS_VPEND1() 用来判断充电截止电压是否满足，CHARGE_IS_IPEND() 用来判断充电截止电流是否满足；根据这几个信息可以基本定位芯片充电过程中的状态是否异常

```

#if TRACE_EN
void dump_charge(void)
{
    static u8 chag_sta = 0xff;
    static u8 dc_in = 0xff;

    u32 tmp = PWRCON1;
    bool vpend = (bool)(tmp & BIT(28));
    bool ipend = (bool)(tmp & BIT(30));

    if (charge_ctl.chag_sta != chag_sta || dc_in != CHARGE_DC_IN()) {
        chag_sta = charge_ctl.chag_sta;
        dc_in = CHARGE_DC_IN();
        printf("charge: sta=%d, dc_sta=%d(%d,%d), pend=i=%d, v=%d\n", charge_ctl.chag_sta, dc_ctl.dc_sta, CHARGE_DC_IN(), CHARGE_INBOX(), ipend, vpend);
    }
}
#endif

```

图 5.5.2.4 充电调试模块

5.5.3 充电参数配置

充电参数配置模块：支持充电电流和截止电压的实时调整，相关的参数可以在配置和 config.h 文件中修改，根据电池性能去做相应的修改

```

/*****
* Module    : 充电功能选择
*****/
#define CHARGE_EN 1 //是否打开充电功能
#define CHARGE_TRICK_EN xcfg_cb.charge_trick_en //是否打开涓流充电功能
#define CHARGE_DC_NOT_PWRON xcfg_cb.charge_dc_not_pwron //DC插入，是否软开机。1: DC IN时不能开机
#define CHARGE_VOLT_FOLLOW_EN xcfg_cb.charge_voltage_follow // 打开电压跟随模式，适用于支持快充的充电仓
#define CHARGE_DC_IN() ((RTCCON >> 20) & 0x01)
#define CHARGE_INBOX() ((RTCCON >> 22) & 0x01)
#define CHARGE_LOW_POWER_EN 0 //是否打开充电低功耗模式，打开此宏时，不能 INTF_HUART 传输数据
#define CHARGE_WORKING_WHILE_CHARGING xcfg_cb.charge_working_while_charging //是否打开边充电边工作功能，开启此功能需要关闭CHARGE_DC_NOT_PWRON和充电仓功能

//充电截止电流
#define CHARGE_STOP_CURR xcfg_cb.charge_stop_curr
//充电截止电压: 0:4.2v 1:4.35v 2:4.4v 3:4.45v
#define CHARGE_STOP_VOLT 0
//恒流充电（电池电压大于2.9v）电流
#define CHARGE_CONSTANT_CURR xcfg_cb.charge_constant_curr
//涓流截止电压: 0:2.9v; 1:3v
#define CHARGE_TRICK_STOP_VOLT 1
//涓流充电（电池电压小于2.9v）电流
#define CHARGE_TRICKLE_CURR xcfg_cb.charge_trickle_curr
//恒压差充电差值选择: 0:125mV 1:187.5mV 2:250mV 3:312.5mV
#define CHARGE_VOLT_FOLLOW_DIFF 3
//漏电流选择
#define CHARGE_LEAKAGE_CURR xcfg_cb.ch_leakage_sel //0-3
//充电截止控制（电压满足后截止电流未满足的情况下的充电计时）
#define CHARGE_STOP_TIME xcfg_cb.charge_stop_time

```

图 5.5.3.1 充电参数配置

系统配置	充电配置	
DAC 配置	充电使能	☒
音量配置	涓流充电使能	☒
电源配置	插入DC禁止软开机	☒
充电配置	边充电边工作使能	☐
蓝牙配置	充电电压跟随模式	☐
无线配置	充电截止电流	30mA
RF参数	恒流充电电流	110mA
MIC 参数	涓流充电电流	30mA
算法参数	充满电蓝灯亮	亮30秒
按键配置	充电截止计时	30min
LED灯配置	充满电自动关机	☒
资源配置	充电仓功能	☒
	充电仓类型选择	兼容5V短暂掉0V后维持电压
	仓允许最低维持电压	1.1V
	从充电仓拿出自动开机	☒
	出仓或入仓VUSB漏电配置	漏电电流X2档
	入仓耳机关机	☒
	短暫停0V的合穩中檢測時間	0 (0 ~ 15)

选择充电截止电流

图 5.5.3.2 充电参数模式配置

电压跟随模式：主要是配合特定的仓（限流）来实现跟随充的功能，进而提升充电效率；可选的恒压差档位有 4 档，通过 CHARGE_VOLT_FOLLOW_DIFF 配置，压差越小，效率越高；

仓允许的最低维持电压 (inbox 电压)：档位有 1.1V 和 1.7V 选择，根据实际项目使用的仓的维持电压去配，配置的电压比仓实际输出的维持电压低就行；

vusb 漏电配置：主要为了给无线麦出入仓的时候给一个漏电去唤醒仓，三档可调

充电仓类型选择：根据实际项目使用的仓的充满输出电压去配，比如耳机在仓内充满之后的仓的电压正常从 5v 掉到 0v，那么这里的类型可以选择 5v 掉电无维持电压；如果仓的充满后的电压从 5v 掉到维持电压，那么就选择 5v 掉电但有维持电压的仓；

充电截止计时：主要是计算电压满足后电流未满足的情况下的充电时长计时，无线麦充电过程中，一般是先满足截止电压判定 (CHARGE_IS_VPEND1())，随着电流的慢慢减小，才到截止电流的满足判定 (CHARGE_IS_IPEND())，这个参数也可通过 charge_set_stop_time()去重新配置

边充电边工作：打开这个功能时，程序可以正常跑模式功能状态下正常充电，但是要注意关闭掉“插入 DC 禁止软关机功能”

充满电唤醒配置：仓内工作时，无线麦设备正常充满电关机后，需要根据仓的实际电压状态去配置一些唤醒源，比如说常规唤醒就是 wko 按键唤醒，也有跟充电仓相关的 vusb_wakeup(5v 唤醒)和 inbox wakeup (0v 唤醒)，5v 唤醒就是插入 5v 唤醒无线麦开机，0v 唤醒就是 vusb 掉电掉到 0v 唤醒无线麦开机，一般来说 5v 唤醒用在无线麦入仓充电，0v 唤醒用在无线麦出仓唤醒；注意芯片本身不支持在一边关机一边给唤醒源，关机过程中，如果有对应的唤醒源 pending 过来，会导致整个系统立马 reset，关机失败；所以，配置 vusb 唤醒，需要等待 5V 掉下去之后才能关机，同理，配置 inbox 下降沿唤醒的时候 vusb 端应该有个维持电压；

5.5.4 电量检测

sdk 内部采用 10bit saradc 电池电压检测机制，由于 saradc 的参考电压是 vddio，由于电池电压要比 vddio 电压高，所以 vbat 需要分压一半进来，内部计算公式为：

电池电压 = (电池电压 SARADC 采样值 * VBG 参考电压 * VBAT2_COEF) / VBG SARADC 采样值

其中 VBAT2_COEF 和 VBG 参考电压一般通过 trim 值做精确调整；SARADC 每次采样计算出 vbat 电压之后，默认采用平均值算法，算出平均值，由于电池电压会有跳动，软件可判定偏差大于一定值 (SDK 默认 30mv) 之后再更新到系统电压值 (sys_cb.vbat)；

```

AT(.com_text.port.vbat)
uint16_t vbat_get_voltage(void)
{
    static u16 vbat_bak = 0;

    u32 vbat = VBAT_CACL_VOLTAGE();
    u32 delta;

    //不同方案可能采用不同 vbat 滤波算法，在方案对应的plugin.c中处理
    plugin_vbat_filter(&vbat);
    //默认的取平均值算法.
    adc_cb.vbat_total = adc_cb.vbat_total - adc_cb.vbat_val + vbat; //均值
    adc_cb.vbat_val = adc_cb.vbat_total>>5;

    if(adc_cb.vbat_val > vbat_bak) {
        delta = adc_cb.vbat_val - vbat_bak;
    } else {
        delta = vbat_bak - adc_cb.vbat_val;
    }

    if (delta >= 30) { //偏差大于一定值则更新
        vbat_bak = adc_cb.vbat_val;
        // printf(bat_str, adc_cb.vbat_val/1000, adc_cb.vbat_val%1000);
    }

    return vbat_bak;
}

```

图 5.5.4.1 电量检测值更新

5.6 外设驱动

5.6.1 SPI

芯片共有两组 SPI，其中 SPI0 为内部 SPI FLASH 使用，不对外开放，当前 SDK 支持使用的为 SPI1，关联文件在 sdk bsp_spi.c 中，使用硬件 spi1 需要在 config.h 中打开 SPI_HW_EN 宏，如下图。

```

/*****
 * Module      : SPI1配置
 *****/
#define SPI_HW_EN 0 //是否使能硬件SPI功能(SPI默认配置:CPOL=0,CPHA=0)
#define SPI_2_WIRE_EN 1 //暂时仅支持2-WIRE模式,即只使用CLK和SPID0,SPID0可作为in或o
#define SPI_MASTER_EN 1 //0:SLAVE 1:MASTER
#define SPI_MAPPING | SPI1MAP_G3 //选择SPI mapping
#define SPI_BAUD_RATE 2000000 //SPI波特率2M

```

目前 SDK 支持 SPI 2/3 线模式，两线模式只使用 CLK 和 SPID0（芯片支持单线时分复用），SPID0 可作为 in 或者 out，三线则 CLK/MOSI/MISO 均使用。如下图 SPI_2_WIRE_EN 不使能，则使用三线模式（当前 SDK 封装的 API 为两线模式）。需要注意当 SPI 处于 Master 模式时，CS 引脚需要用户使用 GPIO 控制。

```

/*****
* Module : SPI1配置
*****/
#define SPI_HW_EN 0 //是否使能硬件SPI功能(SPI默认配置:CPOL=0,CPHA=0)
#define SPI_2_WIRE_EN 1 //暂时仅支持2-WIRE模式,即只使用CLK和SPIDO,SPIDO可作为in或o
#define SPI_MASTER_EN 1 //0:SLAVE 1:MASTER
#define SPI_MAPPING SPI1MAP_G3 //选择SPI mapping
#define SPI_BAUD_RATE 2000000 //SPI波特率2M

```

支持复用成 spi 功能引脚的 GPIO 如下图错误!使用“开始”选项卡将 应用于要在此处显示的文字。-错误!未定义书签。需要配置对应 FUNCMCN1[7:4]寄存器位复用成对应的 SPI 引脚，需要注意的是 GPIO 也需要配置成 func pin，sdk 中已经做好了相关的引脚复用，需要在 config.h 配置需要复用的组号，如下图 5.6.1-2 截图所示。

PAD Name	SPI1
FUNCMCN控制位	FUNCMCN1[7:4]
PA4	
PA5	SPI1DI-G2 / SPI2W_DIO1-G2
PA6	SPI1CLK-G2
PA7 (update)	SPI1DO/SPI1DATA-G2
PB0/WK1	SPI1DI-G3 / SPI2W_DIO1-G3
PB1/WK2	SPI1CLK-G3
PB2/WK3	SPI1DO/SPI1DATA-G3
PB3/USBDP (update)	SPI1CLK-G1
PB4/USBDM	SPI1DO/SPI1DATA-G1
PB5/WK0	SPI1DI-G1 / SPI2W_DIO1-G1
PE0	
PE4/SD_PG	
PE5	SPI1DI-G4 / SPI2W_DIO1-G4
PE6	SPI1CLK-G4
PE7	SPI1DO/SPI1DATA-G4
PF0	

图 错误!使用“开始”选项卡将 应用于要在此处显示的文字。-错误!未定义书签。 SPI 引脚复用

```

/*****
* Module : SPI1配置
*****/
#define SPI_HW_EN 0 //是否使能硬件SPI功能(SPI默认配置:CPOL=0,CPHA=0)
#define SPI_2_WIRE_EN 1 //暂时仅支持2-WIRE模式,即只使用CLK和SPIDO,SPIDO可作为in或o
#define SPI_MASTER_EN 1 //0:SLAVE 1:MASTER
#define SPI_MAPPING SPI1MAP_G3 //选择SPI mapping
#define SPI_BAUD_RATE 2000000 //SPI波特率2M

```

图 错误!使用“开始”选项卡将 应用于要在此处显示的文字。-错误!未定义书签。 SDK SPI 引脚复用

完成上述配置后，用户可调用下图 API 通过 SPI 和外设进行通信，API 在 bsp_spi.h 中声明，使用 DMA 与否用户按项目所需选择。

```

23:
24:
25: void bsp_spi_init(void);
26:
27: void spi_byte_tx(u8 data);
28: void spi_byte_rx(u8 *data);
29:
30: void spi_dma_tx(u8 *buf, uint len);
31: void spi_dma_rx(u8 *buf, uint len);
32:

```

5.6.2 IIC

芯片支持一路硬件 I2C，SDK 有两种 IIC 实现方式，一种是硬件 IIC，另一种为软件模拟 IIC，用户按需打开下图宏，使用对应的 IIC 功能，软/硬件 IIC 对应的源文件均在 bsp_i2c.c 中，用户可自行查阅，调用文件中的 API。若用户需要多路 I2C 通信，可以软/硬件 I2C 结合使用。

```

:
: /*****
:  * Module      : I2C配置
:  *****/
: #define I2C_HW_EN          0           //是否使能硬件I2C功能
: #define I2C_MAPPING        I2CMAP_PB2PB1 //选择I2C mapping
: #define I2C_SW_EN          (0 || SOFT_I2C_EN) //是否使能软件I2C功能
: #define I2C_MUX_SD_EN      0           //是否I2C复用SD卡的IO
:
:

```

使用软件 GPIO 模拟 IIC 时，用户可以按需修改 config.h 中 I2C SCL SDA 使用的 gpio，如工程中默认使用 GPIO PB1 PB2，PB1 为 SCL PB2 为 SDA，示例如下图所示。

```

#define I2C_SCL_IN()          GPIOBDIR |= BIT(1);
#define I2C_SCL_OUT()         {GPIOBDIR &= ~BIT(1); GPIOBDE |= BIT(1);}
#define I2C_SCL_H()           GPIOBSET = BIT(1);
#define I2C_SCL_L()           GPIOBCLR = BIT(1);
#define I2C_SDA_IN()          {GPIOBDIR |= BIT(2); GPIOBPU |= BIT(2);}
#define I2C_SDA_OUT()         {GPIOBDIR &= ~BIT(2); GPIOBDE |= BIT(2);}
#define I2C_SDA_H()           GPIOBSET = BIT(2);
#define I2C_SDA_L()           GPIOBCLR = BIT(2);
#define I2C_SDA_IS_H()        (GPIOB & BIT(2))

```

支持复用成硬件 IIC 功能的 gpio 如下图所示，需要配置 FUNCMCN2[27:24]复用成对用的 IIC，工程中引脚复用配置可在 config.h I2C 配置中 I2C_MAPPING 配置对应的 IO 组。

PAD Name	IIC
FUNCMCON控制位	FUNCMCON2[27:24]
PA4	
PA5	IIC_DAT-G2
PA6	IIC_CLK-G1/G2
PA7 (update)	IIC_DAT-G1
PB0/WK1	IIC_DAT-G4
PB1/WK2	IIC_CLK-G3/G4
PB2/WK3	IIC_DAT-G3
PB3/USBDP (update)	IIC_CLK-G7
PB4/USBDM	IIC_DAT-G7
PB5/WK0	
PE0	
PE4/SD_PG	
PE5	IIC_DAT-G6
PE6	IIC_CLK-G5/G6
PE7	IIC_DAT-G5
PF0	IIC_CLK-G8
PF1	IIC_DAT-G8

```

: ***** /
: #define I2C_HW_EN 0 //是否使能硬件I2C功能
: #define I2C_MAPPING I2CMAP_PB2PB1 //选择I2C mapping
:

```

用户选择完对应软硬件 I2C 配置以及确定好引脚复用后，相关 I2C 软硬件 I2C api 在 bsp_i2c.h 中声明。按需选择对应的软硬件 API 和外设进行通信。

```
void bsp_i2c_init(void);

extern uint8_t auth_cp_online_en;

#ifdef I2C_SW_EN
void bsp_i2c_start(void);
void bsp_i2c_stop(void);
void bsp_i2c_tx_byte(uint8_t dat);
uint8_t bsp_i2c_rx_byte(void);
bool bsp_i2c_rx_ack(void);
void bsp_i2c_tx_ack(void);
void bsp_i2c_tx_nack(void);
#else I2C_HW_EN
#define DATA_CNT_1B 1
#define DATA_CNT_2B 2
#define DATA_CNT_3B 3
#define DATA_CNT_4B 4

#define START_FLAG0 BIT(3)
#define DEV_ADDR0 BIT(4)
#define REG_ADDR_0 BIT(5)
#define REG_ADDR_1 (BIT(5) | BIT(6))

#define START_FLAG1 BIT(7)
#define DEV_ADDR1 BIT(8)
#define RDATA BIT(9)
#define WDATA BIT(10)

#define STOP_FLAG BIT(11)
#define NACK BIT(12)

//i2c_cfg配置I2C数据格式:
//1、发送: [START_FLAG0][DEV_ADDR0] [REG_ADDR_0(REG_ADDR_1, 16位地址)] [WDATA] [STOP_FLAG] (设上WDATA)
//2、接收: [START_FLAG0][DEV_ADDR0] [REG_ADDR_0(REG_ADDR_1, 16位地址)] [START_FLAG1] [DEV_ADDR1] [RDATA] [STOP_FLAG] (设上RDATA)

void bsp_i2c_tx_byte(u16 dev_addr, u16 reg_addr, u32 data);
void bsp_i2c_rx_buf(u16 dev_addr, u16 reg_addr, u8 *buf, u16 len);
void bsp_i2c_init(void);
#endif
#endif
```

5.6.3 IIS

芯片支持一路硬件 IIS，支持 44.1/48K 采样率，数据源从 SRC 直通 IIS 或者从内部 SRAM DAM 到 IIS fifo，支持 16/32 bit 数据模式，主从可切换，支持 IIS 数据正常模式和左对齐模式，

SDK 中在 config.h 中配置 IIS，如下图所示，关联源文件为 bsp_i2s.c。

```
/* *****
 * Module : I2S配置
 * ***** */

#define I2S_EN 0 //是否使能I2S功能
#define I2S_DEVICE I2S_DEV_NO //I2S设备选择
#define I2S_MAPPING_SEL I2S_IO_G1 //I2S IO口选择
#define I2S_MODE_SEL 0 //I2S主从模式选择 0: master; 1:slave
#define I2S_BIT_MODE 0 //I2S数据位宽选择 0:16bit; 1:32bit
#define I2S_DATA_MODE 1 //I2S数据格式选择 0:left-justified mode; 1:normal mode
#define I2S_DMA_EN 0 //I2S数据源选择 0:src; 1:dma
#define I2S_MCLK_EN 1 //I2S是否打开MCLK
#define I2S_MCLK_SEL 2 //I2S MCLK选择 0:64fs 1:128fs 2:256fs
#define I2S_PCM_MODE 0 //I2S是否打开PCM mode
#define I2S_DAC_OUT_SET | SPR_44100 //做从机时，配置dac采样率，做主机时可忽略
```

支持复用为 IIS 的 gpio 如下图所示，需要配置 FUNCMCN2[3:0]寄存器复用成对应的 IIS 组。

PAD Name	IIS
FUNCMCON控制位	FUNCMCON2[3:0]
PA4	IISMCLK-G1
PA5	IISCLK-G1
PA6	IISLRCLK-G1
PA7 (update)	IISDO/DAT-G1
PB0/WK1	IISCLK-G3
PB1/WK2	IISLRCLK-G3
PB2/WK3	IISDO/DAT-G3
PB3/USBDP (update)	IISDI-G1
PB4/USBDM	IISMCLK-G3
PB5/WK0	IISDI-G3
PE0	IISDI-G2
PE4/SD_PG	IISMCLK-G2
PE5	IISCLK-G2
PE6	IISLRCLK-G2
PE7	IISDO/DAT-G2

用户须在 config.h 中配对 IIS 工作模式，如主/从，数据位宽 16/32bit，IIS 数据源从 SRC 还是 DMA，数据格式为左对齐还是正常模式，根据项目需要选择是否需要输出 MCLK，以及 MCLK 为多少采样率倍数。

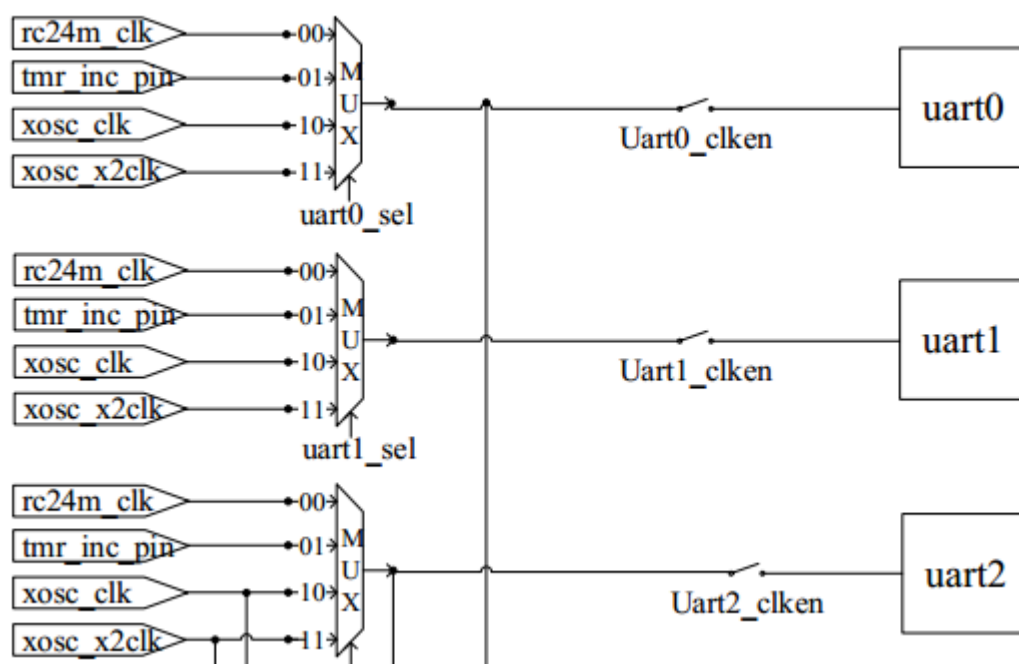
5.6.4 UART

芯片支持三组普通串口，一组高速串口，芯片均支持单线通信能力（时分复用），其中 uart0 作为芯片调试串口使用（波特率为 1.5M），用户可使用 uart1、uart2 和 huart。

SDK 中有使用 uart1 和充电仓通信的示例，如需使用 uart1 或者 uart2，可以参考 bsp_uart.c 中源码修改。支持复用成 uart 的 gpio 如下图所示。

PAD Name	UART0	UART1	UART2	HS UART
FUNCMCN控制位	TXMAP=FUNCMCN0 [11:8] RXMAP=FUNCMCN0 [15:12] when RXMAP=0x7, TX pin will map to RX	TXMAP=FUNCMCN0 [27:24] RXMAP=FUNCMCN0 [31:28] when RXMAP=0x7, TX pin will map to RX	TXMAP=FUNCMCN2 [11:8] RXMAP=FUNCMCN2 [15:12] when RXMAP=0x7, TX pin will map to RX	TXMAP=FUNCMCN0 [19:16] RXMAP=FUNCMCN0 [23:20]
PA4				CTS
PA5				RTS
PA6	RX0-G1	RX1-G1		HSTRX-G6
PA7 (update)	TX0-G1 (RX)	TX1-G1 (RX) - upDate		HSTRX-G1
PB0/WK1				
PB1/WK2	RX0-G2		RX2-G2	HSTRX-G7
PB2/WK3	TX0-G2 (RX)		TX2-G2 (RX)	HSTRX-G2
PB3/USBDP (update)	TX0-G3 (RX) - upDate			HSTRX-G3
PB4/USBDM	RX0-G3			HSTRX-G8
PB5/WK0				
PE0	TX0-G5 (RX)			
PE4/SD_PG				
PE5				
PE6	RX0-G4		RX2-G1	HSTRX-G9
PE7	TX0-G4 (RX)		TX2-G1 (RX)	HSTRX-G4
PF0		TX1-G2 (RX)		HSTRX-G5
PF1		RX1-G2		HSTRX-G10
MICP0				
MICN0				
MICP1				
MICN1				
MICP2				
MICN2				
MICP3				
MICN3				
MICP4				
MICN4				
VDDF				
PG0/SPIWP-MCP_WP				
PG1/SPIDIN-MCP_SO				
PG2/SPICS-MCP_CS				
PG3/SPIHOLD-MCP_HOLD				
PG4/SPICLK-MCP_CLK				
PG5/SPIDOUT-MCP_SI				
DACR				
DACR#				
DACL				
DACL#/VCMBUF				
VUSB (update)	TX0-G6 (RX)	TX1-G3 (RX)	TX2-G3 (RX) - upDate	HSTRX-G11

CLK 控制位如下图所示（需要将对应 BIT 置位才能正常使用）



uart0_sel	:CLKCON1[20:19]
uart1_sel	:CLKCON1[22:21]
uart2_sel	:CLKCON1[24:23]
hut_sel	:CLKCON1[18:16]

clkqat0		c
0	tmr0_clken	1
1	tmr1_clken	1
2	tmr2_clken	1
3	tmr3_clken	1
4	tmr4_clken	1
5	tmr5_clken	1
6	uart0_clken	1
7	uart1_clken	1
8	uart2_clken	0
9	hsut0_clken	1

波特率需要根据选择的时钟源计算，计算方法为如下图所示，注意使用固定波特率需要配置 UARTXCON[5],选择固定波特率。

Register 12-3 UART0BAUD/UART1BAUD/UART2BAUD: UART0/1/2 Baud Rate Register

Bit	Name	Mode	Default	Description
31:16	DARTBAUD	R	0x0	UART detect baud by hardware
15:0	BAUD	WR	x	UART Baud Rate Baud Rate =Fudet clock / (BAUD + 1)

5	FIXBAUD	WR	0	Fix baud enable 0: auto detect baud 1: fix baud
---	---------	----	---	---

高速串口实现源码在 bsp_huart.c 中，用户如需使用到 huart 进行通信，可以参考示例程序对 huart_t 结构体进行赋值，结构体成员如下图所示

```
typedef struct {
    union {
        struct {
            uint8_t tx_port : 4;
            uint8_t rx_port : 4;
            uint8_t rxisr_en : 1;
            uint8_t txisr_en : 1;
            uint8_t rxbuf_loop : 1;
            uint8_t tx_1st : 1;
        };
        uint16_t all_setting;
    };
    uint16_t rxbuf_size;
    uint8_t *rxbuf;
} huart_t;
```

成功初始化后，若配置了 tx rx 中断，tx 或者 rx 成功后，会回调以下两个 API，用户可以自行处理数据。

```
AT(.com_huart.text)
void huart_tx_done_cb(void) ...

AT(.com_huart.text)
void huart_rx_done_cb(void) ...
```

5.7 休眠与关机

5.7.1 休眠功能

休眠是指芯片蓝牙在空闲状态下为保持蓝牙的正常连接，但又避免长时间空闲，持续消耗电量，而进入的一种仅通过 BLE 间歇性唤醒的一种低功耗状态。

目前的方案中，仅有进入了蓝牙直连的 BT 模式才会进入休眠状态，若设置无线麦为接收端（adapter）或发射端（emit）都不会触发休眠。需要注意的是，目前 897X 仅有开启双 buck 的情况下才可以正常进入休眠模式。

5.7.1.1 休眠的开启与退出

默认 SDK 的情况下不开启休眠，若要开启休眠，需在配置文件中，选择自动进入休眠的时间。开启休眠后，有两种情况会进入休眠：1）自动休眠时间内，没有连上蓝牙设备；2）连上蓝牙设备情况下，蓝牙设备空闲并超过自动休眠时间。进入休眠后，若无线麦设备有动作或者蓝牙设备有动作，将会自动退出休眠。



图 5.7.1 自动休眠时间的配置

5.7.1.2 休眠代码的简介

休眠时间的初始化

初始化时，根据配置文件设置的“自动休眠时间”，设置到 sys_cb.sleep_delay，后面根据 reset_sleep_delay() 接口设置 sys_cb.sleep_delay。

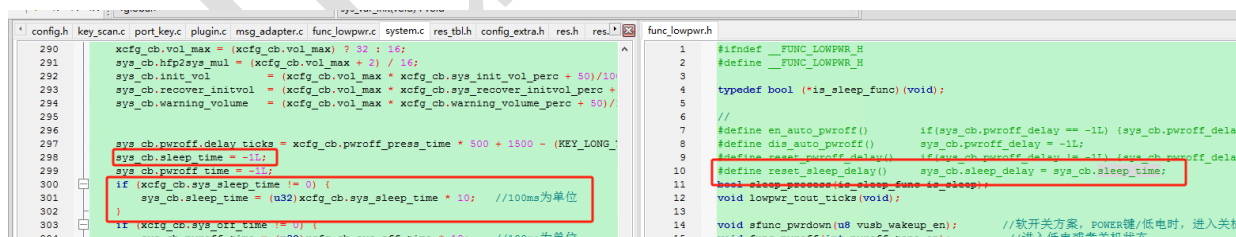


图 5.7.2 自动休眠时间的初始化

自动休眠的流程

自动休眠时间的变量主要的就是 sys_cb.sleep_delay，若没有连接蓝牙设备或者蓝牙空闲时，sys_cb.sleep_delay 在 100ms 的定时器中递减，当 sys_cb.sleep_delay 递减为 0 时，进入休眠。若连上蓝牙设备且不空闲时，会循环调用 reset_sleep_delay() 接口，使得 sys_cb.sleep_delay 一直不会递减至 0，从而不进入休眠。

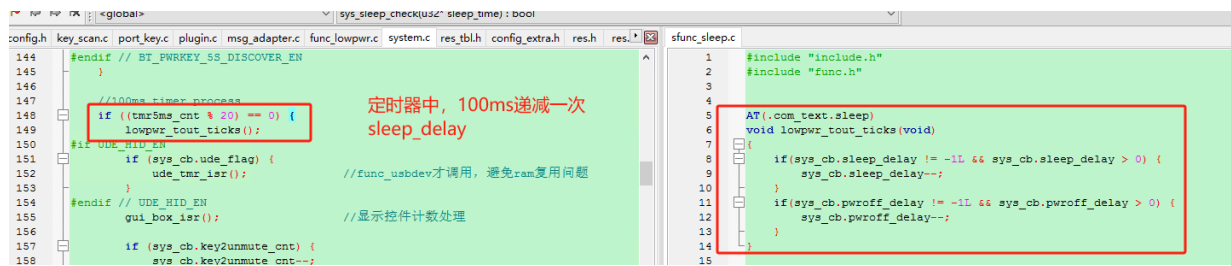


图 5.7.3 自动休眠/关机时间的递减

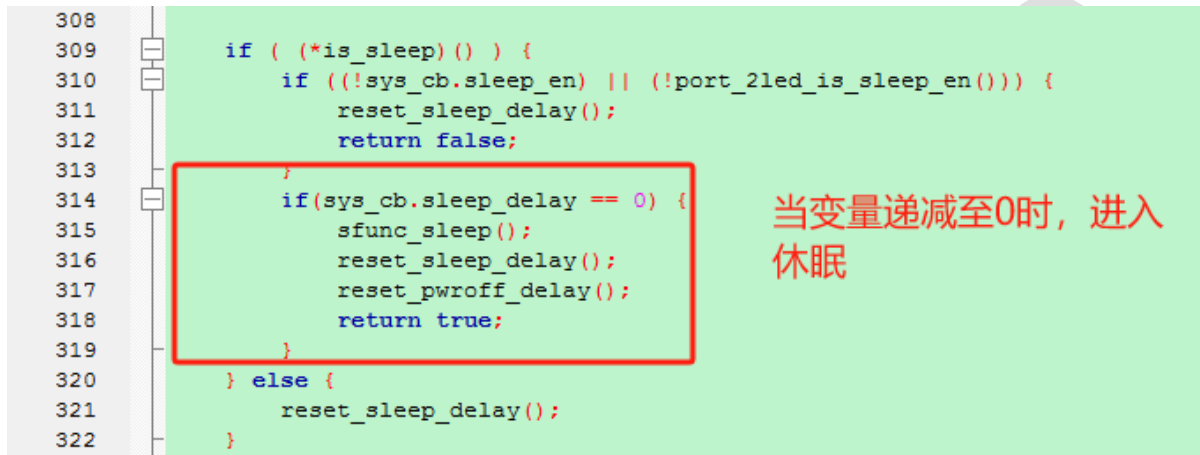


图 5.7.4 进入休眠

休眠后流程

进入休眠后，程序上一直执行 sfunc_sleep()，在该函数中，通过 bt_is_sleep()接口检测休眠状态，同时 500ms 调用一次 sleep_timer()，等待芯片从休眠状态唤醒。

5.7.2 关机功能

关机主要是跑进 func_pwroff()函数，代码里边主要是通过按键或代码流程，设置 func_cb.sta = FUNC_PWROFF 来进入关机。

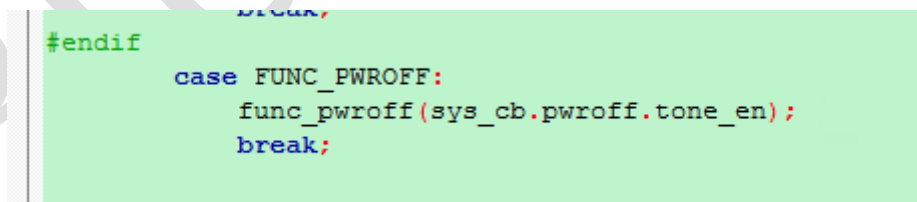


图 5.7.5 关机接口

5.7.2.1 自动关机功能

自动关机功能是指设备长时间没有配对连接任何设备，为避免长时间空闲，持续消耗电量，而选择自动关机的一种低功耗技术。

目前的方案中 adapter、emit 或者是蓝牙直连的 BT 都可以触发自动关机功能。

5.7.2.2 自动关机的开启

默认 SDK 的情况下不开启自动关机，若要开启自动关机，需在配置文件中，选择自动进入关机的时

间。开启后，指设备长时间没有配对连接任何设备就会自动进入关机，只有在芯片被唤醒后，才能退出关机。



图 5.7.6 自动关机时间配置

5.7.2.3 自动关机代码的简介

自动关机时间初始化

初始化时，根据配置文件设置的“自动关机时间”，设置到 `sys_cb.pwroff_time`，后面根据 `en_auto_pwroff()` 接口设置 `sys_cb.pwroff_delay`。`dis_auto_pwroff()` 接口是置位自动关机时间。默认的机制是，当没有连接任何蓝牙设备时，调用 `en_auto_pwroff()` 接口，当连上了蓝牙设备时，调用 `dis_auto_pwroff()` 接口。

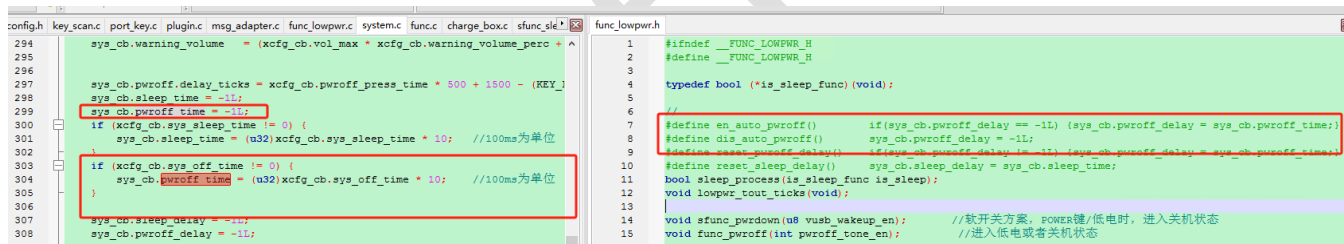


图 5.7.7 自动关机时间初始化

自动关机的流程

自动关机时间的变量主要的就是 `sys_cb.pwroff_delay`，若没有连接蓝牙设备时，`sys_cb.pwroff_delay` 在 100ms 的定时器中递减，当 `sys_cb.pwroff_delay` 递减为 0 时，进入关机，变量递减的接口和自动休眠的接口一致如上图 5.7.3。

当 `sys_cb.pwroff_delay` 递减至 0 时，置位 `sys_cb.pwroff.timeout_ind`，后面在 `pwrroff_process()` 函数中，跳转到关机。



图 5.7.8 自动关机时间初始化

5.7.3 唤醒开机

默认的 SDK 使用 pwrkey 作为唤醒开机的引脚，默认配置下降沿唤醒。关机后 pwrkey 由内部上拉电阻上拉至高电平，此时 pwrkey 按下，产生一个下降沿，芯片则会唤醒。pwrkey 对应芯片中的 WK0 唤醒，对于 BT897X 共有四个唤醒源可配置 WK0~WK3。

PA4	
PA5	
PA6	
PA7 (update)	SD升级接口
PB0/WK1	
PB1/WK2	
PB2/WK3	
PB3/USBDP (update)	USB升级接口
PB4/USBDM	
PB5/WK0	10S Reset 主唤醒源
PE0	
PE4/SD_PG	SD PG
PE5	
PE6	

pwrkey即是PB5

图 5.7.9 唤醒源示意

芯片从关机状态唤醒后，会跑到 power_on_check() 函数，在该函数中判断是否存在足够从唤醒状态进入开机状态的条件。如使用 pwrkey 键唤醒开机，会判断 pwrkey 长按下的状态，若已经长按了，则正常开机。

```

494
495     if(bsp_pwrkey_scan(&pwrkey) && pwrkey.state == PWRKEY_PRESS) {
496         sys_cb.ms_ticks = pwrkey.ticks;           //记录PWRKEY按键按下的时刻
497     }
498
499 #if CHARGE_EN
500     if ((CHARGE_DC_NOT_PWRON) && xcfg_cb.charge_en) {
501         charge_sta = sfunc_charge_process(counter++);
502         if(charge_sta == CHAG_STA_OFF) {
503 #if CHARGE_BOX_EN
504             if(xcfg_cb.ch_out_auto_pwr_on_en) {      //出充电仓，拿起开机
505                 pwr_on_flag = true;
506                 break;
507             }
508 #endif
509             } else if(charge_sta == CHAG_STA_OFF_VBUS_PATH || charge_sta == CHAG_STA_OFF_INBOX) {
510                 if(xcfg_cb.ch_full_1 == CHAG_STA_OFF_VBUS_PATH=5
511                     break;
512                 }
513             }
514             if(CHARGE_DC_IN() || (charge_sta > CHAG_STA_W4_DCIN)) {      //充电情况下需要等待状态稳定
515                 continue;
516             }
517         }
518     #endif
519
520     if (pwrkey.state == PWRKEY_PRESS) {           //PWRKEY已长按
521 #if VBAT_DETECT_EN
522         if (sys_cb.vbat <= 2950) {                //电压小于2.95v不开机
523             continue;
524         }

```

图 5.7.10 pwrkey 开机流程

5.8 无线麦功能

无线麦主要有 2 种连接方式。第一种是 2.4G 连接方式，在 2.4G 私有连接模式下，至少要有有一个接收端 (adapter) 和一个发射端 (emit)，组成连接。若存在 2 个发射端 (emit)，则可以组成一拖二的连接模式，如图 5.8.1、图 5.8.2 所示。第二种就是蓝牙直连方式，发射器可以直接被其他蓝牙主设备（例如手机）进行连接，在蓝牙直连模式下，目前仅支持一拖一模式，如图 5.8.3 所示。



图 5.8.1 一拖一连接模式

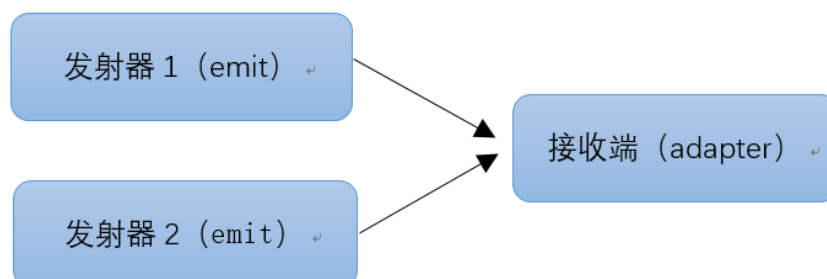


图 5.8.2 一拖二连接模式



图 5.8.3 蓝牙直连模式

在 2.4G 连接下，传输音频的采样率 48K，采用 LC3 编解码算法，并支持一些音效算法处理，其发射端和接收端的音频链路如下。

发射器音频链路（具体链路详见 func_device.c 文件的 mic_tx_cfg_tbl）：

```

MIC_ADC --> MIC_EFFECT (音效算法) --> MIC_ENC (编码) --> PUT_TX_FRAME (编码
帧放到传输 buf) --> DEVICE_TX (硬件 TX)
  
```

接收端音频链路（具体链路详见 func_adapter.c 文件的 mic_rx_cfg_tbl）：

```

ADAPTER_RX (硬件 RX) --> SET_RXPKT (存放数据包) --> MIC_DEC (解码) -->
MIC_EFFECT (音效算法) --> DAC --> USB (UAC)
  
```

在蓝牙直连模式下，传输的音频采样率为 16k，采用的是 msbc 编解码算法，目前支持 AGC 算法、AI 降噪，具体音频处理，详见 bt_sco.c 文件下 bt_adc_process。

```

MIC_ADC --> bt_adc_process --> MIC_EFFECT (音效算法) --> bt_sco_tx
  
```

5.8.1 2.4G 连接

本小节将介绍 2.4G 连接下的组队方式、组队绑定、消息控制、同步等，以及打印 RSSI、PER。

默认 SDK 配置，采用设备名称方式进行组队连接，即只有设备名称相同的设备才能建立连接，而设备名称配置，可在烧录配置文件（wireless_adapter.setting、wireless_emit.setting）里进行修改。



图 5.8.1.1 设备名称修改

具体的组队连接流程，这里进行简单的介绍。在连接阶段，首先接收端（adapter）进行广播，发射器进行扫描（scan），当发射器扫描到的广播包时，先检查广播包的加密信息是否符合连接要求（这个动作在蓝牙库底层完成），若符合要求，则会回调到 app 层，发起连接。

```

config.h wireless.c
101
102 //发送ble_scan_set_enable消息之后，底层收到数据包的回调函数
103 //返回值：0=错误的地址，继续扫描；1=地址正确，自动关闭扫描，可发起连接请求
104 AT(.text.wireless.cb)
105 bool wireless_scan_rx_rep_cb(uint8_t *addr, uint8_t addr_type, uint8_t *adv_ext_data)
106 {
107     #if WIRELESS_CON_AND_BT_PAIRING_EN
108         bool is_con_allowed = wireless_bonding_check(adv_ext_data);
109         if(!is_con_allowed) {
110             return false;
111         }
112     #endif
113
114     TRACE("addr: ");
115     TRACE_R(addr, 6);
116
117     memcpy(con_req_addr, addr, 6);
118
119     ///准备发起连接，关闭scan
120     func_device_set_create_con();
121
122     return true;
123 }
    
```

图 5.8.1.2 发射器发起连接回调函数

广播包的加密信息主要由一些 feat 值（cfg_wireless_feat、cfg_wireless_ext_feat）、设备名称组成。只有当 feat 值和设备名称相同时，发射器和接收端才能连接。feat 值包含了一些重要的信息，例如传输机制，跳频算法版本等，详见 wireless.c 文件里的参数配置。如果用户在调试过程中，出现发射器和接收端连不上的

情况，可先检查特征值以及设备名称是否相同。

```

13
14 #define D2A_CODEC (WIRELESS_CON_CODEC_SEL | WS_CODEC_MONO)
15 #define A2D_CODEC 0
16
17 #if WIRELESS_EN
18 //link
19 //uint8_t cfg_wireless_cntl_bit = WIRELESS_CON_CNTL_BIT;
20 uint8_t cfg_wireless_rssi_thr = WIRELESS_CON_RSSI_THR;
21 uint8_t cfg_wireless_link_nb = WIRELESS_CON_LINK_NB;
22 uint8_t cfg_wireless_d2a_tx_size = WIRELESS_MIC_TX_BUFFER_SIZE; //传输封包大小
23 uint8_t cfg_wireless_con_interval = WIRELESS_CON_INTERVAL*WIRELESS_CON_LINK_NB; //连接interval (v1的传
24 uint8_t cfg_wireless_tx_comb_nb = WIRELESS_MIC_COMB_NB; //传输封包组合数量
25 uint8_t cfg_wireless_tx_interval = WIRELESS_MIC_TX_INTERVAL; //V2、V3的传输interval
26 uint8_t cfg_wireless_tx_retry = WIRELESS_MIC_RETRY_NB; //V2、V3的传输次数
27 uint16_t cfg_wireless_feat = ((WIRELESS_CON_VERS<<8)&FEAT_VERS) | (FEAT_D2A*WIRELESS_MIC_EN) | (FEAT_
28 uint16_t cfg_wireless_ext_feat = (EXT_FEAT_ADV_EXT*WIRELESS_CON_ADV_EXT_EN);
29 uint16_t cfg_wireless_d2a_enc_us = (WIRELESS_MIC_ENC_MAX_US + WIRELESS_MIC_EQ_DRC_DELAY + WIRELESS_MIC_ECHO
30 uint16_t cfg_wireless_d2a_dec_us = WIRELESS_MIC_DEC_MAX_US;
31 uint8_t cfg_wireless_ota_en = ADAPTER_AB_FOT_DEVICE_SUPPORT;
32 uint8_t cfg_wireless_codec[2] = {
33     D2A_CODEC, //D2A Codec
34     A2D_CODEC, //A2D Codec
35 };

```

图 5.8.1.3 基础特征值

在没有绑定功能时，只要发射器和接收端的名称相同就能连接，相当于自由组队。若用户希望使用绑定功能，即发射器和接收端进行唯一的绑定，可打开 WIRELESS_CON_BONDING_EN 配置宏。

```

182 *****/
183 #define AUDIO_EFFECT_DBG_EN 0 //音效在线调试使能
184
185 *****/
186 * Module : 无线功能选择配置
187 *****/
188 #define WIRELESS_MIC_EN 1 //是否支持MIC音频流 (上行音频)
189 #define WIRELESS_SPK_EN 0 //是否支持SPK音频流 (下行音频)
190 #define WIRELESS_DUMP_EN 1 //打印无线PER、RSSI等信息 (调试用)
191
192 //公共配置
193 #define WIRELESS_CON_CODEC_SEL WS_CODEC_LC3F //编解码选择
194 #define WIRELESS_CON_FREQ_BAND 0 //无线mic工作频段 0:2.4G 1:2.2G 2:2.3G 3:2.5G
195 #define WIRELESS_CON_LINK_NB 2 //支持连几路无线麦 (目前只支持1/2路)
196 #define WIRELESS_CON_VERS 6 //传输机制版本 (2=v3)
197 #define WIRELESS_CON_BONDING_EN 1 //组队绑定
198 #define WIRELESS_CON_RSSI_THR 92 //设置连接范围RSSI (-dbm)，该值越小，组队范围越小 (产线建议
199 #define WIRELESS_CON_PWR_CTR 0 //是否支持自动调节pwr，近距离降低，远距离增加
200 #define WIRELESS_MIC_CMD_EN 1 //是否支持无线麦命令 (用于同步降噪、混响等参数设置等)
201 #define WIRELESS_CON_ADV_EXT_EN 0 //是否使能广播数据扩展功能 (用于连接阶段的私有消息传输)
202 #define WIRELESS_CON_AND_BT_PAIRING_EN 0 //无线麦连接和经典蓝牙配对功能是否共存 (该功能仅用于DEVICE
203

```

图 5.8.1.4 组队绑定控制宏

这里简单介绍一下组队绑定的流程。首先，发射器跟接收端进行连接时，双方都需要进入配对状态，当建立连接后，接收端会同步一个绑定密钥给发射器，等再次回连时，只有密钥相同才能连接。详见 wireless.c 文件下的 wireless_bonding_check 等相关函数。发射器和接收端进入配对状态，可以分别调用 wireless_device_pairing_enable 和 wireless_adapter_pairing_enable 函数（在 wireless.c 文件中）。

在 2.4G 连接中，支持传输用户自己的私有命令或者数据，详见 wireless_cmd_api.c 文件。这里举个例子进行解释说明。例如发射器通过私有命令方式，控制接收端的魔音音效切换。首先在 wireless_cmd.h 中定义一个控制命令 MAGIC_LEVEL_CHANGE 控制命令。


```

config.h  wireless.c  wireless_cmd.h
22      } wireless_cmd_t;
23
24      //无线麦控制
25      typedef enum{
26          //adapter opcode
27          ADAPTER_GET_VERS,
28          ADAPTER_GET_SN,
29          ADAPTER_GET_MAC,
30          ADAPTER_GET_BAT,
31          ADAPTER_SET_NR_STA,           //设置降噪开关、等级等参数
32          ADAPTER_SET_ECHO_STA,         //设置混响等级参数
33          ADAPTER_SET_MAGIC_STA,        //设置魔音等级参数
34
35          //device opcode
36          MUSIC_CTL_1ST = 0x80,
37          MUSIC_CTL_VOL_UP = MUSIC_CTL_1ST, //音乐音量加
38          MUSIC_CTL_VOL_DOWN, //音乐音量减
39          MUSIC_CTL_PREV, //上一曲
40          MUSIC_CTL_NEXT, //下一曲
41          MUSIC_CTL_PLAY_PAUSE, //播放/暂停
42          MUSIC_CTL_VOICE_REMOVE, //人声消除切换
43          ECHO_DELAY_LEVEL_UP, //混响等级加
44          ECHO_DELAY_LEVEL_DOWN, //混响等级减
45          MAGIC_LEVEL_CHANGE, //切换魔音音效
46
47          AUDIO_CRT_MUTE, //通知底层音频断开
48      } sub_ws_mic_cmd_t;
49

```

图 5.8.1.5 定义魔音控制命令

其次，发射器通过 wireless_tx_mic_cmd0 函数传输命令，这里为了方便调用，这里使用了宏方式进行重定义。

```

config.h  wireless.c  wireless_cmd.h
92      #define wireless_set_usbmic_status(start)
93      #else
94      #define wireless_set_usbmic_status(start)    wireless_tx_usb_cmd(USB_SET_SPK_VOLUME, start) //mic_
95      #endif
96
97      //device ==> adapter无线麦命令
98      #define wireless_echo_level_up()             wireless_tx_mic_cmd0(0, ECHO_DELAY_LEVEL_UP, 1)
99      #define wireless_echo_level_down()          wireless_tx_mic_cmd0(0, ECHO_DELAY_LEVEL_DOWN, 1)
100     #define wireless_magic_change()              wireless_tx_mic_cmd0(0, MAGIC_LEVEL_CHANGE, 1)
101     #define wireless_music_vol_up()              wireless_tx_mic_cmd0(0, MUSIC_CTL_VOL_UP, 1)
102     #define wireless_music_vol_down()            wireless_tx_mic_cmd0(0, MUSIC_CTL_VOL_DOWN, 1)
103     #define wireless_music_prev()                wireless_tx_mic_cmd0(0, MUSIC_CTL_PREV, 1)
104     #define wireless_music_next()                wireless_tx_mic_cmd0(0, MUSIC_CTL_NEXT, 1)
105     #define wireless_music_play_pause()          wireless_tx_mic_cmd0(0, MUSIC_CTL_PLAY_PAUSE, 1)
106     #define wireless_music_voice_remove()        wireless_tx_mic_cmd0(0, MUSIC_CTL_VOICE_REMOVE, 1)

```

图 5.8.1.6 魔音控制命令传输接口

这里简单解释一下 wireless_tx_mic_cmd0 函数接口中的形参。首先是 index，对发射器来说，它只能连一个接收端，所以从发射端到接收端的命令，index 永远等于 0。但是对于接收端来说，会存在连接 2 个发射器（一拖二）的情况，分别为 emit0 和 emit1，如果接收端要控制 2 个发射器，就要使用 index=0 和 index=1 分别进行控制。而 pcode 和 param 分别为操作命令和传递参数。


```
void wireless_tx_mic_cmd0(u8 index, u8 opcode, u8 param)
{
    wireless_cmd_t pdu;

    pdu.cmd = PRIVATE_WS_MIC_CMD;

    pdu.buf[0] = opcode;

    pdu.buf[1] = param;

    wireless_send_cmd(index, (u8 *)&pdu, 3);
}
```

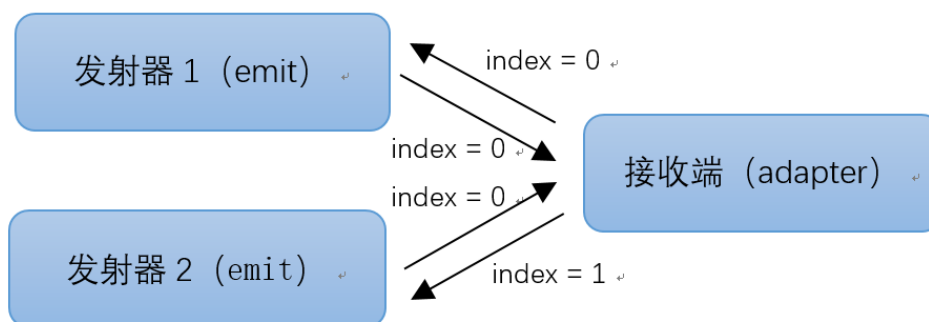


图 5.8.1.7 一拖二模式下的 index

最后，接收端收到 MAGIC_LEVEL_CHANGE 命令后，进行对应的处理。

```
config.h  wireless.c  wireless_cmd.h  wireless_cmd_apic
109  };
110
111  AT(.text.wireless_cmd)
112  void wireless_rx_mic_cmd(u8 index, wireless_cmd_t *pdu)
113  {
114      u8 opcode = pdu->buf[0];
115
116      switch(opcode) {
117      #if ECHO_EN
118          case ECHO_DELAY_LEVEL_UP:
119              echo_delay_level_up();
120              break;
121
122          case ECHO_DELAY_LEVEL_DOWN:
123              echo_delay_level_down();
124              break;
125      #endif
126      #if MAGIC_EN
127          case MAGIC_LEVEL_CHANGE:
128              magic_effect_level_change();
129              break;
130      #endif
131  }
```

图 5.8.1.8 魔音命令处理

若用户希望查看丢包率，信号强度等信息，可打开 WIRELESS_DUMP_EN，当接收端连接上发射器后，会打印相关的调试信息。

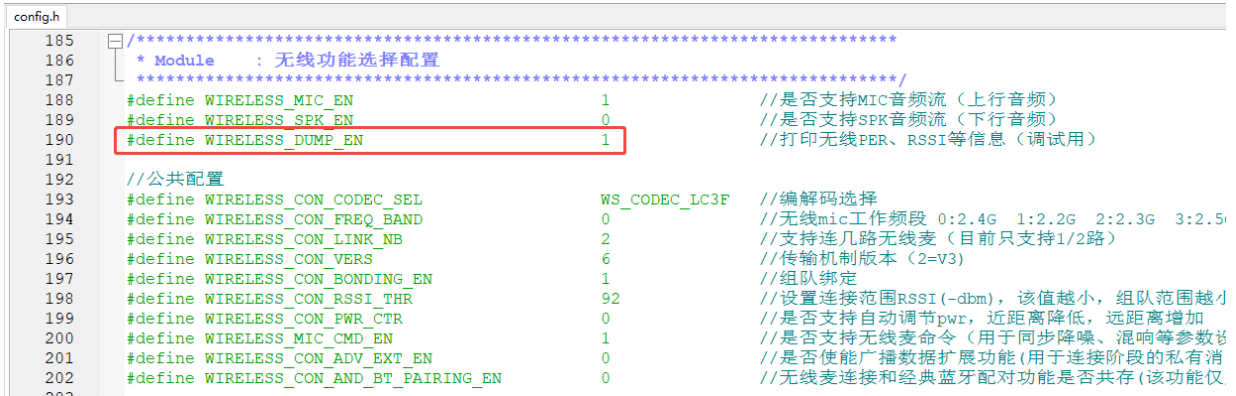


图 5.8.1.9 调试信息打印控制宏

例如接收端连上 2 个发射器，Link0 和 Link1 代表 2 个发射器的调试信息。PER 为错包率，total 为当前周期内收到数据包的总数量，err 为错包数量，rssi 为连接的发射器信号强度，CHMAP 为当前使用的信道。在 BLE 中有 40 个信道（0~39），这里打印了 5byte，每 1byte 有 8 个 bit，这里每个 bit 代表一个信道，若对应 bit 为 1 表示信道正在被使用，若对应 bit 为 0，则代表对应信道已经关闭。接收端正常工作时，会扫描周围的环境干扰情况，在有干扰的信道上，跳频算法会进行筛选和关闭。

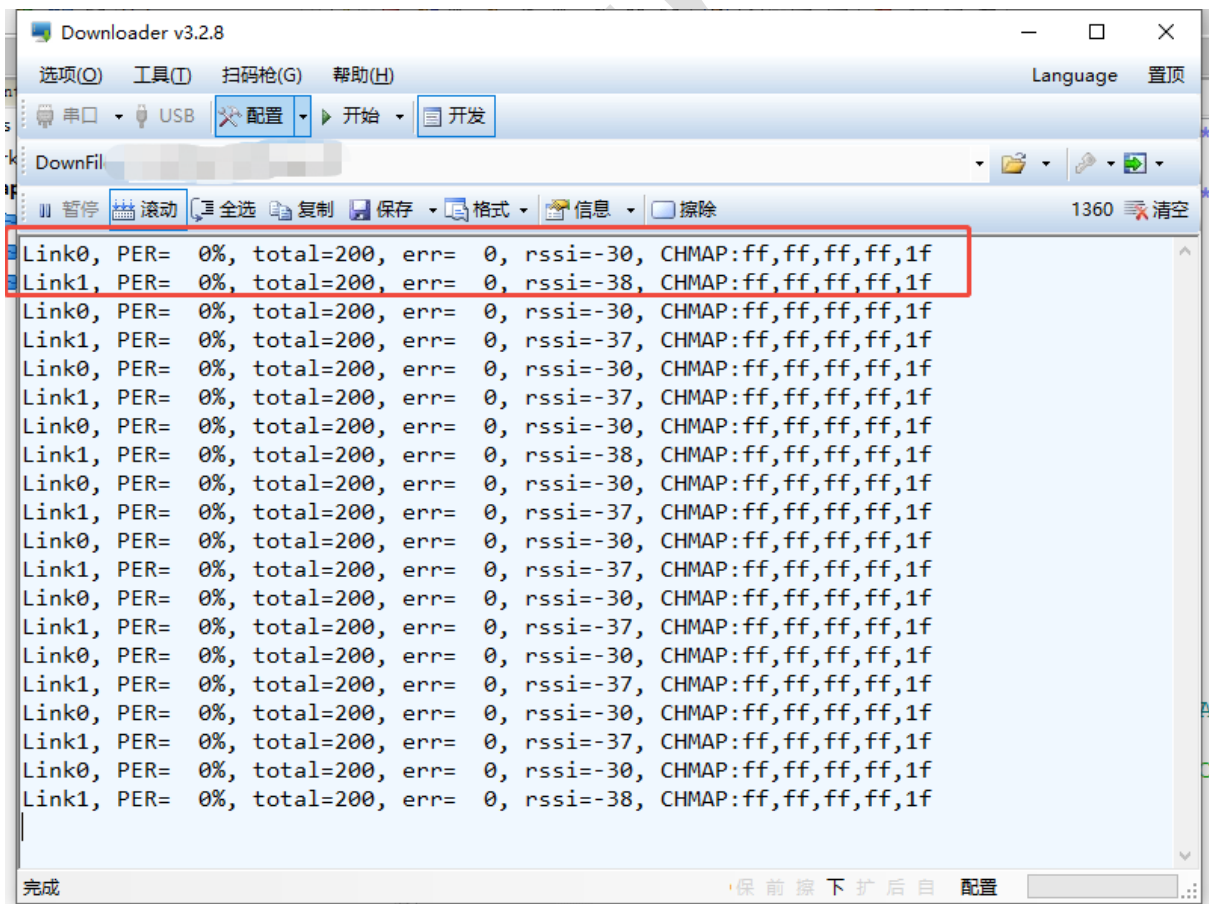


图 5.8.1.9 调试信息打印

5.8.2 蓝牙直连

在一拖二无线麦克风产品中，有 3 个角色，分别是 1 个接收端（adapter），2 个发射器（emit）。蓝牙直连功能，目前仅适用于发射器。当发射器处于（无线麦和经典蓝牙共存）配对状态时，可以被手机或者其他蓝牙设备连接。发射器一旦连上蓝牙设备，就会进行绑定，并记录为经典蓝牙模式，重新开机时，会恢复到这种模式，并回连绑定设备。当发射器处于上述的配对状态，并且没有连接蓝牙设备时，仍可以与接收端（adapter）建立连接 2.4G 连接，但是需要控制接收端（adapter）进入配对状态，此时它们会自动进行 2.4G 连接和绑定，当重新开机时，同样可恢复 2.4G 连接的绑定状态，并回连对应的设备。

当前发射器使用的蓝牙版本是 BT6.0，并可以往下兼容，与旧版本蓝牙设备进行连接。目前蓝牙的 profile 暂时仅支持通话功能，可用于录音通话，其他 profile 功能在开发中。

使用蓝牙直连功能时，需要打开 config.h 文件的 2 个配置。首先是使能 FUNC_BT_EN，如果 5.8.2.1 所示。

```
config.h port_key.c
13  * Module : Function选择相关配置
14  *****/
15  #define FUNC_MUSIC_EN 0 //是否打开MUSIC功能
16  #define FUNC_DEVICE_EN 1 //是否打开发射器功能（只做接收时关闭，节省空间）
17  #define FUNC_ADAPTER_EN 1 //是否打开接收器功能（只做发送时关闭，节省空间）
18  #define FUNC_BT_EN 1 //是否打开蓝牙功能
19  #define FUNC_BT_DUT_EN 1 //是否打开蓝牙的独立DUT测试模式
20  #define FUNC_BT_FCC_EN 0 //是否打开蓝牙的独立FCC测试模式（IODM已独立模式，不需要打开FCC模式）
21  #define FUNC_AUX_EN 0 //是否打开AUX功能
22  #define FUNC_USBDEV_EN 0 //是否打开USB_DEVICE功能
23  #define FUNC_SPEAKER_EN 0 //是否打开Speaker模式
24  #define FUNC_IDLE_EN 0 //是否打开IDLE功能
25
26
27  *****/
28  * Module : 系统功能选择配置
29  *****/
30  #define SYS_CLK_SEL SYS_24M //选择系统时钟
31  #define POWKEY_10S_RESET xcfig_cb.powkey_10s_reset
32  #define SOFT_POWER_ON_OFF 1 //是否使用软开关功能
33  #define PWRKEY_2_HW_PWRON 0 //用PWRKEY模拟硬开关
34  #define USB_SD_UPDATE_EN 0 //是否支持UDISK/SD的离线升级
35  #define GUI_SELECT GUI_NO //GUI Display Select
36  #define UART0_PRINTF_SRT PRINTF_VUSTR //选择UART打印信息输出到，或关闭打印信息输出
```

图 5.8.2.1 SDK 宏配置 1

其次是打开 config.h 文件里面的 WIRELESS_CON_AND_BT_PAIRING_EN 配置，如图 5.8.2.2 所示。

```
config.h port_key.c
193  * Module : 无线功能选择配置
194  *****/
195  #define WIRELESS_MIC_EN 1 //是否支持MIC音频流（上行音频）
196  #define WIRELESS_SPK_EN 0 //是否支持SPK音频流（下行音频）
197  #define WIRELESS_DUMP_EN 0 //打印无线PER、RSSI等信息（调试用）
198
199  //公共配置
200  #define WIRELESS_CON_CODEC_SEL WS_CODEC_LC3F //编解码选择
201  #define WIRELESS_CON_FREQ_BAND 0 //无线mic工作频段 0:2.4G 1:2.2G 2:2.3
202  #define WIRELESS_CON_LINK_NB 2 //支持连几路无线麦（目前只支持1/2路）
203  #define WIRELESS_CON_VERS 6 //传输机制版本（2=V3）
204  #define WIRELESS_CON_BONDING_EN 0 //组队绑定
205  #define WIRELESS_CON_RSSI_THR 92 //设置连接范围RSSI(-dbm)，该值越小，组
206  #define WIRELESS_CON_PWR_CTR 0 //是否支持自动调节pwr，近距离降低，远距
207  #define WIRELESS_MIC_CMD_EN 1 //是否支持无线麦命令（用于同步降噪、混
208  #define WIRELESS_CON_ADV_EXT_EN 0 //是否使能广播数据扩展功能（用于连接阶段
209  #define WIRELESS_CON_AND_BT_PAIRING_EN 1 //无线麦连接和经典蓝牙配对功能是否共存
210
211  //无线MIC音频配置（上行音频，DEVICE --> ADAPTER）
212  #define WIRELESS_MIC_SAMPLE_RATE_SELECT SAMPLE_RATE_48K //无线mic采样率选择
213  #if WIRELESS_CON_CODEC_SEL == WS_CODEC_LC3S
214  #define WIRELESS_MIC_SAMPLES_SELECT 120 //无线mic每帧采样点数选择，48khz
215  #define WIRELESS_MIC_CHANNEL_SELECT 1 //无线mic声道选择，1为单声道，2为双
216  #define WIRELESS_MIC_24B_PCM_EN 0 //无线mic 24bit PCM使能
217  #define WIRELESS_MIC_FRAME_SIZE 25 //每帧压缩后大小（FRAME_SIZE*LINK_N
218  #define WIRELESS_MIC_RETRY_NB 3 //重传次数（暂时支持1/2）
219  #define WIRELESS_MIC_ENC_MAX_US 750 //编码运算时长（单位us）
220  #define WIRELESS_MIC_DEC_MAX_US 700 //解码运算时长（单位us，开PLC时约70
221  #define WIRELESS_MIC_SINGLE_PLC_EN 1 //解码后独立的PLC使能
```

图 5.8.2.2 SDK 宏配置 2

在烧录配置文件(wireless_mic_emit.setting)上，需要打开发射器直连手机功能，完成上述配置，即可使用直连蓝牙功能。用户可修改发射器的蓝牙名称，默认为蓝牙名称为 BT897X-TWS。



图 5.8.2.2 SDK 宏配置

在打开 WIRELESS_CON_AND_BT_PAIRING_EN 宏配置的时候，SDK 默认携带配对 UI，需要配合按键使用。用户可以通过长按按键，控制发射器（emit）或者接收端（adapter）进入配对状态。UI 功能主要有以下几点：

- 通过长按按键，adapter/emit 均可清除配对，并重新进入配对状态。
- 对于发射器来说，进入配对状态时，可连接蓝牙设备，也可以连接接收端（adapter）。
- 若发射器清除配对，而接收端没有清除配对，两者不能建立连接，必须都清除配对，才可以建立连接。
- 发射器和接收端都具有连接记忆功能，已经连过的设备，重新上电时，会自动回连。

在我们的 demo 板上，一般会有 3 个按键（key1，key2，key3），通过不同的电阻，接到 pwrkey 网络上，pwrkey 内部自带上拉电阻，当按键按下时，通过采集 ADC 值，识别按键状态。

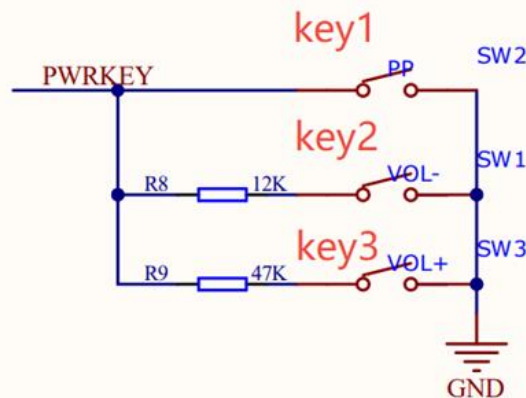


图 5.8.2.3 demo 板子按键

在 UI 程序上，我们定义了 key2/key3 按键，长按进入配对状态。即长按按键时，会触发一个 MSG_PAIRING，控制设备进入配对状态。若用户希望改变配对操作，例如使用双击进入配对状态，可将 port_key.c 文件中，将 MSG_PAIRING 按键消息放到双击位置。

在 port_key.c 文件中，我们可以看到有 5 个模式的按键消息，分别是 ADAPTER 模式、DEVICE 模式、BT 音乐模式、BT 通话模式、USBDEV 模式（暂未支持）。其中 ADAPTER 模式即为接收端使用的按键消息表，而 DEVICE、BT 音乐、BT 通话这 3 个模式通常为发射器使用，需要说明的是，目前我们暂不支持 BT 音乐服务，只是发射器在进入经典蓝牙状态下会借用该按键消息。例如发射器进入配对状态（FUNC_BT 模式）时，引用的是 BT 音乐的按键消息表，当发射器进入通话录音状态时，引用的是 BT 通话按键消息表，用户可根据实际情况进行修改。

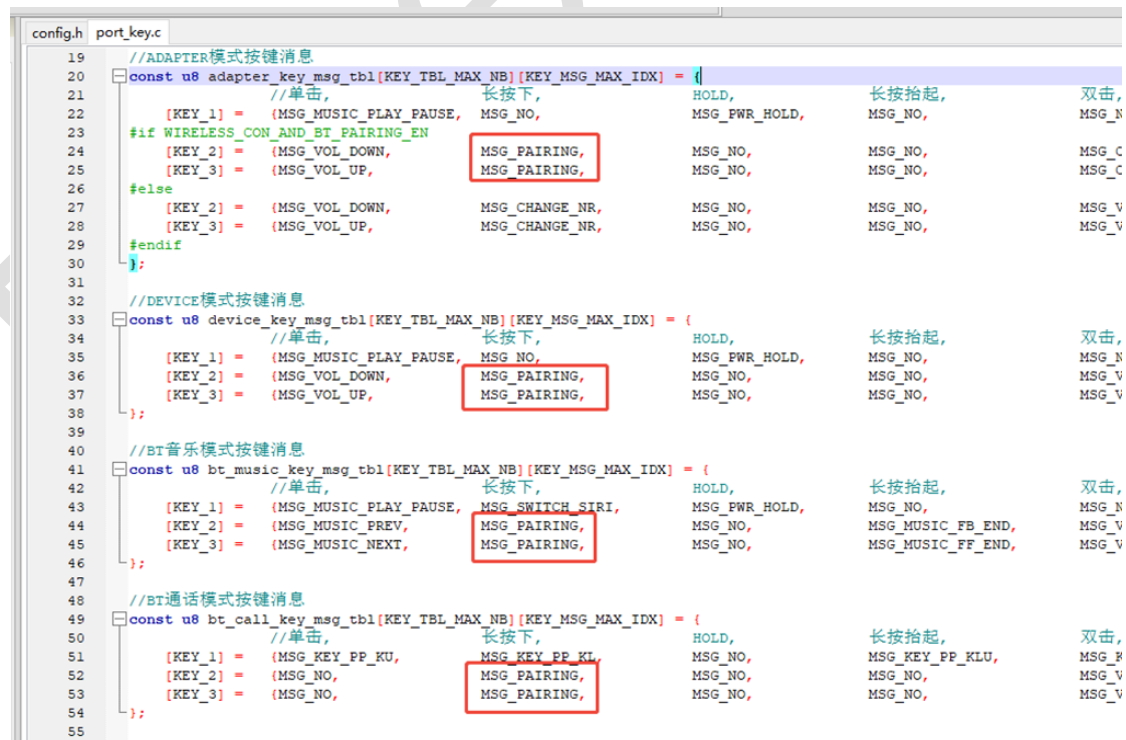


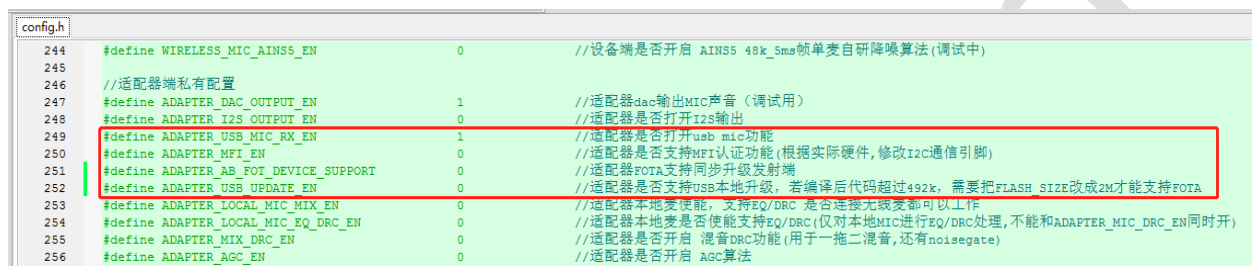
图 5.8.2.4 按键消息表配置

5.9 APP 及 OTA 相关

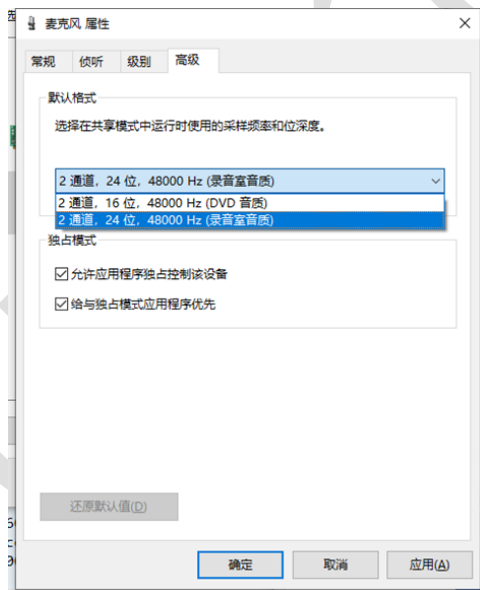
介绍 UAC 相关功能、参数；如何修改 USB 描述符；如何接入 MFI；配合我们的工具 Demo，介绍如何进行 OTA。

1. UAC 功能参数配置

USB Audio Class (UAC) 功能是通过 USB 进行音频数据传输功能，主要包含 USB 上行 MIC 数据和下行 SPK 数据传输，目前该 SDK 暂支持 USB 上行双声道 48K 16/24bit 数据传输。UAC 功能可以通过宏：ADAPTER_USB_MIC_RX_EN 进行控制，该控制宏默认打开。后续所有 USB 相关功能(MFI、USB OTA)的使用都需要在打开该宏。



16/24bit 数据切换可以由电脑/手机主机自动切换，默认首选项为双声道 48K 24bit，如下图所示。



若发射端选择 16bit MIC 采集，接收端主机选择双声道 24bit 模式，则 16bit 音频数据会在接收端扩位为 24bit 数据进行上传。切换 MIC 16/24bit 功能可在 config.h 文件中选择对应的无线编解码算法进行配置，配置选项如下图所示，默认是 24bit MIC 数据采集传输。需要注意的是，修改 MIC 采集数据位宽只能调整蓝牙编解码配置来调整。

```

config.h
190 #define WIRELESS_DUMP_EN 0 //打印无线PER、RSSI等信息(调试用)
191
192 //公共配置
193 #define WIRELESS_CON_CODEC_SEL WS_CODEC_LC3F //编解码选择 1、配置蓝牙编解码算法
194 #define WIRELESS_CON_FREQ_BAND 0 //无线mic工作频段 0:2.4G 1:2.2G 2:2.3G 3:2.5G
195 #define WIRELESS_CON_LINK_NB 2 //支持连几路无线麦(目前只支持1/2路)
196 #define WIRELESS_CON_VERS 6 //传输机制版本(2=V3)
197 #define WIRELESS_CON_BONDING_EN 0 //组队绑定
198 #define WIRELESS_CON_RSSI_THR 92 //设置连接范围RSSI(-dbm), 该值越小, 组队范围越小(产线建议设成70左右, 以防距离太远也能配)
199 #define WIRELESS_CON_PWR_CTR 0 //是否支持自动调节pwr, 近距离降低, 远距离增加
200 #define WIRELESS_CON_CMD_EN 1 //是否支持无线麦命令(用于同步降噪、混响等参数设置等)
201 #define WIRELESS_CON_ADV_EXT_EN 0 //是否使能广播数据扩展功能(用于连接阶段的私有消息传输)
202 #define WIRELESS_CON_AND_BT_PAIRING_EN 1 //无线麦连接和经典蓝牙配对功能是否共存(该功能仅用于DEVICE端, 谁先连上就进行相应的绑定, 并进入对应对应的连接模式)
203
204 //无线mic音频配置(上行音频, DEVICE --> ADAPTER)
205 #define WIRELESS_MIC_SAMPLE_RATE_SELECT SAMPLE_RATE_48K //无线mic采样率选择
206 #if WIRELESS_CON_CODEC_SEL == WS_CODEC_LC3S
207 #define WIRELESS_MIC_SAMPLES_SELECT 120 //无线mic每帧采样点数选择, 48khz 120/240
208 #define WIRELESS_MIC_CHANNEL_SELECT 1 //无线mic声道选择, 1为单声道, 2为双声道(目前只支持单声道)
209 #define WIRELESS_MIC_24B_PCM_EN 0 //无线mic 24bit PCM使能
210 #define WIRELESS_MIC_FRAME_SIZE 25 //每帧压缩后大小(FRAME_SIZE*LINK_NB*RETRY_NB暂时不能超过72byte)
211 #define WIRELESS_MIC_RETRY_NB 3 //重传次数(暂时支持1/2)
212 #define WIRELESS_MIC_ENC_MAX_US 750 //编码运算时长(单位us)
213 #define WIRELESS_MIC_DEC_MAX_US 700 //解码运算时长(单位us, 开Plc时约700us)
214 #define WIRELESS_MIC_SINGLE_PLG_EN 1 //解码后独立的PLC使能
215 #define WIRELESS_MIC_TX_INTERVAL 4 //传输周期=1.25ms(默认值是2, 改为4时传输次数可增加到5次)
216 #elif WIRELESS_CON_CODEC_SEL == WS_CODEC_LC3F
217 #define WIRELESS_MIC_SAMPLES_SELECT 240 //无线mic每帧采样点数选择, 48khz 120/240
218 #define WIRELESS_MIC_CHANNEL_SELECT 1 //无线mic声道选择, 1为单声道, 2为双声道(目前只支持单声道)
219 #define WIRELESS_MIC_24B_PCM_EN 1 //无线mic 24bit PCM使能 2、自动匹配对应的MIC采样位宽
220 #define WIRELESS_MIC_FRAME_SIZE 60 //每帧压缩后大小(FRAME_SIZE*LINK_NB*RETRY_NB暂时不能超过72byte)
221 #define WIRELESS_MIC_RETRY_NB 3 //重传次数(暂时支持1/2)
222 #define WIRELESS_MIC_ENC_MAX_US 1650 //编码运算时长(单位us, 内部plc约2700us, 无plc约1400us)
223 #define WIRELESS_MIC_DEC_MAX_US 1750 //解码运算时长(单位us, 内部plc约2000us, 无plc约1000us)
224 #define WIRELESS_MIC_SINGLE_PLG_EN 1 //解码后独立的PLC使能
225 #define WIRELESS_MIC_TX_INTERVAL 4 //传输周期=1.25ms(默认值是2, 改为4时传输次数可增加到5次)
226 #endif

```

2. USB 设备信息修改

使用 USB 相关功能的话需要配置相关产品的 USB 设备信息。修改 USB 设备信息是通过修改 USB 描述符实现的, 该 SDK 为方便客户修改相关 USB 设备信息, 将一些常用的 USB 设备信息用宏拉出来进行控制如下图所示, 客户可以根据 USB 设备配置宏进行修改 USB 信息, 需要注意修改的 USB 信息不能超过限制最大长度。

```

config.h
282
283
284 /*****
285  * Module : usb device 功能选择
286  *****/
287 #define UDE_STORAGE_EN 0
288 #define UDE_SPEAKER_EN 1
289 #define UDE_HID_EN 1
290 #define UDE_MIC_EN 1
291
292 #define UDE_IAP_EN ADAPTER_MFI_EN //与UDE_STORAGE_EN只能2选1
293 #define UDE_IAP_PROCESS_EN ADAPTER_MFI_EN //是否使用底层iAP协议发送
294 #define SOFT_I2C_EN ADAPTER_MFI_EN //IAP使用的软件iic协议接口
295
296 #define UDE_PROD_NAME "USB Microphone" //最大36个字符
297 #define UDE_SUPPLIER "Generic" //最大18个字符
298 #define UDE_SERIAL_NB "20250201905926" //最大16个字符
299 #define UDE_MODEL "BT893X"
300 #define UDE_MANUFACTURE "Shenzhen Bluetrum" //最大28个字符
301 #define UDE_PID 0x0124 //最大4位
302 #define UDE_VID 0x0C21 //最大4位
303
304 #define USB_DET_VER_SEL 0 //USB插入检测方式, 0-旧方式, 1-新方式

```

USB设备信息

3. 苹果认证接口(MFI)接入

USB 设备如果需要与苹果手机进行取电、USB HID 通信等功能则需要通过 MFI 认证。MFI 认证功能控制宏由 ADAPTER_MFI_EN 进行控制。MFI 认证需要通过外部芯片(例如芯片 LT MFI1343SS00177-L)获取苹果加密信息。与外部加密信息芯片通信一般通过 I2C 进行, 因此在配置 MFI 认证功能的同时还需要配置与外部加密芯片通信的 I2C 引脚, 如下图所示。


```

config.h
531  /* *****
532  * Module      : I2C配置
533  * *****
534  #define I2C_HW_EN                0           //是否使能硬件I2C功能
535  #define I2C_MAPPING              I2CMAP_PB2PB1 //选择I2C mapping
536
537  #define I2C_SW_EN                (0 || SOFT_I2C_EN) //是否使能软件I2C功能
538  #define I2C_MUX_SD_EN            0           //是否I2C复用SD卡的IO
539
540  #if I2C_MUX_SD_EN
541  #define I2C_SCL_IN()              SD_CMD_DIR_IN()
542  #define I2C_SCL_OUT()             SD_CMD_DIR_OUT()
543  #define I2C_SCL_H()               SD_CMD_OUT_H()
544  #define I2C_SCL_L()               SD_CMD_OUT_L()
545  #define I2C_SDA_IN()              SD_DAT_DIR_IN()
546  #define I2C_SDA_OUT()             SD_DAT_DIR_OUT()
547  #define I2C_SDA_H()               SD_DAT_OUT_H()
548  #define I2C_SDA_L()               SD_DAT_OUT_L()
549  #define I2C_SDA_IS_H()            SD_DAT_STA()
550  #else
551  #define I2C_SCL_IN()               GPIOB_DIR |= BIT(1);
552  #define I2C_SCL_OUT()              {GPIOB_DIR &= ~BIT(1); GPIOB_DE |= BIT(1);}
553  #define I2C_SCL_H()                GPIOB_SET = BIT(1);
554  #define I2C_SCL_L()                GPIOB_CLR = BIT(1);
555  #define I2C_SDA_IN()               {GPIOB_DIR |= BIT(2); GPIOB_PU |= BIT(2);}
556  #define I2C_SDA_OUT()              {GPIOB_DIR &= ~BIT(2); GPIOB_DE |= BIT(2);}
557  #define I2C_SDA_H()                GPIOB_SET = BIT(2);
558  #define I2C_SDA_L()                GPIOB_CLR = BIT(2);
559  #define I2C_SDA_IS_H()              (GPIOB & BIT(2))
560  #endif // I2C_MUX_SD_EN
561
562  #define I2C_SDA_SCL_OUT()           {I2C_SDA_OUT(); I2C_SCL_OUT();}
563  #define I2C_SDA_SCL_H()            {I2C_SDA_H(); I2C_SCL_H();}
564
565  /* *****

```

由于该 SDK 的 MFI 认证是基于 USB 设备的，所以 MFI 认证的身份信息是与 USB 保持一致的，可以通过上述 USB 设备信息进行同步修改。

4. USB OTA 功能

该 SDK 可以通过 USB 和无线麦 2.4G 连接进行 OTA 升级，通过 USB HID 接口从上位机获取升级数据，然后通过无线麦 2.4G 转发发射端的升级数据进行升级。

USB 升级功能分别为接收端本地 USB 升级、发射端 2.4G 无线连接升级，升级发射端的时候需要保证发射端与接收端无线连接的稳定性。

如下图所示，打开 ADAPTER_USB_UPDATE_EN 控制宏接收端可以通过 USB 接收升级数据进行本地升级，同时打开 ADAPTER_USB_UPDATE_EN 和 ADAPTER_AB_FOT_DEVICE_SUPPORT 控制宏既可以升级本地端也可以升级已连接的发射端

```
config.h
228 //设备端私有配置
229 #define DEVICE_DAC_OUTPUT_EN 1 //设备端是否使能DAC输出 (调试用)
230 #define DEVICE_I2S_INPUT_EN 0 //设备端I2S输出功能
231 #define DEVICE_MIC_FADE_IN_EN 1 //设备端MIC是否使用软件淡入功能(避免外界声音比较大时,连上无线麦瞬间,声音比较突兀)
232
233 #define WIRELESS_MIC_EQ_DRC_EN 1
234 #define WIRELESS_MIC_EQ_DRC_DELAY WIRELESS_MIC_EQ_DRC_EN*323//EQ 运算时间
235 #define WIRELESS_MIC_ECHO_EN 0 //设备端是否开启 ECHO音效
236 #define WIRELESS_MIC_ECHO_DELAY (WIRELESS_MIC_ECHO_EN*(1+WIRELESS_MIC_24B_PCM_EN)*120)//ECHO运算时间
237 #define WIRELESS_MIC_MAGIC_EN 0 //设备端是否开启 MAGIC音效
238 #define WIRELESS_MIC_MAGIC_DELAY WIRELESS_MIC_MAGIC_EN*220//MAGIC运算时间
239 #define WIRELESS_MIC_AGC_EN 0 //设备端是否开启 AGC算法
240 #define WIRELESS_MIC_AGC_DELAY WIRELESS_MIC_AGC_EN*0//AGC 运算时间
241 #define WIRELESS_MIC_ROOM_REVERB_EN 0 //设备端是否开启 房间混响
242 #define WIRELESS_MIC_ROOM_REVERB_DELAY WIRELESS_MIC_ROOM_REVERB_EN*355//房间混响运算时间
243 #define WIRELESS_MIC_GTCRN_48K_QMF_EN 1 //设备端是否开启 AI DNN大模型自研降噪算法
244 #define WIRELESS_MIC_AINSS_EN 0 //设备端是否开启 AINSS 48k_5ms帧单麦自研降噪算法(调试中)
245
246 //适配器私有配置
247 #define ADAPTER_DAC_OUTPUT_EN 1 //适配器dac输出MIC声音 (调试用)
248 #define ADAPTER_I2S_OUTPUT_EN 0 //适配器是否打开I2S输出
249 #define ADAPTER_USB_MIC_RX_EN 1 //适配器是否打开usb mic功能
250 #define ADAPTER_MFI_EN 0 //适配器是否支持MFI认证功能(根据实际硬件,修改I2C通信引脚)
251 #define ADAPTER_AB_FOT_DEVICE_SUPPORT 0 //适配器FOTA支持同步升级发射端
252 #define ADAPTER_USB_UPDATE_EN 0 //适配器是否支持USB本地升级, 若编译后代码超过492k, 需要把FLASH_SIZE改成2M才能支持FOTA
253 #define ADAPTER_LOCAL_MIC_MIX_EN 0 //适配器本地麦使能, 支持EQ/DRC 是否连接无线麦都可以工作
254 #define ADAPTER_LOCAL_MIC_EQ_DRC_EN 0 //适配器本地麦是否使能支持EQ/DRC (仅对本地MIC进行EQ/DRC处理, 不能和ADAPTER_MIC_DRC_EN同时开)
255 #define ADAPTER_MIX_DRC_EN 0 //适配器是否开启 混音DRC功能(用于一拖二混音, 还有noisegate)
256 #define ADAPTER_AGC_EN 0 //适配器是否开启 AGC算法
257 #define ADAPTER_ECHO_EN 0 //适配器是否开启 混响
258 #define ADAPTER_MAGIC_EN 0 //适配器是否开启 魔音
259 #define ADAPTER_FREQ_SHIFT_EN 0 //适配器是否开启 移频
260 #define ADAPTER_ROOM_REVERB_EN 0 //适配器是否开启 房间混响
261 #define ADAPTER_GTCRN_48K_QMF_EN 0 //适配器是否开启 AI DNN大模型自研降噪算法
262 #define ADAPTER_AINSS_EN 0 //适配器是否开启 AINSS 48k_5ms帧单麦自研降噪算法
263
264
```

需要注意的是打开升级功能宏之后需要检查编译的代码大小, 如果编译后的代码大小超过 $FLASH_SIZE/2 - 24K$ 则会导致升级失败。以下图为例, config.h 配置的是 1M Flash, 而编译的代码为 544K > 488K (1M/2 - 24K), 则会导致升级失败, 需要调整代码空间或者更换更大 flash 空间版本的芯片, 修改配置为 FSIZE_2M。

```
config.h
30 #define SYS_CLK_SEL SYS_24M //选择系统时钟
31 #define POWKEY_10S_RESET xcfg_cb.powkey_10s_reset
32 #define SOFT_POWER_ON_OFF 1 //是否使用软开关机功能
33 #define PWRKEY_2_HW_FWRON 0 //用PWRKEY模拟硬开关
34 #define USB_SD_UPDATE_EN 0 //是否支持UDISK/SD的高线升级
35 #define GUI_SELECT GUI_NO //GUI Display Select
36 #define UART0_PRINTF_SEL PRINTF_VUSB //选择UART打印信息输出Io, 或关闭打印信息输出
37 #define FWRON_ENTIER_BTMODE_EN 0 //是否上电默认进蓝牙模式
38 #define SLEEP_DAC_OFF_EN (is_sleep_dac_off_enable()) //sfunc sleep 是否关闭DAC, 复用MIC检测方案不能关DAC。
39 #define SYS_VDDIO_LP_EN 1 //休眠模式是否打开切换VDDIO功能(省电, 可能会影响VDDIO供电的外设)
40
41 #define OFFLINE_LOG_EN 0 //是否打开离线日志(打开之前要评估flash空间是否足够)
42 #define OFFLINE_LOG_SIZE 0x30000 //离线日志存储区域大小
43 #define OFFLINE_LOG_START (FLASH_SIZE - CM_SIZE - OFFLINE_LOG_SIZE) //离线日志存储区域起始位置(注意避让FOTA、自定义KEEP等其他块)
44
45 /* Module : 电源选择配置
46 ***** */
47 #define BUCK_MODE_EN xcfg_cb.buck_mode_en //是否BUCK MODE
48 #define VDDIO_LIMIT_SEL 2 //vddio电流限制, 0=100mA, 1=200mA, 2=300mA, 3=400mA, 4~7=no_limit
49 #define VDDBT_LIMIT_SEL 2 //vddbt电流限制, 0=109mA, 1=194mA, 2=279mA, 3=366mA, 4~7=no_limit
50 #define VDDCORE_LIMIT_SEL 2 //vddcore电流限制, 0=109mA, 1=194mA, 2=279mA, 3=366mA, 4~7=no_limit
51
52 /* Module : SPIFLASH配置
53 ***** */
54 #define FLASH_SIZE FSIZE_1M //LQFP48芯片内置1MB, 其它封装芯片内置512KB(实际导出prd文件要小于492K)
55 #define FLASH_CODE_SIZE 768K //程序使用空间大小, code_size <= flash_size - [params(20k) + res2 + res.bin]
56 #define FLASH_RESERVE_SIZE 0x5000 //程序保留空间大小, 用作系统参数区params(20k)
57 #define FLASH_RES2_SIZE 0x0000 //参数保留空间大小, 用作在线调参区res2(打开ANC_EQ_RES2_EN需要保留4K)
58 #define FLASH_ERASE_4K 1 //是否支持4K擦除
59 #define FLASH_DUAL_READ 1 //是否支持2线模式
60 #define FLASH_QUAD_READ 0 //是否支持4线模式
61 #define FLASH_SPEED_UP_EN 1 //SPI FLASH提速。
62
63 /* Module : 蓝牙功能配置
64 ***** */
65
66
67
```

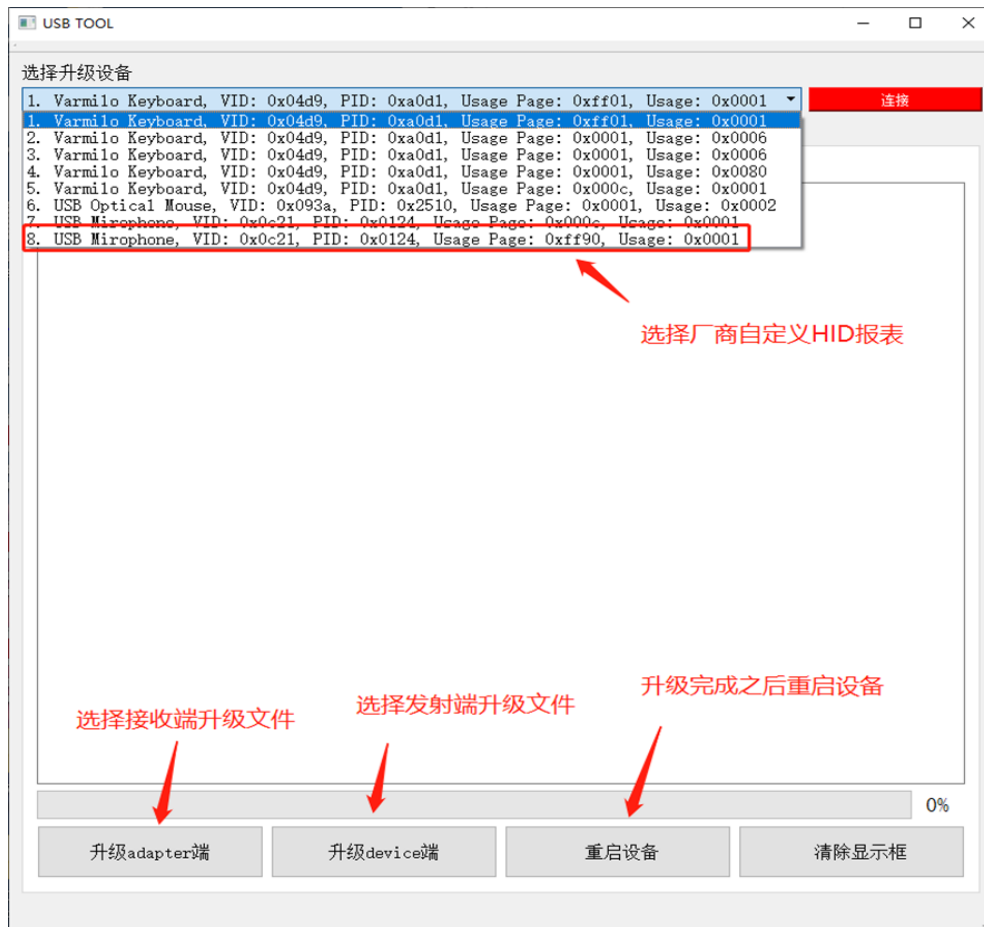
Logs & others

CodeBlocks Search results Build log Build messages Debugger

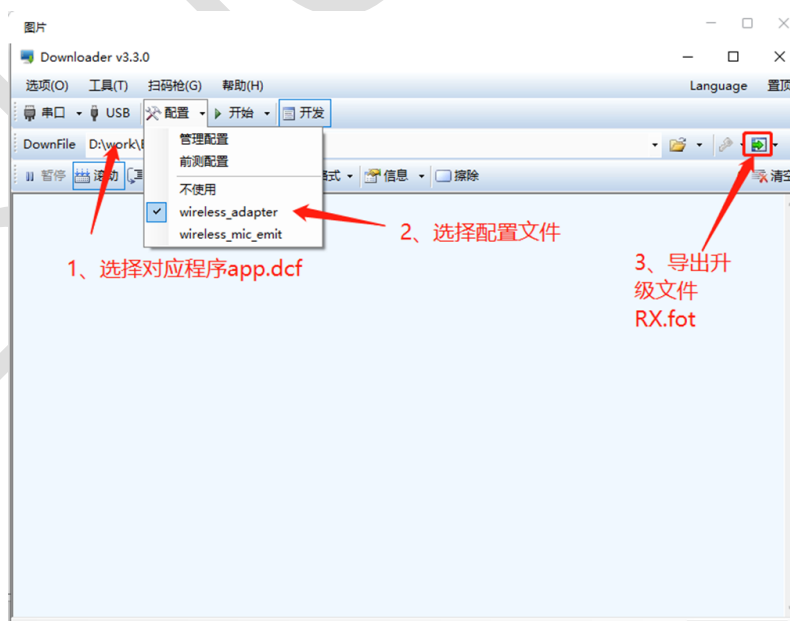
```
C:\Users\Public\Desktop\Work\le_mic\bt8970\trunk\app\projects\microphone\Output\bin\riscv32-elf-xxmaker -b appxxm.o || goto err
CODE SIZE: 544 KB
save file app.dcf successful
C:\Users\Public\Desktop\Work\le_mic\bt8970\trunk\app\projects\microphone\Output\bin\if exist C:\upload\upload.bat (call C:\upload\upload.bat -D BT8970 app.dcf)
C:\Users\Public\Desktop\Work\le_mic\bt8970\trunk\app\projects\microphone\Output\bin\echo off
OK: Upload
Process terminated with status 0 (0 minute(s), 10 second(s))
0 error(s), 0 warning(s), 10 second(s))
```

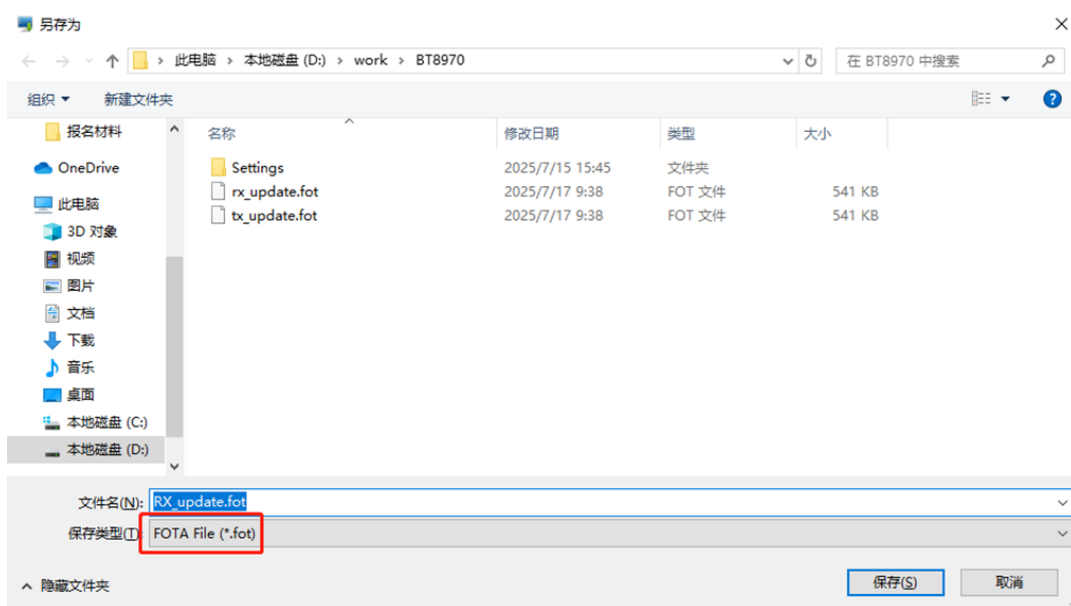
USB 升级上位机如下图所示, 在接收端 USB 接入 PC 之后会出现 0xff90 的枚举信息, 该配置是厂商自

定义的 HID 报表接口，报告 ID 为 0x03。连接之后选择升级 adapter/device，加载 adapter/device 升级文件后就会自动升级。升级完成之后，需要手动点击重启设备选项下发设备重启命令重启设备。



通过 Downloader 工具可以生成 FOT 升级文件，获取步骤如下图所示





该上位机是基于 PC 端的，如果需要在手机端客户自定义 APP 加入升级功能可以参考以下 USB HID 升级协议。USB HID 升级协议如下所示：

1、HID 数据交互格式，数据总长度最多为 65byte

report ID(1byte)	cmd(1byte)	data len(1byte)	data
------------------	------------	-----------------	------

2、HID 命令类型

update_req	0x91	升级请求，用于上位机发给下位机
paket_req	0x92	数据包请求，用于下位机向上位机请求数据
paket_rsp	0x93	数据包响应，上位机发给下位机，携带升级数据
update_sta	0x94	升级状态下位机发给上位机，用于提示升级成功、失败
dev_reset	0x93	上位机发给下位机，复位下位机

这里简单介绍部分命令，便于客户自行开发上位机

update_req(0x91)

类型	数据大小	备注
report ID	1byte	报告 ID
cmd	1byte	命令
payload	1byte	数据长度
file_size	4byte	文件大小
file_hash	4byte	文件哈希值
update_obj	1byte	升级对象，0 是 adapter，1 是 device

paket_req(0x92)

类型	数据大小	备注
report ID	1byte	报告 ID
cmd	1byte	命令
payload	1byte	数据长度
addr	4byte	文件偏移地址请求
paket_size	4byte	上位机发送的数据大小

paket_rsp(0x93)

数据包响应，上位机发给下位机，一般是下位机一次请求 512byte，但是 HID 一次传输不了那么多，只能分多次传

类型	数据大小	备注
report ID	1byte	报告 ID
cmd	1byte	命令
payload	1byte	数据长度
paket_size	4byte	文件大小

update_sta(0x94)

类型	数据大小	备注
report ID	1byte	报告 ID
cmd	1byte	命令
payload	1byte	数据长度
update_obj	1byte	当前升级对象,0:adapter,1:device
sta	1byte	升级状态,0:表示成功,1 表示失败

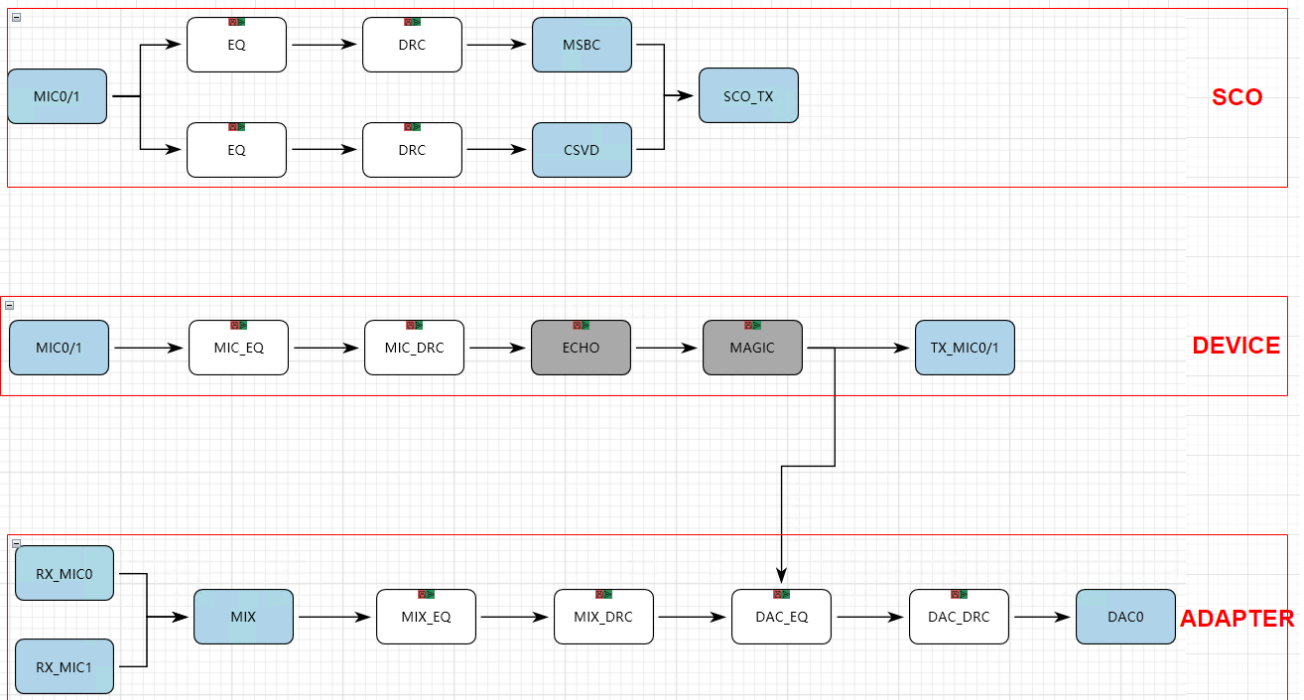
dev_reset(0x95)

类型	数据大小	备注
report ID	1byte	报告 ID
cmd	1byte	命令
payload	1byte	数据长度:0

6 算法相关

6.1 音效算法

BT897x 无线麦方案支持 EQ、DRC、ECHO 等音效算法处理，本章节主要分别介绍每个模块处理使用方法调试指南，SDK 默认支持的调试模块及链路如图，调试过程需要使用到工具 BlueTrumToolKit.exe，关于工具的使用和安装可参考文档《音频调试手册》以及文档《音效工具使用指南》。



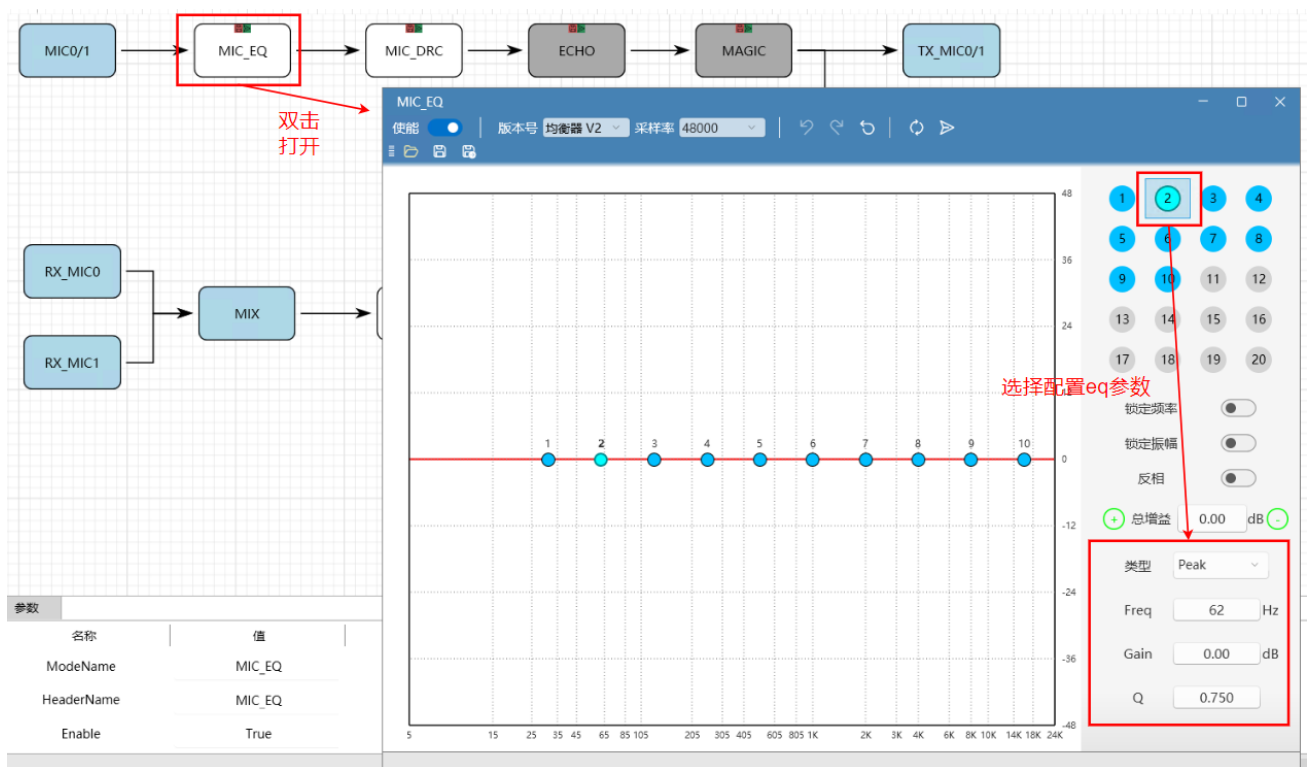
6.1.1 EQ (PACC)

EQ 是 equalizer 均衡器的缩写。它的基本作用是通过对声音某一个或多个频段进行增益或衰减，达到调整音色的目的。通常包括参数如参数表，资源和算力如资源算力表。

参数表	
参数名称	功能作用
频率 (Frequency)	这是用于设定你要进行调整的频率点用的参数
增益 (Gain)	用于调整在你设定好的频率 (Frequency) 值上进行增益或衰减的参数
量化 (Quantize)	用于设定你要进行增益或衰减的频段“宽度”

资源算力表	
代码以及常量 (text+rodata)	0 bytes
缓存(buf)	0 bytes
算力	约 323 us (xx samples + 897 160M);

897x, 用户修改 EQ 参数, 首先使用工具 BlueTrumToolKit.exe 打开目录 SDK\app\projects\microphone\Output\bin 下 effect.bts/effect.blsn 文件, 然后打开后双击打开 xxx_EQ 相关类型的模块即可进行对对应链路的 eq 调试, 如图。



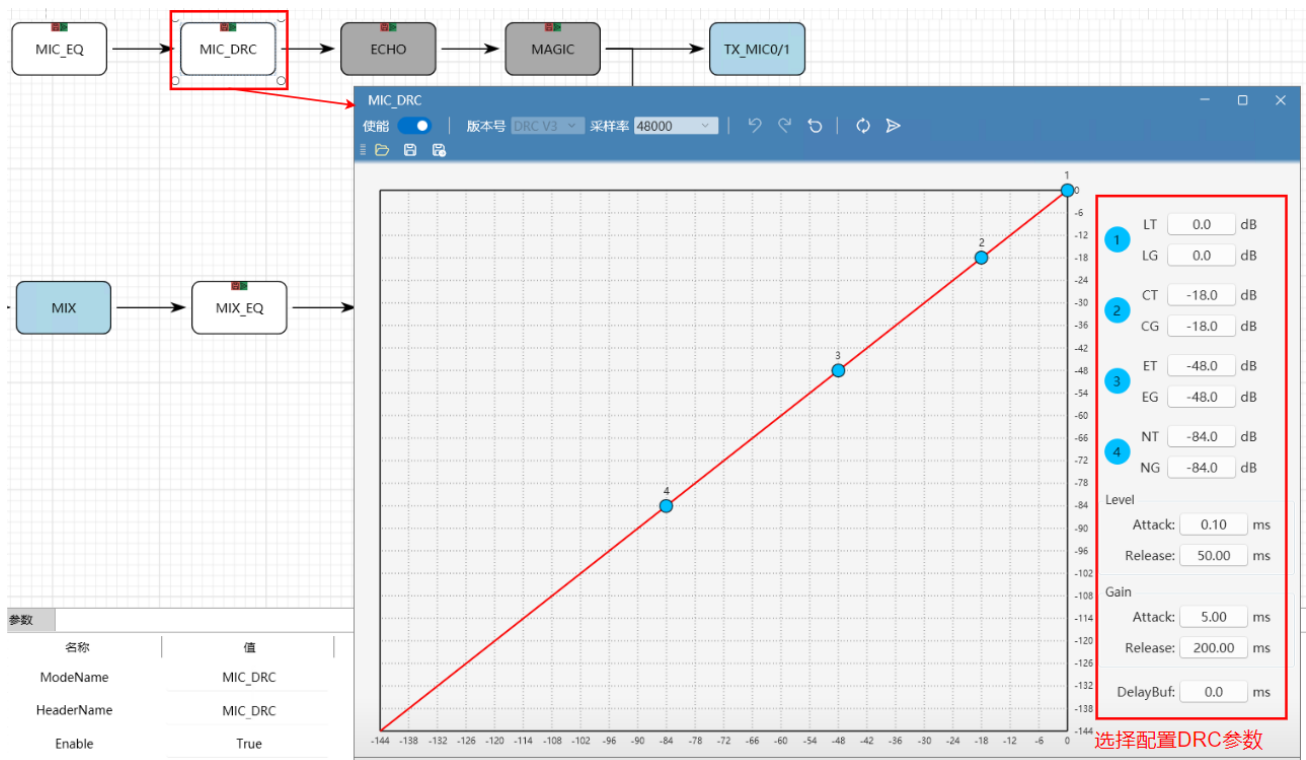
6.1.2 DRC (PACC)

DRC 是 dynamic range control 的缩写。它的基本作用是动态调整音频输出的幅值，包括压缩器 (Compressor)、限制器 (Limiter)、扩展器 (Expander) 和噪声门 (Noise Gate)。通常包括参数如参数表，资源和算力如资源算力表。

参数表	
参数名称	功能作用
限制器 (Limiter)	减小音频信号的动态范围，降低高电平部分，提升低电平部分
压缩器 (Compressor)	减小音频信号的动态范围，降低高电平部分，提升低电平部分
扩展器 (Expander)	增加动态范围，对低于阈值的信号进一步降低
噪声门 (Noise Gate)	完全切除低于阈值的信号
启动时间 (Gain Attack)	压缩器、限制器、扩展器和噪声门开始作用的时间
释放时间 (Gain Release)	压缩器、限制器、扩展器和噪声门停止作用的时间
启动时间 (Level Attack)	输入信号的 peak 或 rms 的计算作用时间
释放时间 (level release)	输入信号的 peak 或 rms 的计算释放时间

资源算力表	
代码以及常量 (text+rodata)	0 bytes
缓存(buf)	0 bytes
算力	约 323 us ;

897x, 用户修改 DRC 参数, 首先使用工具 BlueTrumToolKit.exe 打开目录 SDK\app\projects\microphone\Output\bin 下 effect.bts/effect.blsn 文件, 然后打开后双击打开 xxx_DRC 相关类型的模块即可进行对对应链路的 DRC 调试, 如图。



6.1.3 ECHO

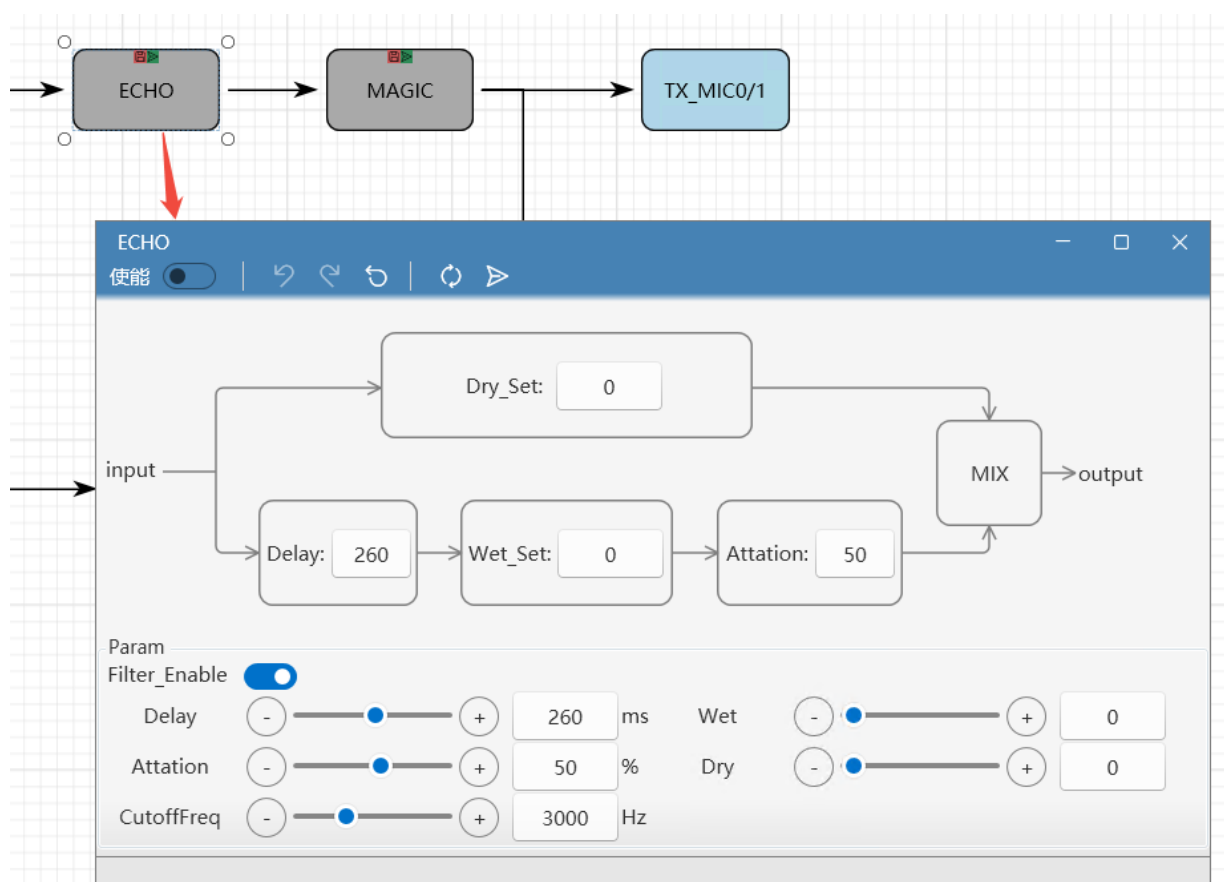
Echo（回声）是音频处理中最经典的效果之一，通过模拟声音在物理空间中的反射特性来创造空间感和深度感。通常包括参数如参数表，资源和算力如资源算力表。

参数表	
参数名称	功能作用
延迟时间（Delay）	50~400（ms），控制回声间隔的时间
衰减度（Attenuation）	0-90(%)，控制回声的衰减比例
干度（Dry）	0-32767，控制原声人声的大小
湿度（Wet）	0-32767，控制叠加回声的大小

资源算力表	
代码以及常量（text+rodata）	待测 1k bytes
缓存(buf)	待测 20 kbytes
算力	约 300 us (240samples + 897x160M)；

897x，用户修改 ECHO 参数，首先使用工具 BlueTrumToolKit.exe 打开目录

SDK\app\projects\microphone\Output\bin 下 effect.bts/effect.blsn 文件，然后打开后双击打开 ECHO 模块即可进行调试，如图。



6.2 通话算法

7 产测相关

sdk 支持 vusb 唤醒 PB3 进行 huart 通讯的串口产测模式，拿到 sdk 只需要打开 BSP_TBOX_TEST_EN 宏即可

```

/*****
 * Module      : 系统功能选择配置
 *****/
/*****
#define SYS_CLK_SEL          SYS_24M          //选择系统时钟
#define POWKEY_10S_RESET    xcfg_cb.powkey_10s_reset
#define SOFT_POWER_ON_OFF    1                //是否使用软开机功能
#define PWRKEY_2_HW_PWRON    0                //用PWRKEY模拟硬开关
#define USB_SD_UPDATE_EN     0                //是否支持UDISK/SD的离线升级
#define GUI_SELECT          GUI_NO           //GUI Display Select
#define UART0_PRINTF_SEL     PRINTF_VUSB      //选择UART打印信息输出IO，或关闭打印信息输出
#define PWRON_ENTER_BTMODE_EN 0              //是否上电默认进蓝牙模式
#define SLEEP_DAC_OFF_EN     (is_sleep_dac_off_enable()) //sfunc_sleep是否关闭DAC，复用MICL检测方案不能关DAC。
#define SYS_VDDIO_LP_EN      1                //休眠模式是否打开切换VDDIO功能（省电，可能会影响VDDIO供电的外设）
#define BSP_TBOX_TEST_EN     1                //是否使能有无线产测模式，通过PB3做串口，用于dut测试指令

```

图 7.1 有线产测宏开关

产测时目标样机的具体参数可以通过看 downloader 打印信息获得，xml 文件和 sdk 上的连接信息都可以进行修改，需要注意 xml 文件参数区与目标样机的连接参数是否一致，只有一致的状态下才能正常连接 (DISCON_AUTO_PWROFF 除外，为可选项)

```

(20:58:51.623) dac ft:1 0
res_play: 23(1),7 --sta=0, en=1, brk=0
res_add(M): 23(1)7
res_item: 23, 1, 3
lock [mp3dec] ok, addr=1001a200, size=3000
(20:58:52.838) unlock [mp3dec] ok
(20:58:52.897) <TX_INTERVAL>          4
<CON_INTERVAL>          120
<WIRELESS_FEAT>          18050
<WIRELESS_CODEC>         5
<FREQ_BAND>              0
<RETRY>                  3
<DISCON_AUTO_PWROFF>     0
<CONFIG_RSSI>            92
<WS_NAME>                LE-MIC2
func_run
====>func0: 0->1
func_adapter
load wl_com: vma=00048800, lma=10008050, size=00005000
load wl_adapter: vma=00030000, lma=1000d050, size=00003c00
lock [wireless] ok, addr=10024200, size=2800
(20:58:52.959) feat = 0x4682, ext_feat = 0x0, codec[0] = 0x5, codec[1] = 0x0

```

图 7.2 无线连接参数

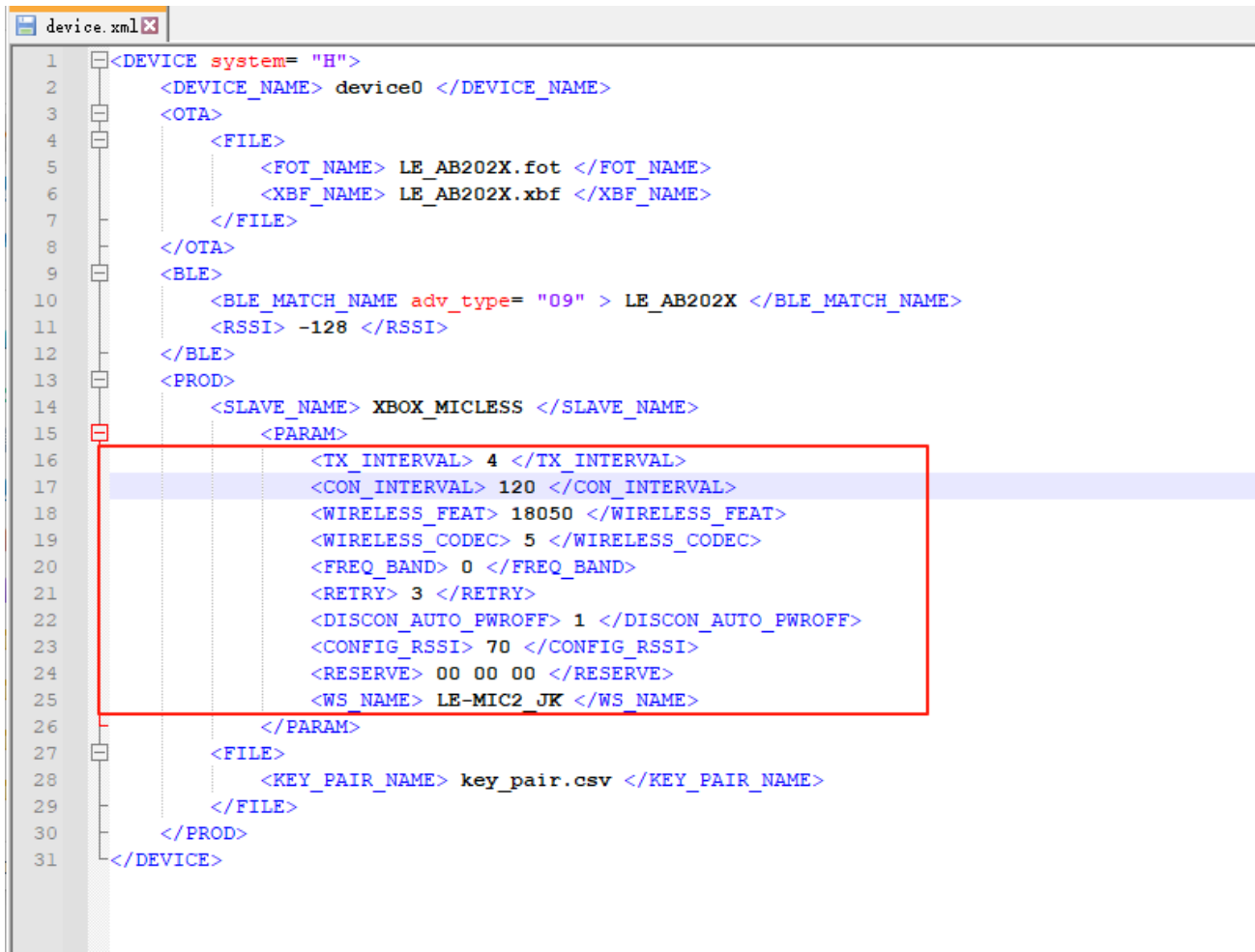


图 7.3 xml 修改参数部分

具体更多的产测操作流程可以参考 wiki 资料上《中科蓝讯蓝牙测试盒二代使用说明书》（较频偏、升级和产测）的 2.11 章节

8 认证相关

8.1 BQB 认证

凡带有蓝牙功能、在外观上标明蓝牙 LOGO 或 BLUETOOTH 字样的产品必须通过蓝牙 BQB（Bluetooth Qualification Body）认证，否则该产品就会被蓝牙技术联盟(Bluetooth Special Interest Group, 简称 SIG) 视为侵权。只有通过蓝牙 SIG 组织认证的产品，才能保证产品的兼容性和相关性能是符合蓝牙标准的，才能合法地使用蓝牙技术。

BQB 认证需要烧录独立软件去过测试，详细的参考资料在 wiki 上的认证相关资料上，参考 BQB 认证篇

2.4G 传输，不走标准流程，如果不贴蓝牙标的话可以不用过 BQB 认证；

8.2 FCC 认证

FCC 全称为美国联邦通讯委员会，通过控制无线电广播、电视、电信、卫星和电缆来协调国内和国际的通信，负责授权和管理除联邦政府使用之外的射频传输装置和装备。根据美国联邦通讯法规相关部分中规定，凡进入美国的电子类产品（主要针对工作频率在 9KHz-3000GHz 的电子产品）都需要进行电磁兼容认证。

FCC 认证需要烧录独立软件去过测试，详细的参考资料在 wiki 上的认证相关资料上，参考 FCC 认证篇

Bluetrum

修 订 历 史

修订日期	版本号	修订记录	作者
20250805	V1.0.0	创建初版本	蓝讯开发工程

Bluetrum

声 明

本文档是中科蓝讯的原创作品和受版权保护的财产。全部或部分复制使用或传播必须事先获得中科蓝讯的书面批准，并经版权所有人明确确认。中科蓝讯有权随时根据法律、法规的变化以及公司经营策略的调整等修改本文档。修改后的文档将会通过适当的方式将进行公示。如您在本文档修订后仍继续使用本文档内容的，则视为您接受本文档的修订。

请您通过各种方式关注中科蓝讯发布的信息，包括中科蓝讯的官方网站、官方公众号等。中科蓝讯对不当使用本文档的后果不承担任何责任，中科蓝讯提供的信息仅作为参考或典型应用。中科蓝讯保留更改电路设计的权利和/或规格的权利，无需另行事先通知。

您不得因用途原因侵犯第三方的专利或其他权利，否则应自行承担相应责任。实施解决方案/产品可能需要第三方许可证，您应全权负责获取所有适当要求的第三方许可证；中科蓝讯不负任何所需第三方许可证的任何许可费或版税。

如果您需要了解进一步的业务和技术支持，请发邮箱至：sales@bluetrum.com/project@bluetrum.com